Lark Builds Itself

Self-Hosting, Optimization, and Proof

Set Lonnert
with AI Coöperation

Code Crafting

A companion to Code Crafting no. 4,
The Language Stack

Lark Builds Itself: Self-Hosting, Optimization, and Proof

Set Lonnert
with AI Coöperation

Cover illustration in: Rudbeck, Olof the Younger. c. 1693–1710. The Book of Birds (also known as Entomologia et Ornithologia). Watercolour manuscript. Uppsala University Library. *An illustration of the woodlark.*
http://urn.kb.se/resolve?urn=urn:nbn:se:alvin:portal:record-103375
The work is marked with Public Domain Mark 1.0 Universal

A companion volume to *Code Crafting no. 4: The Language Stack — From Silicon to Semantics* (BoD, ISBN 978-91-8134-438-7).

Author: Set Lonnert 2026
AI Support: Multiple large language models
Not for print. PDF only.

Contents

CONTENTS

Foreword

The previous volume built a language from the silicon up: a machine, a lexer, a parser, a type checker, an interpreter, a compiler. It ended with a language called Lark that worked, and a proof that its type system meant something. This book is about what happened next, and it is about three questions that only become askable once a language exists.

Can it build itself? Can it be made fast without changing what it means? And what, in the end, can it promise?

That previous volume has a name, and the debt this book owes it is best paid by using it. It is *The Language Stack* (Code Crafting no. 4), which builds Lark once, all the way down — machine, lexer, parser, type system, interpreter, compiler, and the proof that the type checker is not lying. If what you want is that ground-up construction, that is the book to read, and this one will send you back to it by name where the ground it laid is load-bearing.

It is a bridge, though, not a gate. This book stands on its own. A reader who has never opened *The Language Stack* is owed nothing more than the two-page tour of Lark in the introduction, and may begin there; a reader who has read it arrives already at home, and may skim the openings. Nothing here assumes you built the first compiler alongside us — only that you are willing to watch a second one get built, and then to watch it be held to account.

The three questions are three strands, and they are independent. Self-hosting (Part I), optimization (Part II), and proof (Part III) share a language and a method, but not a plot. You may read any one of them without the other two. A reader who cares only how a compiler comes to optimize its own source can start at Part II; a reader who came for the proofs can begin at Part III and treat what precedes it as reference. The book is built to be entered from any of its three doors, and only the third opens onto new country — the first two are a programmer's home ground, and they are there to walk you to it.

A word on how this was made. Like no. 4, the book and the code were written in coöperation with an AI, used as a tireless pair-programmer and a first-draft machine — and, more usefully than either, as an adversary that never tired of

asking whether a green number had ever been watched to fail. The decisions are the author's, and so are the errors; several of the errors are described in these pages rather than sanded off, because a book about proving programs correct that hid its own mistakes would be committing the first mistake it warns against. Where the machine was wrong we say so; where it was right about something we had gotten wrong, we say that too.

A last, practical note. This is not a published book. It is a working document, PDF only, and it will change — the code it describes is a live repository, and the prose is kept true against it rather than frozen ahead of it. Treat a page number as provisional and a claim as checkable: everything asserted here is something a program in the companion repository will confirm or contradict for you. That is the point of the book, and the subject of its appendices.

Chapter 1

Introduction

A compiler is a program that turns text into a machine. That is the story the previous book told. But a compiler is also, awkwardly, a *program* — and so it must itself have been turned into a machine by some other compiler. Follow that regress backwards far enough and you arrive at a strange place: a language that compiles itself, built by a compiler that no longer exists.

This book follows one small language, Lark, past the point where it merely works and into the three things a working language can go on to do: build itself, make itself faster, and make promises about the programs written in it. The first is a feat of engineering a programmer will find delightful and slightly uncanny. The second is the first promise the language makes about *meaning* — run faster, compute the same answer — and it is where the idea quietly enters that "the same answer" is a claim someone has to be able to check. The third is the book's real subject: a language learning to say *this divisor is never zero*, *this index is in range*, *this tree stays sorted*, and to have a machine agree. You are not asked to learn logic to get there. You are shown a thing you want to be able to say about a program, and the logic is what it takes to say it.

1.1 The method, which is really two

Everything in this book is a claim, and no claim is worth more than the way it is checked. So it is worth saying at the outset how the checking is done, because the answer changes halfway through the book, and the change is the plot.

For the first two parts the method is *differential testing against a reference implementation*. There are two Larks: a slow, readable one written in Python — the *oracle* — and the real one written in Lark itself. The oracle is correct by definition, not because it is beyond doubt but because it is what "correct" is declared to mean, and the port is held against it input by input. Two implementations

that disagree have found a bug between them; two that agree on everything you can throw at them are, for practical purposes, the same program. Every green count in Parts I and II is a claim of that one shape: the port cannot be told apart from the reference. It is a strong method, and it has a ceiling.

The ceiling is Part III. A verifier does not make a claim about a program's text, which a second implementation could referee; it makes a claim about the *world* — that a division will never fault, for every input the program will ever see. No second implementation is entitled to certify that, because it is not a fact about the code. Past that ceiling the method turns adversarial: generate hostile programs on purpose, *run* what the checker claims to have proved and let the world overrule it, and — the discipline the third part rests on — re-plant every bug you ever fixed and watch the net catch it before you trust the net again. Nine false proofs were found this way over the course of the work, and none survived. Hold two implementations against each other; and never trust a green you have not seen fail. Those are the two methods, and the second is not a surprise sprung in Chapter 18 — it is named here so that you can watch it coming.

1.2 What you need to know

Some programming. That is all of it. You should be comfortable reading code in some language, comfortable with recursion, and unbothered by the idea that a program can take another program as data. You do not need compiler theory: every term this book leans on — lexer, parser, type checker, intermediate representation, lowering, optimization pass, and later obligation, refinement, soundness — is defined where it is first used, and never assumed.

You do not need to have read *The Language Stack* (Code Crafting no. 4), the companion that builds Lark from silicon upward. A reader who has read it can skim the opening of Part I, where this book recaps what that one constructed. A reader who has not is owed a self-standing picture of the language, and the next section is it — two pages that assume nothing and leave you able to read every program in the book. The link to no. 4 is an invitation to go deeper on the ground-up build, never a gate you must have passed through first.

1.3 Lark in two pages

Lark is a small, purely functional language. *Purely functional* means a function's only effect is the value it returns: there is no assignment, no mutable variable, nothing that happens on the side. It is statically typed, with Hindley–Milner inference, so most types are worked out for you and the ones you write are checked. It has algebraic data types and pattern matching, one built-in notion of an ef-

fect (`IO`, threaded by hand), and one unusual rule — *affine* types — to which we will return. The fastest way to meet it is to read a whole program. This one, `self/samples/03_primes.lark`, prints the primes below fifty:

```
module Primes

type List a =
  | Nil
  | Cons of a, List(a)

impl Copy for List(Int) = {}

fn divides(d : Int, n : Int) : Bool =
  n / d * d == n

fn is_prime_from(n : Int, d : Int) : Bool =
  if d * d > n     then true
  else if divides(d, n) then false
  else is_prime_from(n, d + 1)

fn is_prime(n : Int) : Bool =
  if n < 2 then false
  else is_prime_from(n, 2)

fn primes_upto(limit : Int, k : Int) : List(Int) =
  if k > limit then Nil
  else if is_prime(k) then Cons(k, primes_upto(limit, k +
  ↪ 1))
  else primes_upto(limit, k + 1)

fn length(xs : List(Int)) : Int =
  match xs with
  | Nil           => 0
  | Cons(_, rest) => 1 + length(rest)
  end

fn main(io : IO) : IO =
  let ps = primes_upto(50, 2) in
  print(io, show(length(ps)) + " primes up to 50")
```

Read it top to bottom. A file is a `module` with a name. The `type` declaration introduces an algebraic data type: a `List a` is *either* the empty list `Nil` *or* a `Cons` of a head and a tail — the two ways to build one, named. The `a` is a type parameter, so `List` works for any element type; `List(Int)` is the lists of integers.

The next line, `impl Copy for List(Int)`, is the one rule worth pausing on. Lark is *affine*: a value may be used at most once, unless its type says otherwise. The reason is the language's larger ambition — a value used once can be reclaimed the instant it is consumed, with no garbage collector — and the escape hatch is the `Copy` trait, which marks a type whose values may be used freely, as integers and lists of integers are here. For most of a first reading you can let this sit in the background; it becomes load-bearing only in Part III.

The functions are ordinary. `divides` says d divides n when dividing and multiplying back lands where it started — Lark has integer division but no modulo, so this is how you ask. `is_prime_from` trial-divides upward, stopping once the divisor squared exceeds n. Each has a written type signature; inside, no types appear, because inference supplies them. `match …with …end` takes a value apart by the shapes it could have — the `Nil` case, the `Cons` case — and the `_` in `length` is a pattern that matches anything and binds nothing.

The last function, `main`, shows how a pure language talks to the world. It takes an `IO` and returns an `IO`: the effect of running is a value passed in and handed back, so `print` takes the current `io` and yields the next one, and the order of effects is just the order the values are threaded through. `show` turns a value into a string; it is the one built-in that works at any type. Run this program and it prints `15 primes up to 50`. Every program in the book is this language, and now you can read all of them.

1.4 The three questions

The book is three parts, and they answer three questions that do not depend on each other.

Can it build itself? Part I rewrites the Python oracle in Lark and compiles Lark with it — expressive power, the language demonstrating it is strong enough to describe its own construction. *Can it be made fast without changing what it means?* Part II adds an optimizer and holds every optimized program against the meaning of the un-optimized one — speed at fixed meaning. *What can it promise?* Part III extends the type system until it can state and check properties of a program's behaviour, and then turns the same machinery on the language itself — guarantees.

They are orthogonal on purpose. Self-hosting is about power, optimization about performance, proof about assurance, and a reader may take them in any order or take only one. The recommended order is the book's order, because the language grows a little in each part and the proof work leans on vocabulary the first two establish; but nothing forbids starting at the end, and a reader who came only for the third question will find the first two waiting as reference when a term sends them back.

1.5 How to use the companion repository

The book has a laboratory. It is three self-contained trees — `self/`, `optimize/`, and `prove/` — one per part, each runnable on its own without the others present. Inside each you will find the same shape: the Python oracle it is tested against, the Lark port or the checker that is the part's subject, the sample programs, and the harnesses that hold one against the other. The trees deliberately repeat shared files, because each is meant to be opened and run alone; the appendix that maps the code (Appendix D) says what lives where.

One convention runs through all of it, and it is the reason the repository exists: *every claim in this book is checked by a program you can run.* When a chapter reports that a port matches the oracle on every sample, there is a harness that returns that verdict; when it reports that seventy-five refinement programs check as they should, there is a command that checks them; when it reports that a proof of the language's soundness goes through, there is a checker that accepts it or does not. You are not asked to take the counts on faith. You are handed the thing that produced them. Appendix B is the manual for running them yourself.

Part I

A Language That Builds Itself

Chapter 2

The Chicken and the Egg

To compile Lark you need a Lark compiler. If the Lark compiler is written in Lark, you need a Lark compiler to compile the Lark compiler — and so on, forever. This is not a paradox but a practical problem, and it has a practical solution, one that every serious language has used: write the first compiler in some *other* language, then use it, once, to compile the real one, and then throw the first one away.

The other language, here, is Python. The programs in `self/oracle/` — lexer, parser, type checker, evaluator, code generator — are the *oracle*: slow, readable, and, for the purposes of this book, correct *by definition*. What follows in Part I is the story of writing each of them a second time, in Lark, and proving the second one identical to the first.

Ken Thompson gave the most famous lecture ever delivered about bootstrapping, and its subject was betrayal [Thompson, 1984]. He described a compiler taught to recognise its own source code and, when it saw it, to reproduce the very backdoor you had just deleted from that source. The trick works because a compiler is a program that outputs programs, and one of the programs it can output is itself. Nothing you can read in the source will save you, because the source is not what compiles you.

That lecture is usually read as a warning, and we will read it that way, once, in the conclusion. Here it is a description. Strip the malice out of Thompson's story and what remains is the ordinary mechanism of every self-hosting language: a compiler whose behaviour is inherited from a compiler that no longer exists, carried forward through generations of binaries, none of which you can inspect by reading a file. That inheritance is a fact about compilers, not a conspiracy. What we are going to do in Part I is build one deliberately, watch it happen, and then — this is the part Thompson's audience could not do — check it.

2.1 What "self-hosting" actually means

The phrase gets used for three different achievements, and they are not equally hard.

The first is **the source is written in the language**. Someone has typed out a Lark compiler in Lark. This costs nothing but effort, and proves nothing at all: a file of Lark that nobody has ever run is a file of text with opinions.

The second is **the compiler can compile that source**. Feed the compiler's own source to the compiler, get a compiler out. This is a real result — the language is expressive enough to describe itself — but it is weaker than it sounds, because the thing that did the compiling was still the Python oracle, or a binary built by it. We have shown that Lark *can be* compiled. We have not shown that the Lark compiler compiles *correctly*, because at no point did we run it on a job that mattered and check the result against anything.

The third is **the fixpoint**, and it is the only one that is a claim about correctness. Take the compiler produced in step two — call it C_1 — and use it to compile its own source again. That gives you C_2. Now C_2 was produced by Lark code, running as a Lark program, from the very same input that produced C_1. If the compiler is correct, C_2 must be indistinguishable from C_1: same input, same meaning, same output. Compile once more, with C_2, and get C_3. When

$$C_1 = C_2 = C_3, \quad \text{byte for byte,}$$

the compiler has reproduced itself, and it has done so *through* itself. Every stage after the first was compiled by Lark, ran as Lark, and emitted the same artefact. The property has a name — it is a fixpoint of the function "compile this source" — and it is the strongest statement a self-hosting compiler can make about itself without appealing to a proof.

It is also, importantly, a statement the compiler cannot fake by being uniformly wrong in a boring way, but *can* fake by being uniformly wrong in an interesting one. If the compiler mangles some construct, and mangles it the same way every time, then C_1, C_2 and C_3 are all mangled identically and the fixpoint holds anyway. This is Thompson's hole, and it is the reason the fixpoint is not the only thing we do. It is Chapter 8's result, and it is underwritten by everything in Chapters 3 through 7.

2.2 The oracle

An oracle is a reference you are not allowed to argue with. When the port and the oracle disagree, the port is wrong. That rule is not a claim about Python's

superiority; it is a definition, and its whole value lies in being adopted *before* you know which disagreements are coming.

The oracle here is the Python implementation in `self/oracle/`: seven files, a few thousand lines, written for clarity and not for speed. It lexes, parses, typechecks, evaluates and emits C. It is slow enough that some of our tests take a minute; it is readable enough that when a test fails you can find out why by reading it. Those two properties are the same property.

The essential discipline is that the oracle is **frozen**. Once the port begins, the reference does not move. This sounds like an inconvenience and is actually the entire point, because the alternative — fixing the oracle whenever the port disagrees with it — is not testing. It is a negotiation in which one party is allowed to change the standard, and the outcome of that negotiation is always agreement, which is why it tells you nothing. An oracle that moves is not an oracle; it is a mirror.

The freeze was broken. The honest way to say this — and we did not say it that way for some months — is that it was broken more often than anyone remembered, and that for most of the project the number we would have given you was wrong. The ledger is Appendix C. It is regenerated from the repository's own history rather than from anybody's recollection, which is the only kind of ledger worth keeping, because *a thaw you have forgotten is indistinguishable from a bug you never found*.

The thaws fall into three kinds, and the kinds matter more than the count.

Most were **additions**. Lark, when the port began, could not take a string apart: it could compare strings and concatenate them, and that was all. You cannot write a lexer in a language that cannot index a string, so the language grew primitives — to measure its length, to index, to slice, to build a string from a character, to parse an integer and a float out of one — and later two more, one to read an entire program from standard input, and one to get at the bits of a float so that the code generator could emit a constant to the bit. Eight in all. None of them changed any answer the oracle already gave; each extended what it could be asked.

But notice what that does to the claim. "Lark can express its own compiler" is true *of a Lark that grew eight primitives in order to*. That is the price of self-hosting, it is paid in the language rather than in the compiler, and it is almost never quoted. Whether it counts as cheating depends on what you wanted the claim to mean — and you should decide that now, before the book spends nine chapters making the claim look inevitable.

Some were **limits** rather than errors. The C runtime allocated its arena as a static array, which cannot be sized in gigabytes, and the self-compiling compiler wants gigabytes, so it was made to allocate. Nothing it already said changed.

And some were **genuine corrections**, where the oracle was wrong and the port was right. There are two. The second is the subject of Chapter 5, and it is the best thing in this book. Here is the first.

Lark strings had two primitives that were supposed to be inverses: one that reads a character out of a string, and one that turns a character back into a string. In Python, the first returned a Unicode codepoint and the second truncated to a byte — so above `U+007F` they stopped being inverses, and the oracle quietly contradicted itself. Nobody noticed for as long as nobody wrote a program that depended on it. Then the self-hosted parser did: it rebuilds every string literal it sees, one character at a time, through that very pair. So the parser we had written in Lark corrupted every non-ASCII string literal in every program it parsed — and so did the Python parser, which is why the two agreed, and why the test suite was green. The C runtime had been byte-oriented and correct all along. Python was the odd one out, and Python was changed to match: a Lark string is a sequence of UTF-8 bytes, and a column number is a byte offset.

That episode is worth sitting with, because it is the sharpest limit of the method in this book. The fixpoint could not have caught it: all three stages corrupt string literals in exactly the same way, so $C_1 = C_2 = C_3$ held perfectly while the bug was live. And the differential could not have caught it, because both implementations were wrong in the same place. Two things that agree have not thereby become right. They have only stopped being a source of evidence.

2.3 Differential testing

Here is the method the rest of the book is built on [McKeeman, 1998]. Take one input. Run it through the Python component and through the Lark component. Demand that the two outputs be *byte-identical*.

Not "both succeed". Not "both produce a reasonable token stream". Identical: the same bytes, in the same order, or the test fails and prints a diff. The standard is deliberately brutal, because every weakening of it is a place for a difference to live. A test that checks the token count catches a missing token and misses a wrong column. A test that checks kinds and text catches a wrong column and misses a wrong *order*. The only comparison with no hiding places is the one that admits no difference at all.

What this buys you is a shift in what the test is claiming. An ordinary test suite says *the outputs match the expectations I wrote down*, and its coverage is bounded by the author's imagination on the day. A differential says something else: *on this input, the two implementations cannot be told apart.* The expectations were never written down — they are whatever the oracle does — so the test knows things about the language that the person who wrote the test does not.

Concretely, and in miniature. Here is a line of Lark:

```
let x = 1 in x + 2
```

Both lexers are asked for their tokens, and both are asked to print them in the same shape — one token per line, as `KIND line col text`:

```
LET 1 1 let
NAME 1 5 x
ASSIGN 1 7 =
INT 1 9 1
IN 1 11 in
NAME 1 14 x
PLUS 1 16 +
INT 1 18 2
EOF 1 19
```

Nine lines from Python, nine lines from Lark, and the test is `diff`. Note what is being compared and what is not: the two lexers are not required to have the same internal structure, the same functions, or the same idea of how to recognise a number. They are required to be indistinguishable *from the outside*, on this input. That is the only kind of sameness a compiler user can ever observe, and it is therefore the only kind worth testing for.

The scale is what makes the method bite. That comparison is not run on one line; it is run on every program in the corpus, and the corpus includes the compiler's own source. The self-hosted lexer lexes itself, the self-hosted parser parses itself, the self-hosted type checker type-checks itself — and each time, the frozen Python is standing beside it, producing the same bytes, and the harness is diffing them. `make lextest`, in the `self` strand, is this paragraph made executable.

Three consequences follow, and they will recur.

First, a differential is only as good as the thing you serialise. If the two sides agree on tokens but you print only the kinds, the columns are untested and you will not know it. Every chapter in Part I therefore has a moment where the real question is not "how do I port this?" but "what, exactly, do I put on the line?".

Second, when the two sides disagree, you have learned something and you do not yet know what. The disagreement is a fact; "the port has a bug" is an interpretation, and it is the right one perhaps nine times in ten. The tenth is the UTF-8 story above. The discipline is to read the diff before deciding who is wrong.

Third — and this one is a limit, not a technique — the method can only ever tell you that two implementations are indistinguishable. It cannot tell you that either is right, and it is at its weakest where you would most like it to be strong: the port and the oracle are not independent witnesses, because one was written

from the other, and so they share their misunderstandings as reliably as they share their logic. A differential is blind to any error both sides make. That is not a flaw to be engineered away; it is what the method is. It means the last question in the book — does the generated code do the right thing on a real processor? — is one no amount of agreement can answer, and it is why Chapter 12 ends by leaving the workstation. Two things that agree have stopped being a source of evidence about each other. Somewhere the chain has to touch the world.

2.4 A map of the port

Part I ports the oracle in dependency order, one component per chapter, and each chapter ends with its own differential green before the next one begins. The rule is that nothing is built on an unverified foundation — so the parser is not written until the lexer is byte-identical, and the type checker is not written until the parser is.

Ch.	Lark	What it must reproduce
3	`lex.lark`	the token stream: kinds, text, lines, columns
4	`parse.lark`	the abstract syntax tree, printed
5	`types.lark`, `tast.lark`, `infer.lark`	the inferred type of every top-level definition — and the same rejections, for the same reasons
6	`cek.lark`	the value a program evaluates to, and what it prints on the way
7	`emit_c.lark`	the C source, character for character

Those files are in `self/lark/`, beside the Python they were ported from in `self/oracle/`, and the harness that compares each pair is in `self/harness/`. You can read any chapter's Lark and its Python side by side; that is what they are arranged for.

Chapter 5 is three files rather than one because the type checker is the largest thing in the compiler, and because two of its three pieces are pure data — what a type *is*, and what a typed tree *is* — while only the third is an algorithm. Chapter 6, the evaluator, is the one component the compiler does not strictly need in order to compile; it is there because it is the definition of what Lark programs *mean*, and Part III will need something to be correct against.

Chapter 8 assembles them. The lexer, parser, type checker and emitter are concatenated into a single Lark program that reads a Lark program on standard input and writes C on standard output — a compiler, in Lark, in one file. It is compiled by the oracle, once, and then never again: from that point on the ladder

climbs by itself, and the only question left is whether the rungs are identical. That is `make bootstrap`, and it is the end of Part I.

Chapter 3

The Lexer, Written in Its Own Language

The first thing a compiler does is the easiest thing to describe and the first place a port can go wrong: it turns a flat string of characters into a list of words. Lark's lexer, in Python, is 400 lines of loop and character comparison. Rewriting it in a purely functional language with no mutable variables is the first real test of whether Lark can hold a compiler at all.

It is a good first test just because nothing in it is clever. There is no algorithm here to get wrong, no data structure worth arguing about. If the port is hard, the difficulty is not in the lexer — it is in the language, and we want to find that out on the cheapest component in the compiler rather than the most expensive one.

What the exercise turns up, and what the rest of Part I will keep turning up, is this: the functional lexer is not harder than the imperative one. It is *differently shaped*. And the shape is not a matter of taste — it is forced, in a way you can point at, by two features of the type system.

3.1 What a lexer does

A lexer is a function from text to a list of tokens. That is the whole job.

The input is a string — a flat sequence of characters, in which `let` is not a keyword and `1` is not a number, because nothing in a string is anything yet; it is bytes in a row. The output is a list of *tokens*: the smallest units the rest of the compiler is willing to think about. Each token records what *kind* of thing it is, the exact text it was made from, and where in the source it was found.

```
source:   let x = 1 in x + 2
```

```
    tokens:   LET    1 1   "let"
              NAME   1 5   "x"
              ASSIGN 1 7   "="
              INT    1 9   "1"
              IN     1 11  "in"
              NAME   1 14  "x"
              PLUS   1 16  "+"
              INT    1 18  "2"
              EOF    1 19  ""
```

Three details in that dump matter more than they look.

The token keeps its **text**, not just its kind. `INT` is not enough; the parser will want to know it was `1` and not `2`. So a token carries the slice of source it came from, verbatim — and for a string literal that means the quotes and the uninterpreted escapes come along too. The lexer does not decide what an escape means. It decides only where the literal ends.

The token keeps its **position**: a line and a column of its first character. Nothing downstream needs this in order to compute anything. It exists so that when the type checker refuses a program in Chapter 5, it can say *where*. Position is the compiler's only apology for failing, and it is threaded from here to the end.

And the list ends with `EOF`. Not because the parser could not ask whether the list is empty, but because a sentinel means the parser can always look at the next token without first checking that there is one — a check paid for once, in one place, instead of everywhere.

There is one rule of judgement, and it is the only one: the lexer takes the **longest match**. Reading `=` it does not stop; it looks at the next character, and if that is another `=` the token is `EqEq` and not two assignments [Aho et al., 2006]. Reading `123.45` it takes the whole number — but reading `123.`, with no digit after the dot, it takes `123` and leaves the dot behind. Every such decision is a place where two implementations can quietly differ, which is why we are going to compare them character for character.

3.2 The problem: no mutable cursor

Here is the Python. A lexer object owns a position, a line and a column, and it walks:

```
def _advance(self) -> str:
    ch = self.source[self.pos]
    self.pos += 1
    if ch == "\n":
        self.line += 1
```

```
        self.col = 1
    else:
        self.col += len(ch.encode("utf-8"))
    return ch
```

The three assignments are the lexer. Everything else in the file is a decision about *when* to call this.

Lark has no assignment. There is no `self`, no field to update, no statement that can make `pos` mean something other than what it meant on the line above. A name, once bound, stays bound to that value for the rest of its scope, and that is not a restriction the language relaxes for compiler authors.

The replacement is mechanical, and it is the single idiom the entire port is built out of: **make the state a value, and return the new one**. A function that would have mutated the cursor instead takes a cursor and gives a cursor back.

```
type Pos = | P of Int, Int, Int  (* offset, line, col *)
impl Copy for Pos = {}

fn adv(src : String, st : Pos) : Pos =
  let p = pos_of(st) in
  if string_index(src, p) == 10
    then P(p + 1, line_of(st) + 1, 1)
    else P(p + 1, line_of(st), col_of(st) + 1)
```

Line for line, it is the same lexer. The `if` is the same `if`; the branch that saw a newline still resets the column and bumps the line. What has changed is only *where the answer goes*. Python writes it into the object it was called on; Lark hands it back to the caller, who must now do something with it. The mutation has not disappeared. It has been made visible in the type.

That visibility has a price, and the price is paid on the caller's side. Python's reader functions can consume characters and then look at `self.pos` to see where they ended up — the loop and the cursor share a variable, so there is nothing to carry. In Lark, every function that consumes anything must hand back two things, what it read and where it now is, and the caller must name both. So the lexer's central type is not a token; it is a token *and* a cursor:

```
(* a token, and the state after it *)
type Lexed = | Lexed of Token, Pos
```

and every reader in the file — numbers, strings, names, symbols — returns one. The main loop then reads as what it actually is, a fold threading a value through:

```
fn tokenize(src : String, n : Int, st : Pos) : List(Token) =
  let st2 = skip(src, n, st) in
  if pos_of(st2) >= n then
    Cons(Tok(KEof, "", line_of(st2), col_of(st2)), Nil)
  else
    match lex_step(src, n, st2) with
    | Lexed(tok, st3) => Cons(tok, tokenize(src, n, st3))
    end
```

Beside the Python:

```
def tokenize(self) -> list[Token]:
    tokens: list[Token] = []
    while True:
        tok = self._next()
        tokens.append(tok)
        if tok.kind == TK.EOF:
            break
    return tokens
```

The `while` became a recursion and the `append` became a `Cons`. Those are the changes everybody expects and nobody learns anything from. The change worth noticing is `st2` and `st3`. In Python there is one cursor and it is never named at the call site, because it lives in `self` and every function reaches in. In Lark the cursor is named at every point it changes, and the names form a chain the type checker will not let you break. You cannot forget to advance — the position you would have to pass on is a value you would have to invent. You cannot advance twice by accident, because the second advance would need a cursor nobody gave you.

That is the whole trade. Imperative code shares a cursor and trusts you to update it exactly once. Functional code makes you pass it, and cannot be lied to. The bug where a lexer spins forever because one path forgot `self.pos += 1` does not have a shape in Lark.

One awkwardness shows up here and never quite goes away. `Pos` is a three-field ADT with three accessor functions, where a tuple would obviously do — and Lark has tuples. But Lark's `let` binds a single name and cannot destructure, so `let (p, l, c) = adv(...)` is not something you can write; a multi-value return has to be taken apart with a `match`, and a `match` at every cursor step would drown the file. Wrapping the three fields in a named type with accessors buys the readability back. It is not elegance; it is a small limitation of the language,

paid for in three lines of boilerplate. It is named here because a truthful report of a port is largely a list of things like this.

3.3 Affine types get in the way, once

Lark is *affine*: a value may be used at most once [Wadler, 1990, Tov and Pucella, 2011]. Bind a list to a name, pass it to a function, and the name is spent — you may not pass it again. The rule exists so that the compiler knows statically when a value's last use has happened, and therefore when its memory can be reclaimed without a garbage collector. Chapter 7 is where that promise is cashed. Here it is only a rule to obey.

You would expect it to make this chapter miserable. Threading a cursor through two hundred call sites is just the pattern an at-most-once rule ought to punish: `st` is read by `pos_of`, then by `line_of`, then by `col_of` — three uses of one name in one expression. In fact the lexer never fights it, and the reason is the design decision that makes the entire port survivable:

```
impl Copy for Pos = {}
```

`Copy` is the escape hatch, and it is not a loophole. It is the rule applied faithfully. Affinity is about resources: things that can be consumed, freed, invalidated. An `Int` is not a resource, and neither is a triple of them. Copying one costs a machine word and leaves nothing behind to free, so a type built only from such things may declare itself `Copy`, and the at-most-once rule stops applying to its values. `Pos` is three `Int`s. `Token` is a kind, a string and two `Int`s. Both are `Copy` — so the cursor can be read three times in a line, the source string can be handed to every function in the file, and none of it is a fight.

The lexer therefore meets affinity exactly once, and not where you would look for it. The rule does not merely count uses along a path; it **sums uses across the branches of a `match`**. Two arms that each use `x` once constitute two uses of `x`, even though only one arm can ever run. The checker is being conservative in the direction that costs the programmer, because it will not track which branch was taken.

For `Copy` data this is invisible. For the one thing in the language that can never be `Copy`, it is not. That thing is `IO`: the value representing the right to perform an effect, threaded through a program by hand, and unique on purpose — if you could copy it you could print in two worlds. So this does not typecheck:

```
match result with
| Ok(toks) => print(io, dump(toks))  (* io used here *)
| Err(msg) => print(io, msg)         (* ...and here  *)
```

```
end
```

and the complaint is affine, not type. Both arms are individually correct. Together they spend `io` twice.

The fix is a discipline rather than a workaround, and it is the shape of every `main` in this book: **compute the string purely, print it once**.

```
let out = match result with
          | Ok(toks) => dump(toks)
          | Err(msg) => msg
          end in
print(io, out)
```

Effects get pushed to the edge, and the middle of the program stays a function. This is what it means to say a language "encourages" purity: not encouragement — the alternative does not compile. The lexer is *pure*; it builds the entire token list and returns it, and only `main` touches the world. That was not a stylistic choice. It is the only program the affine checker would accept.

It is fair to call the summing-across-arms rule a wart, and we will. It is stricter than it needs to be: a checker that understood the arms are exclusive would take both programs above. That checker is not the one we have, and the frozen oracle implements the one we have. So the idiom stands, it is written down here, and it will be back — in Chapter 6, where the evaluator has to print, and again in Chapter 8, where the bootstrap compiler must either emit C or report a type error, and cannot do both, and therefore does neither until it knows which.

3.4 The differential test

The lexer is done when it cannot be distinguished from the Python one. Not when it looks right, and not when its tests pass, because both of those are opinions about the lexer held by the person who wrote it.

`make lextest`, in the `self` strand, does this. For every `.lark` file in the corpus it runs `self/oracle/lexer.py` over the source, and separately runs `self/lark/lex.lark` — as a Lark program, interpreted — over the same source. Both are asked to serialise their token list in one shape, one token to a line:

```
KIND  line  col  escaped-text
```

and the two streams must be equal, line for line, byte for byte. When they are not, the harness prints a diff.

That serialisation is the load-bearing decision in the test, and it is worth being explicit about why. Everything on the line is something the two lexers could disagree about — and everything left *off* the line is something they are then free to disagree about undetected. Print only kinds, and every column is untested. Print the text unescaped, and a token containing a newline ends the line early, so a whole family of string-literal bugs becomes invisible. The rule for building a differential is: put on the line everything the component is responsible for, and nothing it is not. The lexer is responsible for kind, text, line and column, so all four go on the line — and the escaping is defined identically on both sides, so that it cannot itself become the difference.

The corpus is every `.lark` file in the strand's tests and samples, *and the compiler's own source*. That last clause is the one that matters. The lexer's first serious input is itself; and by the end of Part I the corpus also holds a parser, a type checker, an evaluator and a code generator, all written in Lark, all of them thousands of lines. The corpus grows every time the compiler does, and each new component immediately becomes a test of every component before it. Today the pinned result is

`lextest`: **52 ok, 0 fail, 0 skip**

and the number on the left will keep moving as the compiler grows. The number in the middle is the invariant. (An earlier pin said 46, which was true when the compiler was a single module; a count is a fact about the corpus, not about the lexer. `self/harness/BASELINES.md` records what each number was when it was last measured, so that we notice when one stops being re-measured.)

This is the first place the method pays, and it pays at once: the port had bugs, and every one of them was found by a diff rather than by a reading. But it also pays in a way that is easy to miss. Consider what has actually been established. Not that the Lark lexer is correct — we have no independent account of what correct would mean. What has been established is that the two lexers agree on the longest match for every construct in fifty-two real programs, including the compiler itself, down to the column of every token. Those lexers were written in different languages, with different control flow, at different times, by someone who had forgotten most of the first by the time they wrote the second. Independent agreement of that kind is evidence.

It is not proof, and Chapter 2 has already shown how it fails: when both sides are wrong in the same place, agreement tells you nothing. It is here, in the lexer, that this actually happened — over a single non-ASCII character. Which is why the column in that dump is a byte offset, and why the Python you saw earlier says `self.col += len(ch.encode("utf-8"))` where any reasonable lexer would say `+= 1`. It says that because the port made us look.

Chapter 4

The Parser, Written in Its Own Language

A parser turns a flat list of tokens into a tree. The Python version is a recursive-descent parser that consumes tokens from a mutable stream. The Lark version cannot consume anything: it must *return* what is left. That single change — from "take the next token" to "here is the tree, and here is the rest of the input" — reorganises the entire program, and it is worth the reorganisation.

It is also the point at which the port stops being an exercise. The parser is a thousand lines of Lark, the largest single artefact so far, and the first one where the language could plausibly have said no. It did not. But it made two things awkward on the way, and one of them is a limitation we are still living with.

4.1 What a parser does

The lexer gave us a list. A list has no structure beyond order, and order is not enough: `1 + 2 * x` and `(1 + 2) * x` are the same tokens in the same sequence, and they are different programs. Somebody has to decide which grouping the flat sequence meant, and that somebody is the parser.

What it produces is an *abstract syntax tree*: a value whose shape *is* the grouping. The tree for `1 + 2 * x` has a `+` at its root, a `1` on the left, and a `*` node on the right. Nothing about it is textual any more. The parentheses are gone — not lost, but *absorbed*, because a tree does not need them: its shape says what they said. That is the sense in which the tree is "abstract". It has dropped every part of the source that existed only to help a reader recover the structure, and kept the structure.

Lark's tree has about twenty kinds of node, in four families that recur for the rest of the book:

Family	Examples	What it describes
`Ty`	`TName`, `TFn`, `TApply`	a type, as *written*: `Int`, `List(a)`, `Int -> Bool`
`Pat`	`PVar`, `PCon`, `PWild`	a pattern, on the left of a `match` arm
`Expr`	`Lit`, `Var`, `BinOp`, `If`, `Match`, `Let`, `Lambda`, `Apply`	anything that has a value
`Decl`	`FnDecl`, `LetDecl`, `TypeDecl`, `TraitDecl`, `ImplDecl`	a top-level definition

and a `Program` is a module name, a list of imports, and a list of declarations. That is the entire language, as data.

One distinction in that table earns its keep later. A `Ty` is a type *as the programmer wrote it* — a piece of syntax, no more meaningful in itself than an expression is. It is not what the type checker computes. Chapter 5 builds a second, separate representation for *inferred* types, and the two are not the same thing: one is what you said, the other is what is true. Keeping them apart is why the type checker is three files rather than one.

Here is the tree, printed. This declaration:

```
fn f(x : Int) : Int = 1 + 2 * x
```

parses to this, in the canonical form both implementations print:

```
(FnDecl false f [] [(Param x (TName Int))] (TName Int)
  (BinOp + (Lit (VInt 1)) (BinOp * (Lit (VInt 2)) (Var x))))
```

Read the last line and the parser's entire job is visible: the `*` sits *inside* the `+`, as its right child. Nobody wrote a parenthesis. The grouping was inferred from precedence, and the tree is where that inference is recorded.

4.2 The token cursor as a value

Python's parser owns the token list and an index into it:

```
def _advance(self) -> Token:
    tok = self.tokens[self.pos]
    if tok.kind != TK.EOF:
        self.pos += 1
```

```
    return tok

def _expect(self, kind: TK) -> Token:
    tok = self._peek()
    if tok.kind != kind:
        raise self._error(
            f"expected {kind.name}, got {tok.kind.name}")
    return self._advance()
```

Every parse function is a method, and not one of them mentions the position. They call `_expect` and `_match`, and the cursor moves somewhere off to the side, inside `self`. This is comfortable, and it is just the comfort Lark withdraws.

The Lark parser cannot move a cursor, so it passes one. Every parse function in the file has the same shape:

`List(Token)` → `(Node, List(Token))`

— give me the tokens, and I will give you back a node *and the tokens I did not use*. That signature is the parser. Once you have accepted it, the thousand lines write themselves; until you have accepted it, nothing works.

The interesting consequence is what becomes of `_expect`. In Python it is three operations — look at the head, check its kind, step past it — and a fourth if you also want the token's text. In Lark it is not a function at all. It is a pattern:

```
match ts with
| Cons(Tok(KUpper, name, _, _), rest) => ...
  (* name and rest, in one step *)
```

The match tests the head's kind, binds its text to `name`, and binds the tail to `rest`: peek, check, extract and advance, in one construct, with the compiler checking that the other cases are handled. Where the Python parser calls a helper that mutates an index, the Lark parser takes the list apart. This is not a workaround for the missing cursor. It is a better spelling of what the cursor was for.

Note also that the return type here is a genuine tuple, `(Expr, List(Token))`, unpacked by the caller with `match`:

```
match pAnd(ts) with | (l, r) => pOrTail(l, r) end
```

The lexer could not do this — it wrapped its two results in a named `Lexed` type instead — and the reason is the limitation named in Chapter 3: `let` binds one name and cannot destructure, while `match` can. The lexer moves its cursor at every character, and a `match` per character would have buried the file. The parser

calls a sub-parser and then does something with both halves, so one `match` per call was the cost anyway. The same missing feature produces two different idioms, because the code around it has different shapes — which is worth noticing in general: a restriction does not have a workaround, it has one workaround per context.

What the value-cursor buys, beyond tidiness, is that **there is no hidden state to get wrong**. A Python parse function that returns without consuming anything leaves the cursor where it was, and the loop above it spins forever — a bug you find by hanging, not by failing. In Lark, the tokens a function did not consume *are* its return value. Forgetting to advance is not a subtle slip; it is returning the wrong thing, and the next function is handed input that visibly still contains the token you thought you had eaten. Backtracking, which in Python means saving and restoring `self.pos`, is likewise free: the old token list is still a value, still in scope, still valid. You do not restore a cursor. You use the one you already had.

4.3 Precedence without mutation

Precedence is where recursive descent earns its name [Nystrom, 2021]. Lark's ladder is the usual one — `or` binds loosest, then `and`, then comparison, then `+` and `-`, then `*` and `/`, then unary minus, then application — and the classical implementation is one function per level, each calling the level below it and then looping:

```
def _parse_add(self) -> Expr:
    left = self._parse_mul()
    while self._at(TK.PLUS, TK.MINUS):
        op = self._advance().text
        left = BinOp(op, left, self._parse_mul())
    return left
```

That `while` is doing two things at once, and they are worth prising apart. It loops, and it *accumulates*: `left` is rebuilt on every turn, each time becoming the left child of the node that replaces it. The accumulation is what makes `a - b - c` parse as `(a - b) - c` rather than `a - (b - c)`. Left-associativity lives in that one rebinding, and it is easy to miss, because it looks like an assignment.

Lark has no `while` and no assignment, so it cannot hide the accumulator. It has to name it — and once named, it is a parameter:

```
fn pAdd(ts : List(Token)) : (Expr, List(Token)) =
  match pMul(ts) with | (l, r) => pAddTail(l, r) end
```

```
fn pAddTail(left : Expr, ts : List(Token))
    : (Expr, List(Token)) =
  match ts with
  | Cons(Tok(KPlus, _, _, _), r) =>
    match pMul(r) with
    | (rhs, r2) => pAddTail(BinOp("+", left, rhs), r2)
    end
  | Cons(Tok(KMinus, _, _, _), r) =>
    match pMul(r) with
    | (rhs, r2) => pAddTail(BinOp("-", left, rhs), r2)
    end
  | _ => (left, ts)
  end
```

Every rung of the ladder is such a pair: `pOr`/`pOrTail`, `pAnd`/`pAndTail`, `pCompare`/`pCompareTail`, `pAdd`/`pAddTail`, `pMul`/`pMulTail`. The `while` became a tail call; the loop variable became the first argument of the function it calls.

Look at what that does to associativity. In the Python it is a side effect of rebinding a local. In the Lark it is written on the page: `pAddTail(BinOp("+", left, rhs), r2)` says, in one expression, *the tree so far becomes the left child of the new node, and that becomes the tree so far.* Right-associativity would be a different expression, not a different loop. The functional version is not shorter, and nobody should pretend it is. It is more explicit — and in a compiler that is usually the trade you want, because what you got wrong is nearly always what you left implicit.

There is no level counter, and this is the part people expect to be hard: if you cannot mutate a "current precedence" variable, how do you climb? The answer is that you never wanted one. **The call chain is the precedence table.** `pOr` calls `pAnd` calls `pCompare` calls `pAdd` calls `pMul`; the ladder is the stack, fixed at compile time, in the source, where a reader can see it. A numeric precedence table is what you write when you need the levels to be *data* — a language with user-defined operators needs just that. Lark's operators are fixed, so Lark's levels are functions, and a mutable counter would have been the worse program in Python too.

One place the affine checker does have to be argued with. Look again at `pCompareTail`: `left` is used in the `==` arm, and in the `!=` arm, and in the `<` arm — six arms, six uses, and only one of them can ever run. By the rule from Chapter 3, that is six uses of a value permitted one. So every AST type in the file carries `impl Copy`, and the objection lapses. The instance is legitimate — a syntax tree is a value, not a resource; nothing is freed by dropping one — but this is the first point in the port where `Copy` is written not because the type is trivially copyable, but

because *the checker would otherwise reject correct code*. That is a wart collecting rent, and it will not be the last time.

4.4 The test that matters

`make parsetest` runs `self/oracle/parser.py` and `self/lark/parse.lark` over every program in the corpus, serialises both trees into the S-expression you saw above, and diffs them. The pinned result:

`parsetest`: **52 ok, 0 fail, 0 skip**

The corpus, as always, includes the compiler's own source — so `parse.lark` parses `parse.lark`, and the tree it builds is compared, node for node, against the tree Python builds from the same file. A thousand-line parser, written in the language it parses, agreeing with its own reference implementation about its own text.

That is a satisfying sentence, and it deserves care, because the temptation is to hear more in it than it says.

It does not prove the parser correct. The self-parse is a test with one input — a large and structurally rich input, which is why it finds bugs, but one input, with fifty-one more around it. Nothing here rules out a construct that nobody in the corpus happens to write.

It does not prove the grammar right, either. Both parsers could accept a grouping the language was never meant to have, and they would agree perfectly while doing it. A differential compares implementations. It has no access to intentions, and it never will.

What it does establish is narrower than either, and it is not nothing: across fifty-two real programs, one of them a thousand lines long, *two independently written recursive-descent parsers make every grouping decision identically*. Every precedence, every associativity, every place where a construct could have ended one token earlier or later. Those are the decisions parsers get wrong, and they are just the decisions an ordinary test suite — which checks the trees its author thought to check — covers worst.

One asymmetry I would rather name than let a reader discover. The Python parser, given a malformed program, raises a `ParseError` with a line and a column. The Lark parser cannot: the port has no exceptions, and its match arms end in a fallback that returns a placeholder node. On the corpus, which is valid Lark, those arms never fire, and the two parsers agree because both are walking the happy path. So `parsetest` tests *acceptance, and nothing else*. The error behaviour of the port is untested, because the port has almost no error behaviour to test. That is a real gap, it is the method's, not the language's, and it is one reason the

type checker in Chapter 5 takes a different route — there, failure is a *value*, a `Result`, and the differential covers the rejections too.

Finally, something the port turned up that no amount of reading would have. The parser `imports` the lexer — that is what a module system is for — and the harness does not use the import. It *concatenates* the two files and type-checks them as one module instead. The reason is a hole in the frozen implementation: the import path re-checks an imported module without the pass that pre-registers mutually recursive functions, so `lex.lark`'s forward references fail to resolve when it is imported, though the same file type-checks perfectly on its own. A module that compiles alone and breaks when imported is a bug in the language, and it was found not by testing the language but by trying to build something out of it. The workaround costs one line in a harness. The finding is that the port has become the most demanding user the toolchain has ever had — and we are three chapters in.

Chapter 5

The Type Checker, Written in Its Own Language

This is the hard one. The type checker is the part of the compiler that decides what a program *means* before it runs, and Lark's — Hindley–Milner inference, plus affine use-checking, plus trait bounds — is the most intricate program in the system. Porting it to Lark required, among other things, discovering that the Python original had a limitation we had never noticed, because no Python program had ever been strange enough to hit it.

That last sentence is the chapter. Everything before it is the machinery you need in order to see why the finding is interesting.

5.1 Inference in one picture

A type checker that made you write every type would be a simple program and a miserable language. Lark makes you write almost none and works the rest out. The algorithm that does this is Hindley–Milner inference — Algorithm W — and it is one of the genuinely beautiful results in the subject [Hindley, 1969, Milner, 1978, Damas and Milner, 1982]. It fits in a paragraph.

Guess, then solve. Walk the program. Every time you meet something whose type you do not know, invent a fresh unknown — call it α_1, α_2: a name standing for a type nobody has determined yet. Every time you meet a construct that *constrains* a type, write down an equation. Then solve the equations. Whatever the solution assigns to each unknown is the type; if the equations have no solution, the program is ill-typed, and the equation that could not be solved is the error message.

Take `fn f(x) = x + 1`. We do not know `x`'s type, so it is α. We meet `+`, which takes two `Int`s and gives an `Int`, so we write down $\alpha = \texttt{Int}$. Solving gives

$\alpha = \texttt{Int}$, so `f` has type `Int -> Int`. Nobody wrote that down. It was the only possibility.

Now `fn id(x) = x`. Again α; this time no constraint ever arrives, and we finish with an unknown that was never pinned. That is not a failure — it is *polymorphism*. The function works for every type, so its type is "for all α: $\alpha \to \alpha$", and each caller gets a fresh copy of α to pin as it likes. A type with quantifiers in front is a *scheme*; turning a solved-but-unconstrained unknown into a quantifier is *generalisation*. Nearly everything difficult in Hindley–Milner is a question about when you are allowed to generalise.

The solving step is *unification*, and it is the entire engine: given two types, find the substitution — the mapping from unknowns to types — that makes them equal. It is short enough to read whole:

```
fn unify(a : Mono, b : Mono) : Result(Subst, String) =
  match (a, b) with
  | (MVar(i), MVar(j)) =>
      if i == j then Ok(SLeaf) else bind(i, b)
  | (MVar(i), _)       => bind(i, b)
  | (_, MVar(j))       => bind(j, a)
  | (MCon(n1), MCon(n2)) =>
    if strEq(n1, n2) then Ok(SLeaf)
    else Err("cannot unify "
             + pretty(a) + " with " + pretty(b))
  | (MApp(h1, as1), MApp(h2, as2)) =>
    if strEq(h1, h2) then unifySeq(as1, as2)
    else Err("cannot unify "
             + pretty(a) + " with " + pretty(b))
  ...
```

Two unknowns: make one stand for the other. An unknown against anything: make the unknown stand for that thing — that is `bind`. Two concrete types: they match if they have the same name, and then their arguments must match pairwise, which is where the recursion comes from. Anything else: the equations have no solution, and this is a type error.

`bind` has one subtlety, and it will matter enormously in a few pages. Before mapping α to a type τ, it checks that α does not occur *inside* τ. Without that check, unifying α with `List`(α) would succeed, and α would stand for a list of itself, of itself, forever — an infinite type. The check is called the *occurs check*, it is two lines, and every textbook mentions it [Pierce, 2002]. It is not, as we shall see, the same thing as being safe from infinite types.

Two last words, both visible above. A *monotype* is a type with no quantifiers — `Int`, `List`(α), α `->` β. A scheme is a monotype with quantifiers in front.

Values have monotypes; names in the environment have schemes; *instantiation* turns a scheme into a monotype by handing out fresh unknowns, and generalisation goes the other way. Keep those four words straight and the rest of this chapter is bookkeeping.

One thing worth noticing early. In Chapter 4 the parser produced a `Ty` — a type *as written*. Here the checker works with a `Mono` — a type *as inferred*. They are different data types in different files, and they must be, because they answer different questions. A `Ty` can say `List(a)`, where `a` is a name the programmer chose. A `Mono` says `MApp("List", [MVar(7)])`, where 7 is an unknown the algorithm invented. Converting one to the other is a function. Conflating them is how type checkers get written twice.

5.2 Three passes

The checker walks the program three times, and the reason is that Lark lets you mention things before you define them.

Pass 1 registers everything that is not a value: type declarations, and with them the type of every constructor; traits and their method signatures; and the `impl Copy` and `impl Show` instances, which pass 2 needs and which may appear anywhere in the file. After pass 1 the checker knows the vocabulary.

Pass 1.5 exists for mutual recursion, and its half-number is not a joke — it is a patch, and it behaves like one. If `f` calls `g` and `g` calls `f`, then whichever the checker reaches first will find the other undefined. So before checking any body, this pass walks the declarations again and pre-registers the type of every function that is *fully annotated* — one whose parameters and return type are all written down, so that its type can be read off the declaration without inferring anything. Functions without complete annotations are skipped, and can therefore be self-recursive but not mutually recursive.

Notice what that means. **In Lark, mutual recursion requires annotations.** That is not a stylistic recommendation; it is the pass structure, visible from outside as a language rule. It is not an unusual rule — inferring a mutually recursive group in full generality is where Hindley–Milner starts to hurt — but it is a rule that exists because of an implementation decision, and the two are worth being able to tell apart.

Pass 2 checks the bodies in order, and this is where Algorithm W runs. Each function is checked, generalised, and added to the environment, so a function may use anything defined above it, anything registered in pass 1, and anything annotated in pass 1.5.

The port reproduces all three, in the same order, with the same skips. It has to: the differential compares the *result*, and a checker that pre-registered one function more than the oracle would accept a program the oracle rejects.

5.3 Substitutions without a mutable map

A substitution is the answer the solver is building: a mapping from unknowns to types. Python represents it as a dictionary, and unification passes dictionaries back and forth:

```
def compose(s1: Subst, s2: Subst) -> Subst:
    """s1 . s2 -- apply s1 to s2's range, then union."""
    result = {k: apply(s1, v) for k, v in s2.items()}
    result.update(s1)
    return result
```

Lark has no dictionary and no mutation, so the substitution is threaded: every function that could learn something takes the substitution it was given and returns the one it now believes. The signature of `unify` says just this — `Result(Subst, String)`: either a new substitution, or the reason there is none. **Failure is a value.** That single fact is why this chapter's differential can test rejection as well as acceptance, where Chapter 4's could not.

The first version represented `Subst` as an association list, which is the obvious functional answer: a list of pairs, searched by walking it. It was correct and it was too slow. A substitution during a real inference run holds hundreds of entries, `apply` consults it at every node of every type, and a linear scan inside a quadratic walk is the sort of arithmetic that turns a two-second type check into a two-minute one.

So it became a red–black tree:

```
type SColor = | SRed | SBlack
type Subst  =
  | SLeaf
  | SNode of SColor, Subst, Int, Mono, Subst
```

— a persistent balanced tree keyed by the unknown's number, with $O(\log n)$ lookup and insertion, sharing almost all of its structure with the tree it was built from. This is where the functional style stops being a constraint and starts being an advantage: the old substitution is still valid after the new one is made, which is what lets the checker attempt a unification, fail, and carry on with what it had. In Python that is a copy, and so in Python nobody does it.

The important thing about the change is what it did *not* do. It did not alter a single answer. `infertest` stayed byte-identical across the rewrite, which is how we know the rewrite was a rewrite and not a rewrite plus a bug. That is the differential being used as a refactoring tool rather than as a porting tool — the move that all of Part II is built on.

There is a second quadratic in this algorithm, and it is worth knowing about because the red–black tree does not touch it. Look again at where a substitution is *used*: after inferring the left side of an application, Algorithm W applies what it has just learned to the entire type environment before inferring the right side. That operation is `applyEnv`, and the checker calls it at fifteen sites — every entry rebuilt, at every node. The tree made each individual lookup cheap; it did not make the environment smaller, and it did not make the walk visit it less often.

On ordinary code nobody notices: the environment is a few dozen schemes and a function body is a few dozen nodes. The cost was found by *torturing* the refinement checker of Chapter 17 with programs no one would write — a single expression three thousand terms deep — and watching eight seconds disappear inside the type-checker before any refinement work began. (That measurement is of the Python oracle, whose `_apply_env` is the same function under a different spelling; the self-hosted checker inherits the shape, and its environment is an association list, which does not help.)

The textbook fix is to stop applying the substitution eagerly and instead apply it at the point of lookup, threading one growing substitution through the inference rather than one rewritten environment. We have not made it, and the reason is the subject of Chapter 17: by the time the cost was measured, this type-checker had become a *frozen oracle*, the fixed point against which a later extension is a differential. A cost that is bounded, terminating, and paid only by programs nobody writes is a poor reason to move the one thing in the system that is not allowed to move. So it is written down instead — which is the candid disposition of a known cost, and a better one than a quiet edit to the file you promised would stay still.

The general lesson is the one the profiler keeps teaching: an asymptotic problem usually has an innocent line at the centre of it. Here the line reads "apply the substitution to the environment," it is just what the inference rule on the page says to do, and it is quadratic.

5.4 Affine checking and trait bounds

Hindley–Milner gives you types. Lark wants two things it does not give you.

Affine use-checking rides along as a second piece of state, threaded beside the substitution: a table from name to use-count.

```
Tracked = dict[str, int]  # name -> uses (affine, local)
```

Only locally-bound names whose type is non-`Copy` go into the table — and that condition is why affinity and inference cannot be split into two passes. Whether a name is affine depends on its *type*; its type is what inference is in the middle of

computing; so the use-counter must run inside the type checker, at the moment a binding's type becomes known. Every `Var` node bumps a count. A count above one is an error, and, as Chapter 3 described, the counts of a `match`'s arms are *summed* rather than maximised, which is the wart we have been paying for ever since.

Trait bounds ride along on the quantifiers. When `Copy` appears as a bound on a type parameter, generalisation records it on the scheme and instantiation checks it at the use site. `Show` is checked separately, by a later walk over the typed tree, because a `Show` obligation can be discovered after the type is settled. Lark's trait system is small — no inference of instances, no overlapping, no coherence problem — and that smallness is the only reason it fits in the same file as the inference without either of them becoming unreadable.

The output of all this is not a yes or a no. It is a *typed tree*: the shape of Chapter 4's syntax tree, with a `Mono` hung on every expression node and a scheme on every declaration. That tree is the chapter's real product. Chapter 7 needs the types to emit C, and Part II needs them to lower to an intermediate form. It is also why the type checker is three Lark files and not one: they separate cleanly into *what a type is* (`types.lark`), *what a typed tree is* (`tast.lark`), and *the algorithm* (`infer.lark`). Two of the three are pure data. Only the third is Algorithm W.

5.5 What the port found

Here is a Lark program. It reverses a list, in the standard way, with an accumulator. It is polymorphic — it reverses a list of anything — it is fully annotated, and it is the first thing anybody writes when they need a `reverse`:

```
fn rev(xs : List(a), acc : List(a)) : List(a) =
  match xs with
  | Nil        => acc
  | Cons(h, t) => rev(t, Cons(h, acc))
  end
```

The frozen Python type checker does not reject this program. It does not accept it either. It **does not terminate**: it runs until the interpreter's stack gives out and dies with a `RecursionError` inside `apply`, the function whose job is to look a type variable up in a substitution.

We found it by writing it. Nothing in the corpus had ever needed a polymorphic accumulating reverse, so in the entire life of the language nobody had run this program. The port needed one — the code generator accumulates a list of strings and has to turn it round — and the type checker hung.

The cause takes three steps to see, and each of them is somewhere a reasonable person would not look.

One: recursion is monomorphic. When the checker begins a function's body, it puts the function's own name into the local environment bound to a bare unknown — `Scheme((), rec_var)`, a scheme with no quantifiers at all:

```
rec_var = fresh.var()
local_env: Env = {**env, decl.name: Scheme((), rec_var)}
```

So inside `rev`, the name `rev` is not polymorphic. It stands for one unknown, about which every recursive call must agree. That much is not a mistake: it is the classical Hindley–Milner restriction, and inferring polymorphic recursion without help is undecidable [Pierce, 2002]. But now the second half, which is the part that is a mistake. The function's *annotation* does not rescue it. Pass 1.5 dutifully pre-registered the annotated type in the outer environment — and `local_env` shadows it. The declared type is unified with `rec_var` only *after* the body has been inferred. So even a fully annotated function recurses monomorphically, where ML and Haskell would take the signature as a licence to recurse at any instance of it.

Two: the occurs check does not cover `compose`. `bind` refuses to map α to a type containing α, and refuses the degenerate $\alpha \mapsto \alpha$ as well. Both guards live in `bind`, and `bind` is the only place they live. But substitutions are not only built by `bind` — they are also *composed*, and `compose` performs no check whatever. Hand it an s_2 containing $\alpha_5 \mapsto \alpha_{16}$ and an s_1 containing $\alpha_{16} \mapsto \alpha_5$ — two mappings, each perfectly acyclic, each produced by a `bind` that had nothing to object to — and applying s_1 to s_2's range yields

$$\alpha_5 \;\mapsto\; \alpha_5$$

which no check ever sees, because no check runs there.

Three: `apply` chases chains. To apply a substitution to a variable, both implementations look it up and then apply the substitution *again* to whatever came back, because the image may itself contain variables the substitution knows about:

```
case TVar(id=i):
    if i in s:
        return apply(s, s[i])        # chase the chain
    return t
```

A mapping $\alpha \mapsto \alpha$ is a chain with no end.

Put the three together and `rev` is just the shape that produces them. Because recursion is monomorphic, the recursive call's type variable and the annotation's type variable are forced to be identified — and, arriving from two different unifications, they are identified in *both directions*. Because `compose` has no occurs check, the two directions collapse into a self-mapping. Because `apply` chases chains, the checker then walks in a circle until the stack runs out. The occurs check — the very guard whose entire purpose is to stop a type containing itself — is present, correct, and standing in the wrong place.

Three things about this finding are worth stating plainly.

It is **a bug in the oracle**, not in the port. The reference implementation is what we *defined* to be correct, and it is wrong. Chapter 2 promised this could happen and that we would say so when it did.

It **is faithfully reproduced**. `types.lark`'s `apply` chases chains too, and its `compose` checks nothing either, because the port's job is to be indistinguishable from the oracle — here as everywhere. Feed both the program above and both will spin. The differential stays green. This is the failure mode Chapter 2 named: two implementations that agree have not thereby become right, and when the agreement was achieved by copying, agreement is what you would expect either way.

And it **was worked around before it was understood**. The port made its reversers monomorphic — `ecRev` in the code generator, `etcRev` in the optimizing back end, `lwPair` in the lowering pass — each one a general function pinned to a concrete element type for no reason a reader of that file could ever guess. Three separate modules, three separate collisions with the same wall, three local deformations, and no diagnosis until the third. That is what a language limitation actually looks like from the inside. Not an error message: a small shrug, repeated, in code that has no business knowing why.

So what caught it in the end? Not the differential — writing the program did. The port is the most demanding Lark program ever written, and it walked into a corner of the type checker that years of test programs had never approached. *A compiler written in its own language is, among other things, the largest test case you will ever have.* That is not a slogan about self-hosting. It is the single most practical argument for it.

5.6 The repair, and what the repair taught

The account above is the one we could give from the traceback, and it is the one this chapter carried for some time. It is true. It is also, as an explanation, *wrong in its emphasis* — and the thing that showed us so was fixing the bug.

The fix is one line's worth of idea. Pass 1.5 has *already computed* the function's type, from its signature, before any body is checked. Pass 2 then shadows

that entry with a bare unknown and re-derives the type from scratch. So: don't. When a function is fully annotated, bind its recursive occurrence to the **generalised** annotated scheme, and let every recursive call instantiate it afresh — which is what a signature means in ML or Haskell, and just what the undecidability result does *not* forbid. Inferring polymorphic recursion without a signature is undecidable. Honouring one you were given is bookkeeping.

```
rec_var = fresh.var()
fully_annotated = (
    decl.return_type is not None
    and all(p.ann is not None for p in decl.params))
prior = env.get(decl.name)          # pass 1.5 put it there
if fully_annotated and prior is not None:
    outer: Env = {
        k: v for k, v in env.items() if k != decl.name}
    rec_sc = generalise(outer, prior.body)
else:
    rec_sc = Scheme((), rec_var)
local_env: Env = {**env, decl.name: rec_sc}
```

With that, `rev` type-checks, and gets the type it should always have had: `List(a) -> List(a) -> List(a)`. The cycle in `compose` never forms, because the recursive call no longer has to agree with the annotation about a single shared unknown — it takes a fresh copy each time, and there is nothing to tie in a knot.

Now look at what that tells us, because it is not what we thought we knew.

The diagnosis named three causes and gave them equal weight: monomorphic recursion, the unguarded `compose`, the chain-chasing `apply`. From the traceback, all three look equally guilty — each is necessary, remove any one and the hang goes away. But the repair is an experiment, and the parts you did *not* have to change are the control. We changed one. The other two are still there, just as they were, and the bug is gone.

So there were never three causes. There was **one defect and two aggravating conditions**:

- **The defect**: the compiler was given the answer and threw it away. That is the entire bug, and it is a bug about *information flow between passes* — pass 1.5 knows something that pass 2 refuses to use. It has nothing to do with occurs checks, and it would still be a defect in a checker that had no substitutions in it at all.
- **The aggravating conditions**: the missing check in `compose` and the recursion in `apply` do not *cause* anything. They decide what the defect *looks like*. With them, you get a hang. Without them, you would have got "oc-

> curs check failed" on a program that is manifestly well-typed — an absurd message, but a message, pointing at the line, and we would have found this in an afternoon instead of routing around it three times over three months.

That distinction is not visible from the failure. It is only visible from the fix. **Repairing a bug is a way of finding out what it was** — and a diagnosis you have not acted on is a hypothesis wearing a lab coat.

We guarded the aggravating condition anyway, in both implementations, but only as far as we could do it without cheating: `compose` now drops a composed binding $\alpha \mapsto \alpha$. That is sound and total for a reason worth pausing on — $\alpha \mapsto \alpha$ *is* the identity map, so dropping it changes no answer at all; it only ends the chain. It is a belt. The braces are the fix.

How we knew the repair was safe

Changing a type checker is how you break a compiler. And this change was made in *both* implementations at once — oracle and port — which is the one manoeuvre the entire method is supposed to forbid, because two things adjusted toward each other will always agree.

Which is why the evidence is not that the two agree. It is that the two agree *and neither of them moved*. Every differential in this book was re-run with both sides changed, and every one came back on its pinned number:

target	pinned	after the repair
`infertest`	42 / 0 / 3	42 / 0 / 3
`typechecktest`	42 / 0 / 3	42 / 0 / 3
`cektest`	35 / 0 / 14	35 / 0 / 14
`emittest`	38 / 0 / 7	38 / 0 / 7
`lowertest`	35 / 0 / 10	35 / 0 / 10
`emittactest`	32 / 0 / 13	32 / 0 / 13
`opttest`	128 / 0 / 52	128 / 0 / 52

`typechecktest` is the one that carries the weight, because it does not compare types — it compares *the C that comes out the far end*, and the diagnostics for the programs that are refused. Every accepted program still emits byte-identical C. Every rejected program still produces a byte-identical `type error:`. The repair changed no type, no message, and no emitted byte anywhere in the corpus. The only difference it makes is to programs that previously did not terminate, and a program that does not terminate is not in anybody's corpus.

That is what an output-neutral change to a type checker looks like when you can actually demonstrate it, and it is the same move as the red–black tree

earlier in this chapter, one notch harder: there, we changed a representation and asked the differential to certify that nothing moved. Here we changed a *judgement* — which programs the language accepts — and asked it to certify that nothing moved *except what we intended to move*. A test suite tells you the code still works. A differential tells you the code still says the same thing, which is a strictly stronger claim and the only one worth making about a compiler.

What we did not do

We did not remove the workarounds. `ecRev`, `etcRev` and `lwPair` are still pinned to concrete types, and they no longer need to be. They are compiler source, and un-pinning them would change the C the compiler emits for itself and move the fixpoint of Chapter 8 — for no benefit, since the checker that forced them is fixed. So they stay, as archaeology: three fossils of a constraint that no longer exists, in three files that never knew why they were bent.

And we did break the freeze. The oracle was supposed to be immovable, and this is the second time in the book that the port has been right and the reference has been wrong, and the second time the reference has been changed. That is a real cost and it is not the last time it will be paid; the full ledger of everything we have done to the oracle, and the argument for why a tracked reference is the most that was ever actually on offer, is Appendix C. Read it before you believe any number in this book, including the ones on this page.

5.7 The self-application

`make infertest` runs both checkers over the corpus and compares what they say: for every accepted program, the inferred signature of each top-level declaration, in a normalised block; for every rejected one, the error. The pinned result:

`infertest`: **42 ok, 0 fail, 3 skip**

The three skips are the programs that use `import` — the wart from Chapter 4, still unpaid. Everything else is byte-identical, *including the entire error suite*. The programs that must be rejected are rejected by both, with the same message, character for character. This is the first differential in the book that tests failure as well as success, and it can only do so because a type error in the port is a `Result` rather than an exception. A value can be printed. A value can be diffed.

Two of those rejection fixtures deserve a note, because both checkers *accept* programs whose file names promise a failure: a non-exhaustive `match`, and a `match` with no arm that fits. Lark has no exhaustiveness checking. The oracle does not do it, so the port does not do it, so the corpus records the absence

rather than hiding it. A gap you have written down is a different thing from a gap you have not noticed.

And at file number nine in the corpus sits `parse.lark`: the self-hosted parser from the last chapter, a thousand lines of Lark. The self-hosted type checker infers a type for every declaration in it; the Python checker infers the same types; and the two blocks of signatures are the same bytes. A type checker written in Lark, type-checking a parser written in Lark, agreeing with the Python type checker about a program neither of them wrote.

Which leaves the obvious question, and a plain answer. Can `infer.lark` type-check `infer.lark`?

Not in this chapter. It can be *run* on itself, and it is — but what you want out of that is not a block of signatures. It is a compiler, one that checked its own source before compiling it, and getting there needs the code generator of Chapter 7 and the ladder of Chapter 8. Even then, the first compiler to close the loop will skip the type checker altogether: it will lex, parse and emit, trusting its input, because putting Algorithm W on the compiler's own path is a separate milestone with its own difficulties. It does get done. The compiler that finally closes the fixpoint type-checks what it compiles, it type-checks itself, and the C it emits for its own source is identical whether that check runs or not.

Nothing in this chapter's differential proves that will work. It only makes it plausible, which is what a chapter is for.

Chapter 6

The Evaluator, Written in Its Own Language

An interpreter written in the language it interprets is the oldest joke in computer science and one of its deepest tools. Lark's evaluator is a CEK machine — control, environment, continuation — which is to say it is an interpreter with the call stack turned inside out and made into an ordinary value. Writing it in Lark is, unexpectedly, easier than writing the parser was: the parser had to invent a discipline the language did not hand it, and this one only has to name things the language already does.

It also earns its place differently from the chapters around it. The lexer, the parser, the emitter — those are steps on the compiler's own path, the machinery that will eventually compile Lark's source to C. The evaluator is not on that path. It runs a program by walking its tree, not by translating it. What it gives the book is not a stage of the pipeline but a *meaning*: a definite, inspectable answer to the question "what does this program compute?" Every later promise — the optimizer preserves meaning, the checker proves a program safe, the metatheory proves the language sound — is a promise about the answer this machine gives. So it is worth building the answer carefully, and out in the open.

6.1 Why a machine and not a function

The obvious way to write an interpreter is a recursive function. To evaluate `1 + 2`, evaluate the 1, evaluate the 2, add them. To evaluate a function call, evaluate the function, evaluate the argument, apply one to the other. Each of those "evaluate the ..." is a recursive call, and the work-still-to-do — the addition waiting on its two operands, the application waiting on its argument — lives on the host language's call stack, where you cannot see it.

For most purposes that is the right thing to write, and what you should write. The trouble is that the stack is invisible. The record of "where we are in the middle of this computation" is real, but it belongs to the runtime underneath you, not to your program, and you cannot hold it, print it, or reason about it. That becomes a problem the moment you want to say something precise about evaluation rather than merely perform it.

A CEK machine makes the same computation out of parts you *can* hold. It replaces the one recursive function with a *state* and a *step*. The state is a complete description of where the computation is — nothing hidden on a host stack, because the machine carries its own. The step is a single small rewrite: it takes one state to the next. Evaluation is then just stepping until there is nothing left to do. This is what an *operational semantics* is, written as a program you can run: not "here is the answer" but "here is each move on the way to it."

Why go to the trouble here? Two reasons, one now and one later. Now: a machine whose stack is a value is a machine that never overflows the *host's* stack, which matters when the program being interpreted is itself a thousand-line compiler. Later, and this is the real reason: Part III proves things about this machine. Soundness — the promise that a well-typed program does not go wrong — is a statement about what steps a program can take. You cannot prove a property of a stack you are not allowed to name. The evaluator is written as a machine because the machine is the thing the proofs will be about.

6.2 The three registers

The letters are the registers. **C** is the control — what the machine is working on. **E** is the environment — what the names mean right now. **K** is the continuation — everything still waiting to happen once the current work is done. In Lark the state is one type with two shapes:

`SEval(e, env, k, out)`	evaluate expression `e` in environment `env`, with `k` waiting
`SRet(v, k, out)`	hand the finished value `v` back to the waiting continuation `k`

The environment is an association list, pairs of name and value. The continuation is a *list of frames* — and that list is the trick. Each frame is one piece of deferred work with a hole in it: "add this once you have the right operand," "take the else-branch if that condition comes back false," "apply this function once its argument arrives." The call stack that a recursive interpreter would have kept invisibly is here an ordinary `List(Frame)`, and the machine has about nine kinds of frame, one for each place evaluation has to pause and remember something:

`BinOpLF`, `BinOpRF`	evaluating the two sides of an operator
`ApplyFnF`, `ApplyArgF`	evaluating a function, then its argument
`IfF`	holding the two branches while the condition is decided
`LetF`	holding the body while the bound value is computed
`MatchF`	holding the arms while the scrutinee is computed
`TupleF`, `UnaryF`	the remaining elements; a pending unary operator

(There is a fourth thing carried in every state, the `out` string. It is not one of the three classical registers; it is the program's accumulated output, threaded quietly through every step. We come back to it in the last section, where it is the point.)

Nothing about this is abstract once you watch it run. Take the expression `1 + 2 * 3`. The parser has already decided that `*` binds tighter, so the tree is a `+` with `1` on the left and `2 * 3` on the right. The machine starts with that tree in **C**, an empty environment, and an empty continuation. Reading the continuation left-to-right, head first — the head is the innermost pending work — it goes:

	control	**continuation**
1	`eval 1 + 2*3`	(empty)
2	`eval 1`	`[+ _ (2*3)]`
3	`ret 1`	`[+ _ (2*3)]`
4	`eval 2*3`	`[+ 1 _]`
5	`eval 2`	`[* _ 3], [+ 1 _]`
6	`ret 2`	`[* _ 3], [+ 1 _]`
7	`eval 3`	`[* 2 _], [+ 1 _]`
8	`ret 3`	`[* 2 _], [+ 1 _]`
9	`ret 6`	`[+ 1 _]`
10	`ret 7`	(empty)

Read it as breathing. Steps 1–2 push a frame: to add, the machine must first know the left operand, so it sets `1` as the work and remembers "...then add the right side" as a frame. Step 3 finishes the `1` and hands it back; the frame updates to hold the left value and now waits on the right. Evaluating `2 * 3` pushes a second frame on top of the first — the stack *grows* — and then, from step 8, values come back and frames are consumed one at a time: `2` and `3` collapse to `6`, `6` and `1` collapse to `7`, and with the continuation empty the machine has nowhere left to return the `7` to. That is the terminal state, and `7` is the answer. The stack grew and shrank the way a recursive interpreter's hidden stack would have — but here every frame was a value the machine put there and took back, in the open.

6.3 Closures, traits, and the floating-point detour

Most of the port is that mechanical: read the Python `step_eval` and `step_ret`, write the Lark `stepEval` and `stepRet`, check the shapes line up. Three parts were not mechanical, and each says something about the language.

The first is closures. A function value is not just a body; it is a body together with the environment it was born in — otherwise a function that refers to a name from its defining scope forgets what that name meant the moment it is called somewhere else. So the runtime value `RClosure` carries three things: the parameter, the body, and the captured environment. Multi-argument functions add a wrinkle Lark resolves by *currying*: a two-parameter lambda becomes a closure over the first parameter whose body is a lambda over the second. Applying it once yields another closure; applying that gives the result. One argument at a time, all the way down — which is why the machine only ever needs a single-parameter closure, and gets multiple arguments for free.

The second is traits. A trait method is resolved by the runtime type of its argument: `show` on an `Int` and `show` on a list are different code, chosen at the call. The machine carries an `RDispatch` value — a method name still looking for its argument — and when that value finally meets a value to act on, the machine reads the argument's shape and picks the implementation. It is dynamic dispatch with the mechanism made visible: not a hidden table lookup, but a value that resolves itself when it learns what it is being applied to.

The third was a genuine detour, and worth admitting. Lark's own runtime, at the point the evaluator was written, could not turn a floating-point literal into a machine float directly; floats arrive as `RFloat` only by way of a `string_to_float` helper. And one generic list operation had to be specialised by hand, because the self-compiler could not yet settle a `List(a) -> Int` at the site that needed it — so the port pins down the element type at that one call and moves on. Neither is deep, and neither is hidden: they are the kind of small friction that appears whenever a language is made to host itself before every corner of it is finished. The chapter reports them rather than sanding them away, because the sanded-away version would teach you that self-hosting is frictionless, and it is not.

6.4 Reading the world

A pure evaluator computes a value and is done. A useful one has to read input and write output, and that is where the interpreter meets the one thing the rest of Lark works hard to keep out: an effect.

Lark has just one, and it is deliberate. Input and output flow through an `IO` token — a runtime value, `RIO`, that carries the still-unread portion of standard input. To read a line is to consume the token and get back a line together with a

new token holding the rest; to print is to fold the printed text into the output that every state has been carrying all along, in that quiet fourth field. The machine never reaches out to the world in the middle of a step. The world is a value it is handed and hands on.

This is more awkward than an ordinary `print` statement, and the awkwardness is the safety. Because the token is threaded — passed in, consumed, a fresh one passed out — there is no way to use the same world twice. You cannot read the same input line in two places, or interleave two prints and be surprised by the order, because each effect takes the token and the next effect must wait for the one it produces. Effect ordering is not a convention the interpreter is careful to respect; it is a consequence of the token being the only way through. That the type system treats this token as a resource you may not duplicate — the affine discipline the language is named for — is what turns "be careful with effects" into "the machine cannot express the careless version." We are only reading the world here; the promise that it cannot be copied is kept elsewhere, and the book returns to it when it can prove it.

6.5 The same answer, twice

The evaluator is a port, so it is judged the way every port in this book is judged: against the original. The Lark machine and the Python one are run on the same corpus of programs, and their outputs are compared byte for byte. The entire point of an evaluator is that it fixes what a program *means*, and two evaluators that disagree about a single program have failed at the one job they have. They do not disagree. Run over the corpus, `self/lark/cek.lark` matches `self/oracle/cek.py` byte for byte on every one of the roughly three-dozen self-contained programs it runs to completion — zero failures, and that zero is the number that matters. The files it does not compare, it declines rather than guesses at: the error suite has no output to match, a handful of import fragments have no `main` to run, and the few most deeply recursive programs run out the clock, because a meta-circular interpreter — an interpreter running under another interpreter — is slow enough that whether the slowest one or two finish in time depends on how loaded the machine is. So the exact count of programs compared drifts by one between runs; the count that ever *disagree* does not drift. It is zero. A skip that is named and explained is a different thing from a difference swept under the rug.

That agreement is quieter than the parser's and more important than it looks. `self/lark/cek.lark` is now a definition of meaning for Lark that is itself written in Lark, checked against the reference and found identical. When the next chapters claim that the compiled program computes what the source program means, or that the optimizer changes speed and not answers, the meaning they

are preserving is the one this machine gives. The evaluator does not advance the compiler's fixpoint. It supplies the standard against which everything after it will be measured — and the compiler that closes the loop, two chapters from now, will be held to this same standard.

Chapter 7

Emitting C

An interpreter runs a program. A compiler writes a program that runs. The last piece of the port is the code generator: the part that reads a Lark program and writes out C — real, compilable, ugly, fast C. Once this works in Lark, every part of the compiler exists in Lark, and the ladder of Chapter 8 can be climbed.

There is one thing to admit at the outset, because it shapes everything the emitter does. It works from the *syntactic* tree — the one the parser produced in Chapter 4 — and not from the typed tree the checker produced in Chapter 5. It reads structure, names, and literals; it never consults an inferred type. That is an accident of history more than a design: the first compiler to close the loop needed only to lex, parse, and print, so the emitter was built to run on what the parser hands it, and the type checker sits to one side. Part II undoes the accident — its optimizing path runs on a typed intermediate form — but for the purpose of building itself, the compiler discovered it did not need the types to emit correct C, and so this emitter does without them.

7.1 Why C and not machine code

The plain answer is that C is a portable assembler, and writing one more assembler by hand is work the world has already done better than we will.

A code generator that emits real machine code has to choose registers, lay out stack frames, respect one processor's calling convention, and then do all of it again for the next processor. Emitting C hands every one of those problems to a tool that has spent forty years getting them right. The Lark emitter produces C; a C compiler turns that into machine code for whatever the reader is sitting in front of. Portability comes free, a mature optimizer comes free, and the emitter itself shrinks to something you can hold in your head: a few hundred lines that walk a tree and print text.

What you give up is control and directness. You cannot see, in the emitter, which instructions will run — that is now the C compiler's business — and you inherit C's sharp edges, its undefined behaviour and its readiness to believe whatever the generated code claims. You also give up the pleasure of the thing, the feeling of having built the tower all the way to the ground. So the book does not leave it here: Part II goes the rest of the way down, emitting real machine code for a RISC-V processor and doing its own register allocation. This chapter is the pragmatic shortcut that gets a working, self-hosting compiler first; the ground floor comes later, on purpose.

7.2 A tree walk that prints

The emitter's shape is the simplest a compiler stage has: one function per kind of node. `emitExpr` handles expressions, `emitPat` handles patterns, smaller helpers handle arms and declarations. Each one looks at the node it was given, emits some C, and — where the node has children — calls the matching function on each child. A tree walk that, instead of computing a value, prints.

The catch is the buffer. In Python the emitter appends each line to a growing list and hands out fresh variable names from a counter it bumps as it goes. Lark has neither a mutable list nor a mutable counter: nothing can be changed in place. So the state that Python mutates has to be *threaded* instead — passed in, returned modified, passed on. The emitter carries one value, an `Emit`, through every call:

four counters	the next fresh C name for an expression, a pattern, a declaration, an arm — so `t0`, `t1`, `t2` never collide
a line buffer	the C emitted so far, as a list of strings

Every function takes an `Emit` and returns a pair: its result, and the `Emit` as it now stands, with more names spent and more lines added. Where Python wrote `self.emit(line)` and moved on, Lark writes "here is the buffer with one more line in it" and passes it along. It is more verbose and it hides nothing — the emitter's entire state is one value you could print.

That threading is also where the chapter's real trick lives, because the goal is not merely correct C but the *same* C, byte for byte, as the Python emitter produces. The two have to agree on the name of every temporary. Python allocates a node's C name on the way *in* — before it visits the children — but appends the node's own line on the way *out*, after the children have emitted theirs. The Lark emitter does the very same dance: take a fresh name first, thread the children, emit this node's line last. Get that order wrong and the C is still correct and still compiles — it just numbers its temporaries differently, and the byte-for-byte

comparison fails. Matching the reference to the byte means matching not just what it emits but the order in which it decides to.

7.3 Representing Lark values in C

C does not have Lark's values, so the emitter has to build them. Every Lark value becomes a *tagged word*: a single machine word that is either a small integer outright or a pointer to something in memory, with a tag that says which and, for the pointers, what shape sits at the other end — a closure, a constructor, a tuple. A `match` in Lark becomes, in the emitted C, a look at the tag and a branch on what it finds.

Everything that does not fit in a word — the closures, the constructors, the tuples — lives in an arena: one large block of memory that the runtime hands out a piece at a time, moving a pointer forward with each allocation. And it never moves the pointer back. Memory is allocated and never freed. There is no garbage collector, and that absence is deliberate: a collector is a substantial program with subtle invariants, and leaving it out keeps the runtime small enough to read in a sitting. A program that runs, produces its answer, and exits never misses the memory it did not reclaim; the operating system takes it all back at once.

This is a decision with a bill attached, and the bill comes due later. When the compiler is asked to compile something very large — its own source, in Chapter 13 — "allocate and never free" means the arena grows for the entire length of the compile, and a string-join buried in the emitter that looked linear turns out to recopy its output as it goes, so the memory climbs into the gigabytes. None of that is visible on a small program, and none of it is wrong — the C is identical either way. It is a performance characteristic, named here so that when it returns in Part II it arrives as a consequence already on the record, not a surprise.

7.4 Byte-identical C

The test is the one every port in this book faces. The Lark emitter and the Python emitter are run on the same corpus, and their output — C source, not a value this time — is compared byte for byte. Across the corpus, `emit_c.lark` produces C identical to `emit_c_ast.py` on all thirty-eight programs that compile, with none in disagreement. The seven it does not compare are the error suite: programs written to be rejected, which have no C to hold up against anything.

There is one fair asterisk on "byte for byte," and it is worth stating rather than burying. The Python emitter names a wildcard parameter — the `_` you write when you must name an argument but will not use it — from an internal counter that

is not stable between runs. That single token is normalised on both sides before the comparison, because it is genuinely arbitrary on both sides; everything else is compared as emitted. It is the one place the differential looks past a difference, and it does so because the difference is noise in the reference, not signal in the port.

The largest single output is the C for the self-hosted parser of Chapter 4 — a thousand lines of Lark become the better part of a thousand lines of C, and that C is identical, character for character, whether the emitter that produced it was written in Python or in Lark. With that, the port is complete. Every stage of the compiler — the lexer, the parser, the type checker, and now the emitter — exists in Lark, checked against its Python original and found to agree. What has not yet been shown is that these stages, run on their own source, produce a compiler that produces itself. That is the loop, and closing it is the next chapter.

Chapter 8

The Ladder, and the Fixpoint

Everything so far has been preparation. Now assemble the six Lark modules into a single program — a compiler, written in Lark, that reads Lark on standard input and writes C on standard output — and run it on itself. Compile the output. Run *that* on the same source. Compile the output again. If the third compiler emits the same C the second one did, the language has closed a loop that nothing outside itself is holding open.

It does. The three C files are byte-identical.

That is the sum of Part I's claim, reduced to a sentence, and the rest of this chapter is spent earning it back — saying what was assembled, what a stage is, what the byte-identity does and does not prove, and what had to be added at the end to make the claim complete rather than merely true.

8.1 Assembling the compiler

The pieces are already built. The lexer of Chapter 3, the parser of Chapter 4, and the emitter of Chapter 7 each exist in Lark, each checked against its Python original and found to agree. What has not happened yet is to *join* them — to stack the three modules into one program that reads Lark on standard input and writes C on standard output, with no Python anywhere in the path.

The joining is mechanical. The harness (`self/harness/bootstrap.py`) takes the bodies of `self/lark/lex.lark`, `self/lark/parse.lark`, and `self/lark/emit_c.lark`, strips each of its `module` and `import` headers, and concatenates them under a single module. On top it adds a `main` that reads all of standard input, lexes it, parses it, and emits its C. The result is one file — 1,858 lines of Lark, assembled fresh every time so it can never drift from the three modules it is made of. This is the compiler that will be asked to reproduce itself.

The emitter drives off the syntactic tree, so the type checker of Chapter 5 is *not* in this program. It lexes, parses, and prints; it never infers a type. That is a real gap, and the last section of this chapter is about closing it. For now it is worth naming plainly: the first loop this book closes is a loop that trusts its input.

One thing had to change to make even this much possible, and it is the one place in the entire self-hosting effort where the frozen reference was touched. Reading all of standard input at once is not something a Lark program could express — the language had a way to read a line, but not a way to say "give me everything until end of input." So a single primitive, `read_all`, was added to the runtime, on both sides: the Python oracle grew it, and so did the Lark runtime it is checked against. The oracle had been frozen on purpose — it is the fixed point of reference against which every port is judged, and a reference that moves while you measure against it is no reference at all. Unfreezing it, even for one primitive, is the kind of thing to be uneasy about and to record rather than smooth over. It was the right call for a narrow reason: `read_all` is not part of the compiler's logic, it is the compiler's *mouth* — the way the program gets its input at all — and a self-hosting compiler that cannot read its own source in one gulp is missing a plumbing fitting, not an idea. The primitive was added identically on both sides, so the differential it protects still holds: the two implementations were changed in the same way at the same seam, and go on agreeing everywhere it matters.

8.2 Stage 0, 1, 2, 3

There is one distinction that makes the ladder legible, and it trips almost everyone the first time. A compiled program has two histories, not one: what it was compiled *from*, and what it was compiled *by*. The source it was compiled *from* is the same at every rung of this ladder — always the one assembled compiler, the 1,858 lines from the previous section. What changes as you climb is what did the compiling.

Read the ladder from the ground up:

stage	compiled *by*	
0	the Python oracle	The reference emitter, still written in Python, compiles the assembled Lark compiler to C. Link that C against the runtime and you have **stage 1**: the Lark compiler, now an executable.
1	stage 1 (from the oracle)	Feed the same Lark source to stage 1. Out comes `c1.c`. Link it: **stage 2**.
2	stage 2 (from stage 1)	Feed the same source again. Out comes `c2.c`. Link it: **stage 3**.
3	stage 3 (from stage 2)	Feed the source a third time. Out comes `c3.c`.

Stage 1 was born from Python; its C could carry some fingerprint of the oracle that made it. Stage 2 was born from stage 1 — the first executable with no Python in its ancestry at the moment of *its* making. Stage 3 was born from stage 2. The source never varies; only the compiler compiling it does, and after stage 1 that compiler is Lark all the way down.

Now the test writes itself. Compare the three C files the stages emitted. If `c1.c`, `c2.c`, and `c3.c` are the same file, then the Lark compiler, run on its own source, produces a compiler that produces *itself* — and the Python oracle, having lifted the first rung into place, can be kicked away without anything falling. That is what the harness checks, and that is what it finds:

```
c1.c   49a4921c53b2…
c2.c   49a4921c53b2…
c3.c   49a4921c53b2…
```

Three identical hashes. The fixpoint is reached at stage 1 already — stages 2 and 3 only confirm that it holds, that the compiler has stopped changing what it emits and would go on emitting the very same C for as many rungs as anyone cared to build. Why run the extra rung at all, when stage 1 and stage 2 agreeing would seem to settle it? Because stage 2 still had Python one step back in its ancestry, and stage 3 is the first rung compiled by a compiler that was itself compiled with no Python in the loop. The third rung is where the oracle's fingerprint, if there had been one, would have had its last chance to show. It does not show.

8.3 What byte-identical proves

It is worth being careful about what has and has not been shown, because "the compiler reproduces itself" is the kind of sentence that sounds like it proves more than it does.

What the byte-identity establishes is real and specific: the Lark compiler is a fixed point of the map from source to emitted C. Run it on its own source and it produces C source that, compiled and run on that same source again, produces the identical C. The language is expressive enough to describe its own compiler, and that description, executed, is stable — it does not drift, does not slowly diverge from the reference, does not depend on some Python detail smuggled in through stage 0. Everything the compiler decides — every temporary name, every line of output, every branch of every `match` — it decides the same way about its own text whether the hand on the crank is Python's or its own.

What it does *not* establish is that the compiled *binaries* are identical. They are not, and it is instructive to see why. The fixpoint lives in the C *source* — `c1.c`, `c2.c`, `c3.c` match to the byte. Link any of them into an executable, though, and the platform's toolchain stamps the result with things that have nothing to do with the program's meaning. On the machine this was built on, the linker writes a Mach-O binary carrying a freshly minted UUID and the absolute paths of the build directory; build the same C twice, a second apart, and the two binaries differ in those bytes while computing the identical function. The fixpoint is a property of what the compiler *says*, not of the artifact the operating system happens to wrap around it.

This is the territory of *reproducible builds* — the discipline, now taken seriously across the software world, of making a binary a deterministic function of its source, so that anyone can rebuild it and check byte-for-byte that a shipped artifact really came from the source it claims to. The obstacles are the ones just met in miniature: embedded timestamps, embedded paths, embedded UUIDs, hash-table orderings that vary from run to run — meaningless differences that nonetheless break the comparison. Serious projects hunt them down one by one and stamp them out. This book does not go down that road; it stops at the C, where the meaning lives and where the fixpoint is genuine. Naming the line plainly — source reproduces, binary does not, and here is the reason — is worth more to a reader than a stronger claim that would not survive a careful look.

8.4 Closing the last gap

Return to the admission made while assembling the compiler: the program that closed the loop lexed, parsed, and emitted, but did not type-check. It trusted its input. For a compiler being run only on its own known-good source that is

harmless, but it leaves the claim smaller than it should be. A compiler is not only a machine for turning good programs into C; it is also a machine for *refusing* bad ones, and a self-hosting story that quietly drops the refusing half is telling half the truth. So the gap has to be closed: put the type checker back on the path, and show that the Lark checker rejects the same programs the Python checker rejects, with the same message, while still emitting byte-identical C for everything it accepts.

Closing it means assembling a larger compiler. Where the first program stacked three modules, this one stacks six — `self/lark/lex.lark`, `self/lark/parse.lark`, `self/lark/types.lark`, `self/lark/tast.lark`, `self/lark/infer.lark`, and `self/lark/emit_c.lark` — behind a gate. The gate runs the inference of Chapter 5 over the parsed program first. If the program type-checks, its C is emitted as before; if it does not, the compiler prints `type error:` followed by the reason, and emits nothing. The harness for this stronger loop is `self/harness/bootstrap_tc.py`, and it drives the same four-stage ladder — oracle, then stage after stage of Lark — to the same kind of fixpoint. It reaches one: the type-checking compiler reproduces its own source to the byte, just as the emit-only one did.

The verdict that matters is the differential over the corpus. Run the Lark type-checking compiler and the Python reference side by side on the full test suite: 42 programs are accepted or rejected just as the oracle does them — every accepted program emitting byte-identical C, and every rejected one printing the same `type error:` that the reference prints, for the same reason. The compiler now agrees with its reference not only on what good programs mean but on which programs are good.

There is a scar on this one, and it belongs in the record because it is the kind of thing the earlier, smaller loop never had to face. Compiling three modules is a light task; compiling six — and type-checking all of them — is a heavy one, and running that heavy compiler *on its own source* pushed two quiet inefficiencies into the open. A string-join in the emitter that looked linear was quadratic, recopying its growing output on every step; and a helper deep in the type module allocated a structure once for every node and, under the allocate-and-never-free arena of Chapter 7, that meant the memory climbed past twelve gigabytes before the compile could finish. Both were the consequences named earlier in the book arriving on schedule. Both were fixed — a balanced join, a reworked allocation — and, because this is a book about differential testing, both fixes were checked to be *output-neutral*: the corpus still comes out 42 to the byte, the emitted C unchanged, the fixpoint still closed. The compiler was made able to survive itself without being allowed to change what it says.

That is the loop, closed twice: once for a compiler that only emits, and once for a compiler that also judges. The language describes its own translation and

its own discipline, and both descriptions, executed, reproduce themselves. Part I set out to ask whether Lark could build itself; the answer is on the record, hash by identical hash. What it cannot do by these means is vouch for the *compiler that runs the very first stage* — the oracle, and the C compiler beneath it, are trusted, not proved, and no amount of self-hosting reaches back to certify the ground it stands on. That older worry — who compiles the first compiler, and what could hide there — is the thread the interlude picks up next, before Part II turns to a different question entirely: not whether the same meaning can be rebuilt, but whether it can be made *faster*.

Interlude: What the Port Taught Us About the Language

A compiler is the most demanding program you can write in your own language, because it is the one program guaranteed to use every feature. Writing Lark in Lark was, incidentally, the most thorough language review we could have run — and it returned a list of complaints. This interlude collects them.

None of what follows is a bug. Each entry is a place where a decision made early, for good reasons, turned out to have a consequence nobody had priced — the sort of thing that stays invisible until a large, unforgiving program leans on it. The compiler was that program. Here is what it found.

The one token you cannot branch on. Lark threads a single `io` value through a program to stand for the outside world, and the affine discipline of Chapter 5 insists it be used at most once so that no two parts of the program can each believe they hold the live connection to the terminal. Checking "at most once" means counting uses, and the checker counts them across a `match` by *adding up* the arms. That is where the trouble is. Only one arm of a `match` ever runs, so a value used once in each of three arms is, at runtime, used only once — but the checker, summing, sees three uses and refuses. The casualty is `io` itself: you would like to read the world in one branch and write it in another, each branch spending the token once, and the language will not let you, because on paper the sum is two. The workaround is to thread `io` *around* the branch rather than through it. The fix is a small change of arithmetic: over the arms of a `match`, the checker should take the maximum, not the sum — arms are alternatives, not a sequence, and counting them as though they all happen is the one place the affine rule is stricter than the truth.

String equality by hand. Comparing two strings for equality is a primitive you stop noticing you rely on until it is missing. In one of the two runtimes it was, and so the self-hosted compiler could not simply ask whether two strings

were equal — it had to take them apart and compare them piece by piece where it wanted a single `==`. This is the least interesting complaint on the list, because it is not about the language at all: it is a library function that had not been written. The fix is to write it. It earns its place here only as a reminder that "the language" a real program meets is the language *plus* its runtime, and a gap in the second is indistinguishable, from where the programmer sits, from a gap in the first.

A function that cannot recurse at two types. The type inference of Chapter 5 gives Lark polymorphism: write a function once and use it at many types. But there is a fence around it. A function becomes polymorphic only once its definition is finished; *inside* its own body, while it is still being defined, it has one fixed type. So a function cannot call itself at two different types within its own definition. A helper that would like to recurse on its own result — passing, say, a list of pairs where it received a list of elements — cannot, because that would demand it be two types at once before it is allowed to be any. This is a known and deliberate line in the Hindley–Milner world: general polymorphic recursion cannot be inferred (the problem is undecidable), so languages that allow it ask you to write the type down by hand. That is the change — let an explicit type annotation unlock the recursion that inference alone must refuse.

No empty hands, and a word you cannot have. Two small grammar facts, banked together. The first: `fn` is a reserved word, so nothing in your program may be named `fn` — harmless once known, briefly baffling when a parse fails on a name that looks perfectly ordinary. The second: every function must take at least one argument. There is no way to write a function of no arguments, so a computation you want to hold and run later has to be handed a throwaway argument it will ignore, purely to give it a shape the grammar accepts. Both are consequences of keeping the grammar small, and both would be undone the same way — reserve fewer words, and admit an empty parameter list as the plain spelling of "a computation waiting to be asked."

The missing operator. There is no remainder operator. Where another language writes `a % b` to ask what is left after dividing, Lark makes you say it the long way, in terms of the division and multiplication it does have: subtract from `a` the largest multiple of `b` that fits. It is a one-line omission with a one-line fix, and it is on the list for the same reason the others are — not because it was hard, but because writing a whole compiler is what made its absence impossible to keep overlooking.

Read together, the complaints have a shape. Not one of them is a flaw in an idea; every one is a place where an idea was carried out a little too plainly

— the affine rule that counts too faithfully, the inference that draws its fence just where the theory says it must, the grammar kept one notch smaller than convenience would like. A specification review would have caught none of them, because a specification is read by someone looking for what the language *intends*. The compiler was not looking for intentions. It was trying to get its work done, and it pressed on every corner with its whole weight until the corners that did not quite hold gave a little. That is the review only a real program can run, and building the language in itself is how the book arranged to run it.

Part II

The Same Meaning, Faster

Chapter 9

An In-Between Language

The compiler of Part I walks a tree and prints C. That works, and it is slow — not the compiler, but the code it produces. To make the output faster you must first be able to *look* at it, and a tree is a bad thing to look at. So the compiler grows a middle: a small, flat, boring language, in between the tree and the C, designed for nothing except being easy to analyse.

That middle has a name that sounds grander than the thing itself. An *intermediate representation* — an IR — is a form the program passes through on the way from source to output, belonging to neither end. It is not what the programmer wrote and not what the machine runs; it is a deliberately plain restatement of the program, chosen so that a machine can reason about it. And the act of translating the tree down into it has a name too: *lowering*, because each step gives up some of the structure a human needs and gains some of the explicitness a compiler needs. This chapter builds that in-between language and says why it earns its keep. The passes that exploit it come in Chapter 11; the translation into it, in full, in Chapter 10. Here we only need the language, and the reason for it.

9.1 Why a tree resists optimization

Start with a program so small the waste is impossible to miss:

```
fn f(x : Int) : Int =
  (x + 1) * (x + 1)
```

A person reads that and sees the redundancy at once: `x + 1` is computed twice, and it is the same both times, so one of the two is wasted work. The fix is obvious — compute it once, multiply it by itself. The question is why the compiler of Part I cannot see what any reader sees in a second.

The answer is in the shape of what it looks at. The parser handed it a tree, and the tree of this expression is a multiplication with two children, each of which is an addition:

```
        Mul
       /   \
    Add      Add
   /   \    /   \
  x     1  x     1
```

There are two `Add` nodes. They are built the same way and they hold the same things, but they are two different nodes at two different places in the tree, and nothing in the tree records that they compute the same value. The tree is a record of *structure* — what contains what — and structure is the wrong thing to be asking about. "Are these two subtrees equal?" is a question you have to *ask* of a tree, walking both and comparing them node by node; it is not something the tree tells you. And the emitter, walking the tree to print C, never asks. It arrives at the left `Add`, emits code for it, arrives at the right `Add`, and — having no memory that it has seen this shape before — emits code for it again. Two additions, one of them pointless, because the form it is reading makes the pointlessness invisible.

This is not a defect in that emitter; it is a limit of the form. A tree is the right thing for a parser to produce and a type checker to check, because those jobs are about structure and the tree *is* the structure. But an optimizer's questions are different — *is this value ever used? was it already computed? does this name still hold what it held a moment ago?* — and a tree answers none of them without a search. To make those questions cheap, the compiler needs a different form, one where a value computed once has one name, and asking whether two computations are the same is asking whether two lines match.

9.2 Three-address code

The form it uses is called *three-address code*, TAC for short, and the name is a literal description: every instruction does one thing to at most three names. There are no nested expressions. `(x + 1) * (x + 1)` is not one instruction with a subexpression inside a subexpression; it is a short list of instructions, each so simple it could not be broken down further, each naming its result so a later instruction can refer to it.

Here is the same function, lowered. On the left the source; on the right what the compiler produces:

```
fn f(x : Int) : Int =          fn f(x):
```

```
(x + 1) * (x + 1)          t0 = x + 1
                           t1 = x + 1
                           t2 = t0 * t1
                           return t2
```

Every line on the right is an instruction. `t0`, `t1`, `t2` are *temporaries* — freshly invented names, one for each intermediate value, that exist only inside this function. The nesting is gone: where the tree had a multiplication wrapped around two additions, the flat code has three separate steps in the order they run, and the last of them, `t0 * t1`, refers to the earlier two by name instead of containing them.

And now look at what became visible. The redundancy that hid inside the tree is sitting on the surface: `t0 = x + 1` and `t1 = x + 1` are two lines with identical right-hand sides. A pass that wants to remove repeated work no longer has to walk subtrees and compare them; it slides down the list keeping a note of what it has already computed, reaches `t1 = x + 1`, finds `x + 1` in its note, and rewrites every later use of `t1` to say `t0` instead. That pass is Chapter 11's business, not this chapter's — but the reason it is easy to write is entirely this chapter's point. Flattening the tree did not find the redundancy. It made the redundancy *a thing you can see*, and everything Part II does is downstream of that.

The trade has a cost, and it is worth naming so the enthusiasm does not run away with the argument. TAC is longer than the tree, and far less pleasant to read — four lines where the source had one, and a scatter of invented names that mean nothing to a human. You would never write a program this way, and the book is not suggesting the language should look like this. TAC is not for people. Its ugliness is the same ugliness as a disassembled watch: not how you wear it, but how you see what every piece is doing.

9.3 Names, temporaries, and blocks

Straight-line arithmetic is the easy half. A real program branches, and TAC has to express a branch without nesting, because nesting is the thing it gave up. It does this the way a processor does: with *labels* and *jumps*. A label is a name for a position in the code; a jump is an instruction that says "continue from that position instead of the next line." An `if` in the source becomes a test followed by a conditional jump — go here if the test held, there if it did not.

Take absolute value:

```
fn abs(n : Int) : Int =      fn abs(n):
  if n < 0 then 0 - n            t0 = n < 0
```

```
  else n                                  if t0 jump .then1 else
↪ .else2
                                      .then1:
                                          t5 = 0 - n
                                          r4 = t5
                                          jump .end3
                                      .else2:
                                          r4 = n
                                          jump .end3
                                      .end3:
                                          return r4
```

The `if` expression has become five plain instructions and three labels. The test `n < 0` is computed into `t0`; the conditional jump sends control to `.then1` or `.else2`; each branch computes its answer into the *same* temporary, `r4`, and jumps to `.end3`, where the function returns whatever `r4` now holds. Both arms of a Lark `if` produce a value, and both must leave that value in the same place, so that the code after the branch — which does not know which way the branch went — can find it. That shared destination is how an expression-shaped `if`, which *is* a value, survives being flattened into control flow, which is not.

The stretches between labels have a name of their own: a *basic block* is a run of instructions with no label in the middle and no jump until the end — code that, once entered, runs straight through to its last line without control ever leaving or arriving partway. `.then1` above is a basic block; so is `.else2`. Blocks matter because they are the unit of certainty: within one block the compiler knows the instructions run in order, every time, with nothing jumping into the middle, and that certainty is what several of Chapter 11's analyses stand on. This chapter only needs the reader to have met the word.

That is all the vocabulary Part II runs on. A handful of value kinds — temporaries like `t0`, and constants like the literal `1`, an instruction's other possible operand. A dozen or so instruction kinds: assign a value to a temporary, apply a binary or unary operator, call a function, allocate a record on the heap and read its fields, read a constructor's tag, build a closure, return, label, jump, jump-if. Add labels and blocks to bind them into control flow, and there is nothing else. The language is deliberately small; a form you invented to be easy to analyse should not have many parts, or analysing it stops being easy.

9.4 The IR is a language too

It would be fair to ask, after all this, whether TAC is really a language or just a way of drawing the compiler's scratch work. The answer this book gives is that

it is a language, and it treats it as one — with the same discipline every other stage of the compiler is held to.

A language has a syntax, which means it can be printed, and every TAC listing in this chapter is real output from a printer that exists. That printer is not a convenience for the book; it is load-bearing. The entire method of Part I was to write each stage twice — once in Python as the reference, once in Lark — and demand that the two agree byte for byte. The IR gets the same treatment. The lowerer that turns the typed tree into TAC exists in both languages, and the test feeds both the same program and compares the TAC they print, character for character. If the printer did not render TAC as definite text, there would be nothing to compare, and "the Lark lowerer agrees with the Python one" would be a claim with no way to check it. Across the corpus the two agree on every program that runs — thirty-five of them, with none in disagreement, the remainder being imports and programs written to be rejected, which lower to nothing. You can run that comparison yourself with `make lowertest`.

Treating the IR as a language turned up the kind of small telling snag that only appears when you actually build the thing. The Lark port assembles the entire compiler by concatenating its stages into one program, and the IR names one of its value kinds `Val` — the natural word for "a temporary or a constant." But the parser, earlier in the same concatenated program, had already claimed `Val` for the literals it reads out of source. Two stages, one name, one collision, and the only cure available in a language without namespaces is to rename: the IR's version became `Operand`, and the port moved on. It is a trivial fact, and it is just the sort of fact you meet when an intermediate representation stops being a diagram and becomes a program among other programs, subject to the same rules — a language too, with all the mundane obligations of one.

With the middle language built, the two ends can be connected to it. The next chapter lowers the typed tree into this form — the translation only sketched here, done in full, with the interesting cases (pattern matching, closures, dispatch) given the room they need. Then Chapter 11 does to this flat code what no one could do to the tree: reads it, and makes it faster.

Chapter 10

Lowering

"Lowering" is the trade the compiler makes: it gives up structure to gain explicitness. A `match` expression, which a human reads at a glance, becomes a sequence of tag tests and jumps that no human wants to read but every optimizer can. Chapter 9 built the flat language and argued for it; this chapter performs the translation into it, node kind by node kind, and shows what it costs to do the trade without losing anything the program meant.

10.1 From typed tree to flat code

The lowerer is one recursive walk over the tree, and it has one job at every node: emit the instructions that compute the node's value, and return the operand that will hold it. A temporary, or a constant — that is all a subexpression can hand back to whatever contained it, because in flat code nothing contains anything. The walk is the entire pass. There is no cleverness to it; the cleverness is that after it runs, the program is in a form clever things can be done to.

Most nodes lower without incident, and their dullness is the point. A binary operation lowers its two sides, invents a fresh temporary, and emits one line that combines them — three temporaries where the tree had a subtree, the flattening Chapter 9 promised. A literal is a constant and emits nothing. A `let` lowers its value, binds the name to whatever operand came back, and lowers its body — the name never survives into the flat code at all; it was only ever a way for the source to point at a value, and the value now has a temporary's name instead. The straight-line half of the language is like this: tedious, mechanical, done.

The interesting nodes are the ones that are a *value* in the source and must become *control flow* in the target. Chapter 9 showed `if` becoming a conditional jump with both arms writing the same destination. Pattern matching is the same problem, larger. Take an option type and the function that opens it:

```
type Opt =
  | None
  | Some of Int

fn unwrap(o : Opt, d : Int) : Int =
  match o with
  | None    => d
  | Some(x) => x
  end
```

A person reads that as a single choice. Lowered, it is a ladder:

```
fn unwrap(o, d):
    tag4 = tag(o)
    tag_ok5 = tag4 == 'None'
    if tag_ok5 jump .arm2 else .next3
.arm2:
    r0 = d
    jump .match_end1
.next3:
    tag8 = tag(o)
    tag_ok9 = tag8 == 'Some'
    if tag_ok9 jump .tag_ok10 else .next7
.tag_ok10:
    fld11 = o[0]
    jump .sub12
.sub12:
    jump .arm6
.arm6:
    f13 = o[0]
    r0 = f13
    jump .match_end1
.next7:
    call __lark_match_fail()
.match_end1:
    return r0
```

Read it against the source and every piece has a job. Each arm becomes a test that either jumps into the arm's body or falls through to the next arm. The first test reads the scrutinee's tag — `tag(o)` — and compares it against the constructor `None`; on a match, control goes to `.arm2`, which puts the arm's answer into `r0` and jumps to the end. On no match it falls through to the `Some` arm, which checks the tag again, and, because `Some` carries a field, reads that field out with `o[0]`

and binds it to the temporary the arm body knows as `x`. Every arm writes its result to the same `r0`, and `.match_end1` returns whatever `r0` holds — the same trick that let `if` survive flattening, now carrying a choice between two shapes of data rather than two branches of a test.

Two details are worth pausing on because they are true to what a real pass does. The last arm falls through, on no match, to `call __lark_match_fail()` — a trap for a case that cannot arise, because the type checker already proved the match exhaustive. The lowerer emits it anyway. It is belt and braces: cheap to include, and the one thing standing between a future bug elsewhere and a silent wrong answer. And the tag of `o` is read twice, once per arm, into `tag4` and `tag8` — plainly wasteful, and plainly the kind of waste Chapter 9 said the flat form makes visible. The lowerer does not care. Its job is to be correct and mechanical; removing the second read is Chapter 11's job, and it can do it only because the redundancy is now two lines it can compare rather than two subtrees it would have to walk.

10.2 Where the types go

The tree the lowerer walks is not the parser's tree. It is the *typed* tree — the output of the inference of Chapter 5, every node carrying the type the checker gave it. Until now those types have done one thing only: reject programs. A great deal of Part I was spent computing them, and every use of them so far was a use that ended in an error message or in silence. Here, for the first time, the compiler *reads the answers and does something with them*, and the flat code comes out different depending on what they say.

Watch one function that shows three values of three types:

```
fn describe(n : Int, x : Float, b : Bool) : String =
  show(n) + " " + show(x) + " " + show(b)
```

Three calls to `show`, three calls to `+`, one source. Lowered:

```
fn describe(n, x, b):
    t0 = call show(n)
    t1 = call __str_concat(t0, ' ')
    t2 = call __show_float(x)
    t3 = call __str_concat(t1, t2)
    t4 = call __str_concat(t3, ' ')
    t5 = call __show_bool(b)
    t6 = call __str_concat(t4, t5)
    return t6
```

The three `show` calls became three different functions. `show(n)`, where `n` is an integer, stayed `show`; `show(x)` became `__show_float`; `show(b)` became `__show_bool`. Nothing in the source said which; the lowerer read the type off each argument node and picked the runtime function that handles it. The `+` operators went the same way: written between strings, they are not addition at all but `__str_concat`. And an ordinary arithmetic `+` splits on the same seam. Between two integers it is one instruction —

```
fn g(a, b):
    t0 = a + b
    return t0
```

— and between two floats it is a call into the floating-point runtime:

```
fn g(a, b):
    t0 = call __float_add(a, b)
    return t0
```

Identical source, `a + b` both times, and two different programs come out, because the type on the node is different and the lowerer is reading it. This is a small instance of something with a large name: *devirtualisation*. A dynamically-typed language would carry `show` and `+` as decisions made at run time — every call would ask, afresh, what it was handed and branch on the answer. Lark asks once, at compile time, because inference already knows, and bakes the answer into a direct call. The match ladder of the previous section is the same economy in another guise: it tests a constructor tag rather than dispatching through a table, because the set of constructors a value could be was fixed the moment its type was. Everywhere the tree carries a type, the flat code can carry a decision instead of a question. That is what the types were for. The whole of Part I computed them; this is the pass that spends them.

10.3 Lambda lifting

Flat code has no inside. A function is a top-level thing with a name, a list of parameters, and a body of instructions — and TAC offers no way to nest one function within another, because nesting is the structure it gave up. But the source language allows just that: a function may return a function, and the returned one may mention variables belonging to the function that made it. Something has to become of the inside.

```
fn adder(n : Int) : (Int) -> Int =
```

```
  fn(x : Int) => x + n
```

The inner function uses `n`, which is not its own parameter but `adder`'s. Lowered, the inside is gone — lifted out to sit beside `adder` as an equal:

```
fn adder$lam0(env, x):
    cap0 = env[0]
    t1 = x + cap0
    return t1

fn adder(n):
    clos0 = closure(adder$lam0; n)
    return clos0
```

The nameless function got a name, `adder$lam0`, and moved to the top level. It grew a first parameter it did not have in the source, `env`, and its first act is to read `cap0` out of `env[0]` — that is `n`, the variable it captured, retrieved from a record rather than found in an enclosing scope, because there is no longer an enclosing scope to find it in. And `adder` itself no longer returns a function; it returns a *closure* — `closure(adder$lam0; n)` — a record that pairs the lifted function with the values it needs to have closed over, here the single value `n`. Calling the closure later will hand that record back as `env`, and the lifted function will unpack from it what the source once read from the air around it.

Getting there is two steps the lowerer does at every lambda. First it computes the lambda's *free variables* — the names the body uses that are neither its own parameters nor globals visible everywhere — by walking the body and subtracting what each construct binds. `n` is free; `x` is not. Then it lifts: it manufactures a fresh top-level function whose parameters are `env` followed by the lambda's own, prefixes its body with one field read per captured variable, and at the original site emits the closure record holding the captured values in the matching order. Every lifted function takes `env` whether it captures anything or not, so that a caller invoking a closure never has to know or ask — the calling convention is one shape, always. A lambda of several parameters is curried into single-parameter closures before any of this, so the lifter only ever meets the one case it knows how to handle.

10.4 The port, and the bug it found

Everything above describes what the lowerer does. Writing the Lark version of it — the port that has to agree with the reference to the byte — ran the pass into a wall the book has already met once, and it is worth showing where it bit

the second time, because the second time is what turned a local shrug into an understood thing.

The pass threads a great deal of state through a single recursive function. Lowering one expression needs, at once: the expression itself; the environment mapping source names to the operands now holding them; the read-only tables a single scan of the program built, the constructors of each type and the arity of each function; the function being emitted into, which the walk keeps handing back enlarged; and the bookkeeping that outlives any one function, the lambda counter and the growing list of lifted closures. Five things flowing through one call and coming back changed — that is the shape of a lowerer written without mutation, and it is a demanding thing to keep straight.

The wall was not there. It was in a line so small it is embarrassing. Lowering a binary operator, the natural code pairs the two operands into a little list to hand to a helper — two operands, both of the same type, wrapped up. That expression, innocent as it reads, tripped the monomorphic-recursion limit Chapter 5 diagnosed: the two operand values, arriving from two recursive calls to the very function being written, were forced to share a type, and the sharing closed a loop the checker walked until it fell over. The entire account — why recursion is monomorphic, why the annotation did not rescue it, why the occurs check stood in the wrong place — belongs to Chapter 5, which tells it in full. What belongs here is only the local view: from inside the lowering pass it was not a diagnosis but a deformation. The pair got routed through a helper whose signature pins each operand to a concrete type, `lwPair`, and the loop never closed, and the emitted code was unchanged to the byte. A general idea, pinned to one type for no reason a reader of this file could ever guess. Chapter 5 names `lwPair` as one of three such scars in three different modules — the second collision with a wall that was understood only at the third. This is where the second one happened, and what it looked like before anyone knew why.

With the deformation in place the port agrees with the reference. The differential feeds both the same programs and compares the flat code they print, character for character; across the corpus the two match on every program that lowers to anything, thirty-five of them, none in disagreement. You can run it with `make lowertest`. The pass is faithful, the wall is worked around, and the flat code is ready for the thing this part exists to do: read it, and make it faster. That is Chapter 11.

Chapter 11

The Passes

An optimizer is not one clever idea. It is a dozen small, dull, individually unimpressive rewrites, applied over and over until nothing changes. No single one of them is worth explaining to a stranger at a party; together they take a fifth of the instructions out of a program without altering a line of its output, and set up the deeper cuts the backend makes later. This chapter takes them one at a time, in the order the compiler runs them, and shows each firing on a program small enough to hold in your head. Every one operates on the flat code of Chapter 9, because every one asks a question a tree could not answer.

A word on the honesty of the numbers first. These passes make code *smaller* — across the benchmark corpus the instruction count falls by around a fifth from the unoptimized build — and that shrink is real and measurable. The large *speed* wins come later and elsewhere: register allocation and instruction selection in the backend of Chapter 12 cut the instructions a program actually executes by a third again. So this chapter is not where a program gets fast. It is where a program gets *clean*, and where the backend's later work gets most of its opportunities. That is worth doing plainly and without inflation.

11.1 Constant folding

The first pass is the one every other pass imitates, because it is the shape of them all: walk the instructions, find one that can be improved on its own, replace it. Constant folding looks for an instruction whose inputs are both literals — a computation the program will do the same way on every run — and does it now, at compile time, so the program never does it at all.

```
fn f(x : Int) : Int =
  x + (2 + 3 * 4)
```

Lowered, that is three instructions; folded, the multiplication is gone:

```
fn f(x):                    fn f(x):
    t0 = 3 * 4                  t0 = 12
    t1 = 2 + t0                 t1 = 2 + t0
    t2 = x + t1                 t2 = x + t1
    return t2                   return t2
```

`3 * 4` had two literal operands, so it became `12`. `2 + t0` did not: `t0` is a temporary, not a literal, even though we can see it now holds `12`. The fold is *instruction-local* — it judges each line by what is written in it, and `t0` is a name, not a number. Threading the `12` forward into the next line so that `2 + 12` could fold too is a different pass's job, and, as it happens, one Lark does not do: the copy propagation of the next section moves *names* onward, not constants. So Lark stops here, with a named constant, and leaves the last few percent on the table. A more aggressive compiler would chase the constant forward and fold again; this one takes the cheap, certain win and moves on. Every pass in this chapter is like that — a narrow, obviously-correct rewrite, not a clever one — and the power is in the accumulation, not in any single step.

11.2 Copy propagation and common subexpressions

Two passes belong together because they finish each other's work: one stops the program moving a value around under different names, the other stops it computing the same value twice. Return to the example that opened Chapter 9, the one whose whole reason for existing was that a tree could not see its own redundancy:

```
fn f(x : Int) : Int =
  (x + 1) * (x + 1)
```

Chapter 9 promised the flat form would make the waste visible. Here is the pass collecting on that promise:

```
fn f(x):                    fn f(x):
    t0 = x + 1                  t0 = x + 1
    t1 = x + 1                  t2 = t0 * t0
    t2 = t0 * t1                return t2
    return t2
```

Three things happened, in three passes, and it is worth separating them because each is doing one job. *Common-subexpression elimination* slides down the list keeping a note of every expression it has computed; it reaches `t1 = x + 1`, finds `x + 1` already in its note under the name `t0`, and rewrites the line to `t1 = t0` — stop recomputing, just copy. *Copy propagation* then sees that `t1` is merely another name for `t0` and rewrites every later use of `t1` to say `t0` instead, so `t2 = t0 * t1` becomes `t2 = t0 * t0`. And now `t1` is defined and never used, which is the cue for the pass after next: *dead-code elimination* deletes the line, and the redundant addition is gone. What a person saw at a glance and a tree could not see at all, three mechanical passes found by sliding down a list and comparing lines.

One quiet thing is worth stopping on, because it is the kind of soundness argument that only holds in a language built for it. Collapsing two computations into one is safe here even when the computation *allocates* — two calls that each build a fresh heap object become one call whose single object is shared. In most languages that is a bug waiting to happen: share an object, mutate it through one name, and the other name sees the change. Lark is safe from this because its values are immutable and its equality is by value, not by address. Nothing can mutate a shared object, so aliasing is unobservable; and no program can ask whether two values live at the same address, so sharing is undetectable. The pass leans on both facts. Weaken either — add a mutable value, expose an address — and this pass would quietly start producing wrong programs. It does not, because the language does not let it.

11.3 Dead code elimination

The pass just used in passing deserves its own look, because it is the one that has to know the future. Deleting an instruction is only safe if the value it computes is never read again — and "never again," in a program full of branches and jumps, is not something you can read off the line in front of you. You have to look at every path the program might take from here and ask whether any of them reads this value.

That question has a name, *liveness*: a value is *live* at a point if some path forward from that point uses it before overwriting it, and *dead* otherwise. Answering it means treating the function not as a list but as a graph — the *control-flow graph*, whose nodes are the basic blocks of Chapter 9 and whose edges are the jumps between them. Liveness flows *backward* along that graph: a value is live at the end of a block if it is live at the start of any block you can jump to, and the pass pushes that information back through each block until it settles. This is the one place in the entire optimizer that needs the graph and needs a fixed-point

calculation over it; every other pass in this chapter is content to walk a block from top to bottom. So we introduce the machinery here, use it, and set it down.

With liveness in hand the pass is simple. Any instruction that defines a value, has no other effect, and defines something dead — delete it.

```
fn f(x):                    fn f(x):
    t0 = x * x                  t2 = x + 1
    t1 = t0 * x                 return t2
    t2 = x + 1
    return t2
```

The source computed `x * x * x` and never used it. `t2` is returned, so it is live; `t1` feeds nothing live, so it goes; and once `t1` is gone `t0` feeds nothing either, so it goes too — which is why the pass iterates, each deletion possibly exposing the next. Two guards keep it sound: it never deletes a call, because a call might do something the program can observe even if its result is thrown away, and it never deletes a write to a top-level global, because a global is read from other functions the pass is not looking at. Purity is what makes the rest safe to remove: a computation with no effect and no reader has, by definition, no way to matter.

11.4 Inlining

The passes so far each clean up what is in front of them. Inlining is different: it is the pass whose real value is not what it removes but what it *shows the others*. It takes a call to a small function and replaces it with a copy of that function's body, parameters filled in with the arguments — and the point is that the body, now sitting in the open at the call site, is exposed to every pass that runs after.

```
fn sq(n : Int) : Int = n * n
fn f(x : Int) : Int  = x + sq(4)
```

Before inlining, `sq(4)` is opaque — a call, and constant folding cannot see through a call. After:

```
fn f(x):                    fn f(x):
    t0 = call sq(4)             _i0_t0 = 16
    t1 = x + t0                 t0 = _i0_t0
    return t1                   jump .i0_ret
                            .i0_ret:
                                t1 = x + t0
                                return t1
```

Inlining pasted `n * n` in with `n` bound to `4`, giving `4 * 4`, and *then* constant folding — which could do nothing while the call stood in the way — reached in and turned it into `16`. That is the whole argument for the pass. On its own it removes only the overhead of a call and a return; its worth is that const-fold, copy propagation, CSE, and dead-code elimination all run again afterward and now see a bigger straight-line stretch to work on. It is the pass that feeds the others.

It also leaves a mess, and the mess is worth naming rather than hiding. The inlined body arrives wrapped in scaffolding — a copy `t0 = _i0_t0` and a jump to the very next line — because the mechanism that splices a body in has to turn the callee's `return` into "assign the result here and continue," and it does so bluntly. The TAC passes do not tidy it; the jump-to-the-next-line and the redundant copy survive to the backend, where the peephole pass of Chapter 12 sweeps them away. Inlining is greedy about opening work up and careless about the litter, on the correct theory that something downstream cleans litter for a living. Only small, non-recursive functions are inlined — bodies past a dozen instructions are left as calls — so the pasting cannot run away.

11.5 Devirtualisation and closure elimination

Two passes cash in what the types bought. Chapter 10 showed the compiler resolving `show` to a specific function at lowering time, when the type was known right there. But not every dispatch is that easy. A call to a trait method whose implementer is not obvious on the spot lowers to a *dispatch stub* — a little function that reads the argument's constructor tag at run time and jumps to the right implementation. *Devirtualisation* is the pass that removes that run-time question when the answer is, after all, statically knowable.

```
fn go(n : Int) : String =
  describe(Blue)
```

The argument `Blue` is built right there. Its constructor is not a mystery, so the tag test the stub would perform is a foregone conclusion:

```
fn go(n):                    fn go(n):
    t0 = alloc Blue              t0 = alloc Blue
    t1 = call describe(t0)       t1 = call describe$Color(t0)
    return t1                    return t1
```

The generic `describe` became `describe$Color`, the concrete implementation, called directly. The pass earns this without access to the trait tables at all — it

works on flat code, which no longer remembers there were traits. It recognises the dispatch stub by its shape, a tag read followed by a chain of `tag == 'Red'`, `tag == 'Blue'` tests each jumping to an arm that calls some `describe$Type`, and reconstructs the tag-to-implementation map by reading the stub's own body. Then, at any call whose argument was allocated with a known tag in the same block, it looks the answer up and rewrites the call. And — the theme again — the direct call it produces is now something the inliner can inline, which the dispatch stub never was.

Closure elimination does the same trick one level up, for functions rather than methods. A closure that never escapes — built, called, and discarded without being stored or returned — does not need to exist as a heap object at all.

```
fn f(x : Int) : Int =
  let add = fn(y : Int) => y + x in
  add(10)
```

Chapter 10 lowered this to a heap-allocated closure record and an indirect call through it. When the closure never leaves the function, all of that apparatus is waste:

```
fn f(x):                          fn f(x):
    clos0 = closure(f$lam0; x)      _c0_t1 = 10 + x
    t1 = clos0(10)                  ...
    return t1                       return t1
```

The closure allocation and the indirect call are gone; the lifted body, `y + x` with `y` bound to `10`, is spliced straight in as `10 + x`. An allocation avoided and an indirect call turned direct, for a closure the program could not tell the difference about. This is the pass that repays the heap cost of Chapter 10's closure conversion wherever the closure was local enough not to need it — and, run against the corpus, it is the pass that most reduces heap traffic, because higher-order code makes short-lived closures constantly.

11.6 Loop-invariant code motion

Here is a pass that is present, correct, and, on every Lark program ever written, does nothing at all. It is worth including for just that reason.

The idea is textbook. A *loop-invariant* computation is one inside a loop whose inputs do not change from one turn of the loop to the next — so it computes the same value every time, uselessly. The optimization is to *hoist* it: move it out of the loop, above the entry, where it runs once and the loop reads the cached

result. To find such computations you first have to find the loop, which in a control-flow graph means finding a *back-edge* — a jump from a block back to one that dominates it, closing a cycle — and the invariant instructions are the ones inside that cycle whose inputs are all defined outside it.

Lark's control-flow graphs have no back-edges. They never can. A functional language does not repeat work by looping back; it repeats work by *calling itself*, and a recursive call is an ordinary call instruction, not a jump that closes a cycle in the graph. Every function Lark lowers is a directed *acyclic* graph of blocks — an `if` fans out and rejoins, a `match` branches forward, nothing ever loops back. So the pass that looks for cycles finds none, and hoists nothing, on every program in the corpus, at every optimization level.

Why keep it? Because it is correct, and correctness is cheap to keep. The pass is a faithful dominator-based loop finder; it is exercised on a synthetic control-flow graph built to contain a real back-edge, where it hoists the invariant and leaves the variant in place, just as the textbook says. It sits in the pipeline as a capability the language does not currently exercise — a working answer to a question Lark's own programs never ask. There is a small honesty in shipping a pass that does nothing and saying so, rather than either pretending it earns its place or deleting the one piece of the optimizer that would come alive the day the language grew a loop.

11.7 Sweeping to a fixpoint

The passes are ordered, and the order matters, because each one makes work for the others. Devirtualisation exposes a direct call; the inliner inlines it; inlining exposes a constant expression; folding collapses it; the collapse leaves a dead temporary; elimination removes it. Run the passes once, top to bottom, and you catch the first round of this. But the first round creates new opportunities the passes already went by. So the optimizer does not run them once. It runs the full sequence, checks whether the program changed, and if it did, runs them all again — *sweeping* until a pass over everything changes nothing, the program having reached a shape where no pass has anything left to do. That settled shape is a *fixpoint*.

Watch it collapse a higher-order call to nothing. `twice` applies its function argument twice; `go` passes it `inc`:

```
fn twice(f : (Int) -> Int, x : Int) : Int = f(f(x))
fn inc(n : Int) : Int = n + 1
fn go(x : Int) : Int = twice(inc, x)
```

Lowered, `go` builds a closure for `inc` and calls `twice` through it — a call, a closure, and two indirect applications. Optimized to a fixpoint, stripped of its scaffolding, `go` is:

```
fn go(x):
    ...           x + 1
    ...           (that) + 1
    return ...
```

No call to `twice`, no closure for `inc`, no indirect application — just `x` plus one, plus one. No single sweep could reach this. The first inlines `twice`, which exposes the closure it was handed; the next eliminates that closure, which exposes the two applications of `inc`; the next inlines those. Each sweep peels one layer, and only the accumulation reaches the floor. A counter caps the sweeping at eight, a safety net against a pathological program whose opportunities never quite run out; in practice the corpus settles in two or three. Stopping early would only leave optimization undone — never break anything — and that asymmetry is the reason the loop is safe to bound: every pass either shrinks the program or opens it up by a strictly bounded amount, so the process cannot spin forever and cannot go wrong by stopping.

The scaffolding this leaves — the copies and the jumps-to-the-next-line the listing above elides — is real, and it is not this chapter's to remove. It goes to the backend.

11.8 Levels

Every programmer has typed `-O2` and never asked what it selects. Here is the whole of it. An optimization level is nothing but a named set of passes, and the compiler switches on just the ones the level lists:

Level	What it turns on
`-O0`	nothing — the code as lowered, untouched
`-O1`	const-fold, copy propagation, algebraic simplify, CSE, DCE
`-O2`	the above, plus inlining, devirtualisation, LICM
`-O3`	the above, plus closure elimination
`-O4`	the above, plus the backend's register allocator and peephole

`-O0` is not a degenerate case; it is the *ruler*. With no pass enabled the optimizer returns the code just as Chapter 10 lowered it, and that untouched output is the reference every other level is checked against: an optimized build must

produce, on every program in the corpus, byte-for-byte the same program output as its `-O0` build. The passes may rewrite the code however they like; they may not change what it computes. `-O0` is what "the same" means.

Two honesties about the higher levels. The first: `-O2` and `-O3` are where the eight-pass sweep does its work, and their gain over `-O1` is the inlining and closure elimination that open code up. But `-O3`'s bundle names a few passes — unboxing, fusion, arena reuse — that are declared and not yet written, so at present `-O3` is `-O2` plus closure elimination and nothing more. The names are reservations, not features, and it would be a small dishonesty to let the table imply otherwise. The second: `-O4` is where a program finally gets meaningfully *faster* rather than merely smaller, and the reason is that its additions are not TAC passes at all — they are a graph-colouring register allocator and an assembly-level peephole, both of which live in the backend of Chapter 12. The TAC passes of this chapter take about a fifth of the static instructions out of a program. The backend's allocator, keeping hot values in registers instead of shuttling them to memory each turn of a recursion, takes better than a third off the instructions a program actually runs. This chapter cleans; the next one, with the clean code this one hands it, is where the speed is.

Chapter 12

The Back End

The optimized IR still has to become something a processor will run. There are two ways down, and this book takes both: emit C and let a C compiler finish the job, or emit machine instructions directly and take responsibility for the registers yourself. The first is what we ship. The second is what teaches you what the first is hiding. And between them sits a bug that neither path could have found alone — a bug that only appeared when the compiler was made to compile itself. That bug is the spine of this chapter; the two backends are the shoulders it hangs from.

12.1 TAC to C, again

There are now two C emitters in this compiler, and it is fair to ask why. The first, in Chapter 7, reads the *syntactic* tree the parser produced and links its output against a hand-written runtime; it was built to close the bootstrap loop and knows nothing of types or optimization. The second, `optimize/oracle/emit_tac_c.py` and its Lark port `optimize/lark/emit_tac_c.lark`, reads the *flat* code of Chapter 9 after the passes of Chapter 11 have run over it. It sits at the bottom of the optimizing pipeline:

```
parse -> typecheck -> lower -> optimize -> emit C
```

The reason for a second emitter is not redundancy but *level*. The first emitter works from a tree, where a single source expression is still a nested shape; it must walk that shape and invent the temporaries and control flow C wants. The optimizer has already done that work — flattening, folding, threading temporaries, deleting the dead — and thrown away nothing a C compiler could use. Emitting

C from the optimized IR is therefore almost a transcription: each three-address instruction becomes one line of C. Here is `sq(x) = x * x`, taken verbatim from the emitter's output at O1:

```
static lkw lk_sq(lkw v_x) {
  lkw v_t0 = 0;
  v_t0 = (v_x * v_x);
  return v_t0;
  return (lkw)0;
}
```

Every temporary the optimizer named becomes a local; every instruction becomes an assignment; the machine word is `lkw`, a full-width integer wide enough to hold either a boxed pointer or a raw function address. The second `return` is unreachable — a trap the emitter plants after every function body so that a control-flow path the exhaustiveness checker already proved dead still has somewhere to land in C, which does not know it is dead. It is the same belt-and-braces reflex as the match-failure trap of Chapter 10, and the same honesty: the compiler emits the guard it can prove it will never need, because the cost is one line and the alternative is a warning from clang.

One deliberate departure from Chapter 7's emitter is worth naming. That one linked against a shared runtime; this one inlines everything — the bump heap, the string routines, the float and show and I/O helpers — into a single self-contained C file that compiles with a bare `clang out.c -o out` and no link step. The two emitters are independent witnesses to the same meaning, reached from two different intermediate forms, and each is differentially checked against the interpreter. That independence is about to matter.

12.2 The real machine, briefly

The C emitter never sees a register. It writes `v_t0`, `v_t1`, `v_x` and leaves it to clang to decide which of those live in the processor's registers and which spill to the stack. The second way down — the one that emits RISC-V assembly directly — has to make those decisions itself, and doing so is the clearest possible window into the work the C compiler was quietly doing on our behalf. Here is the same `sq` function, now as assembly the compiler emitted for a real board:

```
sq:
  ...
  mv   s11, a0
  mv   t0, s11
```

```
mv   t1, s11
mul  t2, t0, t1
mv   s10, t2
mv   a0, s10
...
```

The multiply is one instruction; the five moves around it are the price of not being clever. The argument arrives in `a0`, is copied to `s11`, copied again into both `t0` and `t1` so the multiply can read it twice, and the result is walked back out through `s10` to `a0`. A person would have written `mul a0, a0, a0`. The gap between those two is what register allocation is for.

The allocator that produced this is linear scan (`optimize/oracle/regalloc.py`, after Poletto and Sarkar): lay the instructions out in a line, compute each value's live interval — the span from its first definition to its last use — and walk the intervals in order of start point, handing each the first of eleven callee-saved registers not currently busy. When they run out, spill the interval that ends furthest away. It is one pass, it is fast, and it never reconsiders. Its weakness is on display above: it never notices that `s11` and `t0` hold the same value and could share a register, so the copies survive.

The textbook cure is to see allocation as *graph colouring* (`optimize/oracle/coloring.py`). Build an interference graph: one node per value, an edge between any two values that are ever live at the same moment. A colouring of that graph with k colours — assigning colours so no edge joins two nodes of the same colour — is a register assignment using k registers, and a node that cannot be coloured is one that must be spilled. Because the graph also records which values are copies of each other, the colourer can *coalesce* them — give a value and its copy the same register on purpose — and the redundant `mv` collapses to `mv sX, sX`, which the later peephole deletes. Chaitin's observation was that this problem is NP-complete in general and yet routine in practice, because the graphs real programs produce are not adversarial. Three pages of graph algorithm buy back what linear scan left on the floor — and, more to the point of this book, they show you the shape of the thing clang does for the C path in silence. This is a sidebar, not a second compiler: the exercises at the end of the chapter ask you to port the colourer, and it is there that the one genuinely new idea in it comes out.

12.3 A bug the fixpoint found

Here is where the two backends earn their keep, because a fault lived in the gap between them and nothing short of the compiler compiling itself would route a program through it.

In the flat code, two different source constructs arrive wearing the same clothes. A `match` on a constructor lowers to a tag test — read the value's integer tag, compare it against a constant — and shows up as `t == 3`. A comparison of two strings, `name == "let"`, also lowers to `t == <const>`. The TAC-to-C emitter, reading only the instruction, saw an equality against a constant and compiled it the one way it knew: an integer comparison. For a genuine tag test that is correct. For a string comparison it is nonsense — it compares a heap pointer against a small integer, and the two are never equal.

The one place this mattered was the lexer. Deep in it, a function decides whether an identifier is actually a keyword by comparing it against `"fn"`, `"let"`, `"if"`, and the rest. Compiled through the native path, every one of those comparisons was an integer test that always failed, so every keyword was classified as an ordinary name. Feed the parser a token stream in which no keyword is a keyword and it descends into a production that can never complete, calls itself, and does it again — unbounded recursion, a blown C stack, and a segmentation fault inside a generated function called `lk_pDecls`.

Notice what did *not* catch this. The interpreter of Part I compares strings correctly, so the differential test against the interpreter passed. The first C emitter of Chapter 7 was never on this path. Every check the book had built so far held two things against each other and looked for a disagreement, and here the two things that disagreed — the interpreter and the native backend — were never directly compared, because the pipeline that used the native backend was the *optimizing self-compile*, and until the compiler was pointed at its own source, no test program in the corpus exercised a dynamic string comparison in native code at all. The bug needed a specimen large and self-referential enough to use every feature the compiler itself uses. The only such specimen is the compiler.

So the fixpoint of Chapter 8 found it: compile the compiler with the compiler, run the result, watch the self-built parser loop and die. And because the fault was in emitted C and not in any Lark source, it was run to ground with a C debugger pointed at generated code — stepping through `lk_pDecls` until the integer comparison that should have been a string comparison stood exposed. The fix disambiguates the two cases by data flow: thread through the emitter, per function, the set of temporaries that are the destination of a get-tag instruction. If the compared temporary is one of those, it is a genuine tag test and stays an integer comparison; otherwise it is a string comparison and lowers to a call to the string-equality routine. The same correction went into both the oracle emitter and its Lark port, so the two stay byte-for-byte in step.

The lesson generalises past the incident. Differential testing finds any bug that makes two implementations disagree, and it is worth every bit of the trust this book has placed in it. But it is blind to a fault that lives in a path neither of the two witnesses travels, and a compiler has more paths than any small test

exercises. The fixpoint is a different instrument. It does not compare two things; it feeds the compiler the one input guaranteed to use every construct the compiler is built from, and lets the program fall over if any of them is wrong. It is the largest test a compiler can be given, and it is free, because the compiler was lying around.

12.4 The last link

Every check in this book so far has had the same shape. Two implementations of the same idea are made to disagree, and where they do not disagree, we call the thing correct. The lexer against the oracle's lexer, the optimizer against the unoptimized run, the compiler against itself across a fixpoint. It is a powerful method — it found every bug in Part I — and it has a ceiling, which this chapter is where you finally hit.

The ceiling is this: *agreement between two implementations is evidence about their relationship, not about the world.* Our back end emits RISC-V assembly and so does the oracle, and the two agree byte for byte on every program in the corpus. What that establishes is that the port was faithful. It cannot establish that either is right about the processor, and the reason is that the two are not independent witnesses: one was translated from the other, so they inherit each other's misunderstandings along with each other's logic. A frame layout off by one word, a spill to the wrong stack slot, a tail call that quietly clobbers a register the caller wanted back — every one of these survives byte-identity, and every one of them is invisible to an emulator that was written from the same assumptions as the compiler. Differential testing cannot see a shared blind spot. It can only see a difference, and there is no difference to see.

There is one authority on what a processor does with an instruction, and it is that processor. So at the end of a chain of models, each checked against the model beside it, you eventually have to walk out of the building and consult the thing itself.

We did, and the nine programs printed on a real board what the interpreter in Part I says they should print — including the parser, which is to say Lark's own parser running on a microcontroller with a couple of hundred kilobytes of memory, compiled by a compiler written in Lark, producing the same trees it produces on a workstation. The allocator survives contact with silicon. That claim is the only one in this chapter that no amount of further testing on a workstation could have produced, and it is why the last link has to be a physical one.

The instrument and the specimen

Getting that sentence took a day, and almost none of the day was spent on the compiler. The board was silent — not wrong, silent — and the hypotheses arrived in order of glamour: a code generation bug, a linker script, a hardware fault, a subtlety of the processor. Each was investigated. Each was innocent. The fault was in the program that was *reading* the board, which was discarding the output before it looked at it, for reasons that are a matter of a default argument in a standard library and are of no interest to anyone.

The generalisation is worth more than the incident, and there are two.

The first is that **a test that has never passed is not evidence about the system under test. It is evidence about itself.** This is the oldest rule in experimental work and it does not survive the trip into software, where we are used to tests that are born working and only later break. A new harness is an uncalibrated instrument. Its first reading is a claim about two things at once — the specimen and the apparatus — and until it has been made to produce a known result, you cannot tell which of the two it is telling you about. We had a harness whose entire life history was a single observation, silence, and we spent a day reading that silence as a statement about a microcontroller.

The second is the move that ended it, which is the cure for the first: **run the new instrument against a known-good specimen before you trust what it says about an unknown one.** We flashed an old image, one built months earlier by a pipeline we trusted and verified on that same board at the time. It was silent too — and a program known to work cannot be a bug in the compiler that did not produce it. That one observation collapsed the search inward from the silicon to the toolchain to the compiler to the apparatus, in about ninety seconds, having been available from the first minute.

Which is, of course, the method of this entire book, turned around and pointed at the method itself. Hold something you are sure of against the thing you are unsure of, and let the difference speak. It works on compilers. It works on the tools you use to test compilers. The only mistake is to assume you know, in advance, which of the two you are currently holding.

Exercises

12.1: Read the spills Compile `03_primes.lark` to assembly and find the stack traffic. The linear-scan allocator in `optimize/oracle/regalloc.py` assigns registers by scanning live intervals in order of start point, and when it runs out it spills the interval that ends last. Identify one value that is spilled and reloaded, and say what a human would have done instead. You are not asked to fix it — only to see it, because everything that follows is motivated by what you find here.

12.2: Colouring, on paper The interference graph of a function has one node per value and an edge between any two values that are live at the same time; a k-colouring of it is a register assignment using k registers, and a value that cannot be coloured must be spilled. Build the graph for `sum_to` by hand from the liveness information the optimizer already computes. How many registers does it need? Compare that with what linear scan gives it. (Chaitin's insight was that this is graph colouring, which is NP-complete in general and yet perfectly tractable in practice, because the graphs that compilers produce are not adversarial.)

12.3: The allocator, ported — large `optimize/oracle/coloring.py` and `optimize/oracle/igraph.py` implement what the previous exercise did on paper. Port them to Lark and swap the result in at the allocator hook, so that the self-hosted back end can generate code either way.

This is the largest exercise in the book, and it differs from every port that came before it in one respect that is worth dwelling on. Every other component in these three parts was checked by demanding *byte-identical* output against the oracle — and a colouring allocator, correctly ported, will not give you that. It will assign different registers, spill different values, and emit different assembly than linear scan does, and it will be right. The differential you need is therefore a weaker one, and it is the first genuinely interesting test in this project: two allocators must produce *different assembly with identical behaviour*.

Which is to say the exercise quietly asks you to give up the standard this book has been built on, and to notice what you get in exchange. Byte-identity was never the goal; it was a proxy for behavioural equivalence that happened to be free. Here it stops being free, and you have to say what you actually meant.

12.4: Peepholes After the allocator runs, the instruction stream contains moves from a register to itself, loads of values that were just stored, and branches to the next instruction. Write a pass that removes them, and confirm on the board that the programs still print what they printed before. Then ask the question this chapter asked: how would you know if it were wrong?

Chapter 13

The Optimizer Optimizes Itself

Part I ended with a compiler that could compile itself. Part II ends with a harder claim: an *optimizing* compiler, written in Lark, that optimizes its own source code — and produces, from the optimized version of itself, the same code the unoptimized version produced. Same meaning, faster machine. That is the promise of optimization entire, and here it is tested against the most demanding input available: the optimizer.

13.1 Two claims, not one

It is easy to say "the optimizing compiler works," and the sentence hides two different promises that are worth pulling apart, because they are tested differently and they fail differently.

The first is about *meaning*. The optimizer was written twice — once in Python, as the reference of Chapters 9 through 11, and once in Lark, as the port that ships. The first claim is that the two agree: the Lark-written optimizer, run over a program, produces the same code the Python-written optimizer does, at every optimization level. This is a claim about whether the port is faithful, and when it fails it fails as a *difference* — a byte in the generated C where the two optimizers disagree, which is to say a pass that was ported subtly wrong: a constant folded one way here and another way there, a temporary spilled differently, two instructions emitted in the other order.

The second is about *scale*. It is the claim that the optimizing compiler can be run on its own source — all nine of its modules — and produce working C, and not merely once but as a *fixpoint*: the C it emits, compiled and pointed back at the same source, emits the identical C again. This is a claim about whether the compiler *runs* on the largest and least forgiving input in the building, itself, and when it fails it does not fail as a difference. It fails as a crash or a resource cliff

— the compiler loops, or it asks for more memory than the machine has. The first claim can be green while the second is nowhere near, because a pass can be perfectly faithful on every small program and still bring the machine to its knees when the program is the compiler.

Keep them apart. The rest of the chapter is one section for each.

13.2 The first claim, green

Every differential test before this one cheated a little, and the cheat is worth naming. To check the Lark lexer we fed it source and compared its tokens against the oracle's; to check the Lark optimizer we fed it a serialised intermediate form the *Python* pipeline had produced, ran one pass, and compared. Each module was checked in isolation, with the oracle standing in for every module upstream of it. That is a fair test of one module and a blind spot for the seams between them.

The first-claim test closes the seams. It assembles all nine modules — lexer, parser, type checker, the typed tree, inference, the flat-code datatype, the lowering, the optimizer, the C emitter — under a single `module Selfhost`, embeds one corpus program as a string, and runs it end to end, printing the C for that program at a chosen level. Nothing from Python is in the loop: the typed tree that inference produces is the one the lowering consumes, the flat code the lowering emits is the one the optimizer rewrites, and the seam that every earlier test had bridged with the oracle is now bridged by the compiler itself. Against that, the harness runs the Python reference over the same file at the same level, and demands the two C sources be *byte-identical* — for every single-file program in the corpus, at every level.

They are (`optimize/harness/optcompiler_difftest.py`). A byte-for-byte equality over generated C, held at every optimization level, is about as unforgiving as a test of an optimizer can be: there is nowhere for a faithless pass to hide, because a single instruction reordered or a single constant folded differently is a differing byte, and the diff points straight at it. That the port clears it at every level is the first claim, and it is the easy one — easy to state, easy to check, and, because the port was built pass by pass against the reference, green by the time it was assembled.

13.3 The second claim, and the wall

The second claim is the one that fought back.

Point the optimizing compiler at its own source — some seven thousand lines of Lark — and it emits about a megabyte and a half of C, tens of thousands of

lines of it. Compile that C, and you have a new optimizing compiler, a native one. Run *it* on the same source and it emits C again; run the compiler that C builds, once more. The three outputs are byte-identical. That is the self-application fixpoint: the compiler has reached a place where compiling itself changes nothing, because there is nothing left to change. And the optimizer earns its name on the way through — pointed at its own source, it emits eighteen percent less C than the unoptimized build does from the same input. Same compiler, same behavior, a fifth less code.

Getting there hit a wall, and the wall is instructive because it was not a bug at all. Wherever the self-compile finished, its output was correct. But at any optimization level above zero, it did not finish — it consumed twelve gigabytes of memory in about two seconds and fell over, and it did so *before a single optimization pass had run*. The cost was not in the optimizing. It was in the bookkeeping the optimizer does to know when to stop.

An optimizer runs its passes in a loop, sweeping until a sweep changes nothing. To detect "nothing changed," this one rendered the entire program — all 1.8 megabytes of flat code — to a string, and compared that string against the previous sweep's string, twice per sweep. On a small program that is free. On a program the size of the compiler it is ruinous, because building a string by appending to it, in a language with no mutable buffer, copies the entire accumulated string on every append: the work to build a string of length n is proportional to n^2, and when n is a megabyte and a half the n^2 term is counted in gigabytes. The fixpoint check — the one piece of the optimizer whose only job is to compare, and which produces no output at all — was allocating more memory than everything else combined.

The fix did not touch the passes and did not touch a byte of the output. Instead of rendering both programs to text and comparing the text, compare the programs *structurally*: walk the two trees, function by function, and stop at the first place they differ. That is linear in the program and allocates essentially nothing, and it turned a twelve-gigabyte overflow into 5.6 gigabytes that runs to completion — with the byte-for-byte differential of the previous section still green, because comparing two things is not supposed to change either of them, and now it doesn't.

Notice the shape of what remains. 5.6 gigabytes is a workstation number. The board of Chapter 12 has half a megabyte. The wall was lowered, not removed — the compiler can compile itself on a laptop and could not begin to on the microcontroller its own output runs on. And this was the *second* time in the project that an operation which looked like appending to a list turned out to be copying it, at a scale where the difference is the difference between running and not. The first time taught nothing durable enough to prevent the second. That is worth an interlude of its own, and it gets one.

Interlude: The Quadratic Wall

More than once in this project, a program that was plainly correct became unusable at scale for one reason: an operation that looked like appending to a collection was, underneath, copying the collection wholesale. It happened in the emitter, joining lines of output. It happened again in the type checker, threading a substitution. It happened a third time in the optimizer of Chapter 13, deciding whether a sweep had changed anything. Each was written by people who had already met the last one. This interlude is about why purely functional code keeps setting this trap, and how to recognise it before the machine runs out of memory.

An append that is a copy

The trouble begins with a fact so ordinary it is invisible. In a language with mutable memory, appending one item to a list is a constant-time act: find the end, write one cell. In a language without it — a language where a list, once made, never changes — there is no end to write to. To "append" is to build a new list that has the old one's contents *and* the new item, and building it means walking the old one. Appending to a list of length n costs n. Do it once, and nobody notices. Do it inside a loop, appending as you go, and the cost is $1 + 2 + \cdots + n$, which is $n^2/2$: the same quadratic that turns a program that runs in a second on a small input into one that exhausts the machine on a large one, with no error, no wrong answer, and nothing to point at.

The same shape hides in string building (a string is a list of characters) and in any structure you grow by re-deriving it. And it is invisible under the very conditions this book works under, because the reference implementations are written in Python, where a list *does* have a mutable end and a dictionary *does* have constant-time lookup. The Python optimizer builds its output by appending to a list and joining it once; it interns its string constants in a `dict`. Both are $O(1)$ operations that CPython performs in place, and so the quadratic is simply absent

from the oracle. It appears only in the port — and it appears there not as a bug the port introduced, but as a cost the oracle was hiding all along.

The three faces

The first face was the emitter's line join. Assembling the output C meant gluing thousands of lines together with newlines, and the natural recursive form — *this line, then a newline, then the join of the rest* — re-copies the entire growing tail at every step. On the emitter's own output it drove the arena past three gigabytes. Rewriting the join to combine lines in balanced pairs, halving the count at each level instead of peeling one line at a time, brought the same output down to under two hundred megabytes. Not one byte of the C changed; concatenation does not care how you parenthesise it.

The second face was subtler and lived in type inference. The checker threads a substitution — a growing map from type variables to types — and to apply it, it walked the map end to end, once per node of the program, as the map itself grew with the program. Two quadratics, braided. The reference hid both behind a dictionary; the port, carrying the substitution as an association list, paid for both, and the self-compile overflowed twelve gigabytes. The cure was to stop scanning: a closed-over piece of the map can be recognised and skipped whole, and the map itself became a balanced search tree, so a lookup that had been a walk became a descent. Again the output did not move by a byte.

The third face is the one Chapter 13 told: to know when to stop sweeping, the optimizer rendered the entire program to a string and compared it against the previous sweep's string — the append-is-a-copy trap wearing the mask of a mere equality check, in the one part of the optimizer that produces no output at all. Comparing the two programs by walking their structure and stopping at the first difference, instead of rendering both, retired it.

Three subsystems, three authors' worth of hindsight, one mistake. That is the measure of how well it hides.

The constant that proves it

The reason it hides is that it does not look like a bug; it looks like a program that needs more memory. So the useful question is how to tell quadratic *work* from a genuine leak, and there is a clean test. A leak grows with what the program keeps *alive*: hold on to more, and memory climbs in proportion. Quadratic work grows with something else — with the total churn, the corpses of intermediate values that were dead the instant after they were copied — and it climbs as the *square* of the output size. Measure both, and the two hypotheses separate.

We measured. Across the compiler's self-compilation, over a range of input sizes spanning a hundred and eighty to one, the peak heap divided by the square of the emitted bytes stayed near a single constant, around 0.025. A ratio that holds steady across that span is not a coincidence and it is not a leak; a constant of heap/output^2 *is* the statement that the work is quadratic in the output. The live set, measured alongside, was linear throughout. Nothing was being retained that should have been freed. Memory was being spent, in enormous quantity, on values that were already garbage — generated, copied past, and abandoned, faster than any amount of freeing could have kept up with, because the fault was in the generating, not the keeping.

Does Lark need a garbage collector?

A reader who has followed the memory numbers — three gigabytes, twelve, five and a half — will by now be asking the obvious question, and it deserves a straight answer. The instinct is that a language which piles up this much dead memory must need a collector to sweep it away. The instinct is wrong, and the measurement above is why. A tracing garbage collector earns its keep when a program cannot tell which of the things it is holding are still needed — when liveness is tangled and must be discovered by chasing pointers. That was never our situation. The live set was linear and known; the heap filled not with things we might still need but with things provably finished the moment after they were made. Collecting them would have worked, but at the price of the one property the language was built to keep: determinism, the absence of a pause whose timing you do not control.

There is a better answer, and it is the one the type system of Chapter 5 already points at. Lark's affine discipline tracks how many times each value may be used, which is to say it already knows when a value's last use has gone by. A value known to be finished need not wait for a collector to notice; it can be reclaimed — or, better, its memory reused in place for the value that succeeds it — at a moment the compiler can name, deterministically, with no scan of the heap. Reset an arena at the boundary of a compiler phase; reuse an affine-dead cell for the record about to replace it. This is not garbage collection made faster; it is garbage collection made unnecessary, by a language that knows enough about ownership never to have generated the garbage as a mystery in the first place. It is also the doorway into Part III, where knowing what a program is allowed to do with its values stops being a performance trick and becomes the subject itself.

Part III

What the Machine Can Promise

Chapter 14

What "Correct" Could Mean

The compiler passes its tests. Every one of them. And that tells you almost nothing, because the tests were written by the same person who wrote the bugs. This part of the book is about the other kind of assurance — the kind that does not depend on having thought of the right input — and about how much of it is actually within reach.

The job of this chapter is to lower expectations to the right level and no lower: to lay out, before any proof is shown, what a proof can buy and what it cannot, what testing already bought and where its coin stops spending, and which of the grand-sounding guarantees a language might offer this one actually holds. The chapters after it deliver; this one draws the map, so that when a later chapter says *proved*, the word arrives already sized.

14.1 Tests, and their ceiling

Parts I and II leaned on one technique above all others, and it was a good one: *differential testing*. Run the new thing and a trusted reference on the same input; compare their answers; a difference is a bug. It is how the self-hosted compiler was trusted — three stages emitting byte-identical C — and how the type checker was trusted against its Python original, program by program. Differential testing is cheap, it is ruthless, and it caught nearly everything this book caught.

But state what it actually establishes, because the true statement is narrower than the feeling of a green suite. A passing differential test says: *on the inputs we tried, the new implementation could not be told apart from the reference.* Not that it is correct — the reference could be wrong, and then both agree and both are wrong together. Not that it agrees everywhere — only on the inputs someone thought to supply. The ceiling of testing is the imagination of the person

choosing the inputs, and above that ceiling lies the region where the interesting bugs live, because a bug you imagined, you would have tested for.

A specimen, found late and almost too neat. The refinement work of the coming chapters runs on a fork with two evaluators: the reference interpreter, and a second runtime compiled to C. For the fork's whole life the two produced identical answers on every test — genuinely identical, not approximately. Then the reference interpreter gained two small built-in operations, and the compiled runtime, which sat in no test target, silently did not gain them. Two evaluators, in lockstep for the length of the project, now disagreed on those two operations — and every suite stayed green from the day of the change onward, because no test ever drove those operations down the compiled path. The disagreement was found because a person went looking, not because a harness failed; it was fixed, and the compiled path checked against the interpreter to confirm the two now matched. The lesson is worth keeping past this one bug: a correctness claim covers the paths a harness drives, and a runtime no target exercises is not covered by the claim — it is hiding behind it. Green does not mean correct. It means nothing has yet contradicted you where you happened to look.

14.2 Three things one might prove

“Prove the compiler correct” is a sentence that hides at least three different projects of wildly different cost, and confusing them is how a modest result gets oversold and an expensive one gets waved at. Separate them.

The first is that *the type system is sound*: every program the type checker accepts will run without hitting a certain class of failure — it will not, part way through, find itself trying to add a number to a function or branch on a value that is not a boolean. This is a claim about the *language*, proved once, covering every program anyone will ever write in it. It is the cheapest of the three, and Lark has it.

The second is that *the compiler preserves meaning*: the program the machine runs computes what the source said, so that nothing is lost or altered in the translation from Lark to C to instructions. This is a *verified compiler*, and it is a different order of undertaking — famously, one of the largest proof efforts ever mounted for a mainstream language spent years on just this claim for a C compiler. Lark does not have it. The C backend is trusted, as Chapter 8 was careful to say.

The third is that *a particular program meets its specification*: not that the language is safe or the compiler faithful, but that *this* sort function sorts, *this* parser accepts the right grammar. This is per-program, it needs a specification to check against, and its cost scales with how much you want to promise. The refinement checker of Chapter 17 reaches the low, common end of it — an index

in range, a divisor non-zero — without a handwritten proof; the dependent types of Chapter 15 reach the high end at the cost of proving everything by hand.

There is a fourth claim the project backed into, and it deserves a row because it is the one Part III is really about: *the verifier itself is sound*. A tool that proves your program has no out-of-bounds access is only worth as much as the tool's own correctness — and that single claim splits, on inspection, into three mountains of very different height. Prove the *calculus* the checker implements is sound; prove the *decision procedure* it uses to settle each obligation is sound; prove the *code* that runs them both is a faithful implementation of either. Only the first is chapter-sized. Chapter 19 returns to this split and says how far up each mountain the project actually climbed.

14.3 What we have

Of those, Lark holds the first outright, and holds it in the strong form: for *every* program the type checker accepts, evaluation does not get stuck on a type error — not for the programs in the suite, but for all of them, the unwritten ones included. This is not a claim resting on tests. It is a theorem, stated in a proof checker and verified by it, and Chapters 15 and 16 are the machine that checks it and the proof it checks.

It is worth pausing on how much and how little that is. How much: a whole category of failure — the category that dominates crashes in untyped languages — is ruled out for the entire language at once, permanently, by an argument a machine has checked line by line. How little: it says nothing about whether any program does what its author wanted. A program that sorts backwards, divides by a zero it was handed, or loops forever is not stuck in this sense; it is behaving, in a way its author did not intend. Type soundness is a floor, not a ceiling. It guarantees the program keeps making well-typed moves; it is silent on whether those moves add up to the right thing. The rest of Part III is about buying pieces of that silence back.

14.4 What we do not

Draw the far boundary just as plainly. Lark has no verified compiler: the path from well-typed Lark to running machine code passes through a C compiler and a runtime this book trusts and does not prove, and a proof about the language does not reach down to certify the metal. That boundary was named when the fixpoint was closed and it has not moved.

What *has* moved is the boundary around specifications, and this section is the place to record the shift plainly. An earlier plan for this book would have said,

here, that Lark proves nothing about what individual programs compute — that specifications were out of reach. That is no longer true. The refinement checker of Chapters 17 through 19 exists, runs, and checks real specifications on a suite of programs with both its verdicts exercised — the ones it should accept and the ones it should reject. The reachable claim was reached.

But reaching it moved the boundary rather than dissolving it, and the new boundary is the true subject of the chapters ahead: the verifier is not itself verified. The tool that proves your program keeps its promises is trusted code, tested to the edge of what testing reaches — Chapter 18 is the account of how hard it was tortured — but not, as a whole, proved. Every guarantee in Part III is only as strong as that trusted tool, which is why the part does not end with a victory lap. It ends by walking up to the newest boundary, naming what sits on each side of it, and being clear about which side a given promise is standing on. A proof does not remove the last thing you must trust. It moves the trust somewhere smaller, and asks you to look hard at where it went.

Chapter 15

A Proof Checker in C

A proof, to be worth anything, must be checked by something small enough to trust. This chapter presents that something: about four thousand lines of C that implement Martin-Löf type theory — a system in which a program and a proof are the same object, and running the program and verifying the proof are the same act.

The claim of the last chapter was that a weak logic buys you automation: the refinement checker proves what it can and never asks you for a proof. This chapter is the other end of that trade. Here the logic is as strong as mathematics, the machine no longer finds the proofs, and a program arrives with its proof already attached. What follows is not a course in type theory — that would swallow the book. It is the smallest tour that lets Chapter 16 say what it needs to: what the identification between programs and proofs is, what makes a type *dependent*, how little code the trusted base actually is, and which of its machinery this book leaves unused. The checker itself lives in `prove/lcore/`, and the reader who wants all of it should read it there.

15.1 Propositions as types

Start with a coincidence that turns out not to be one. A function that takes an `A` and returns a `B` is, read one way, a program: feed it a value of type `A` and out comes a value of type `B`. Read another way, it is a procedure that turns any *proof* of `A` into a proof of `B` — which is what it means to say *A implies B*. The two readings are the same function. A type is a proposition; a value of that type is a proof of it; and the arrow that has meant "function from…to" since Chapter 5 is also the word *implies*.

Follow the coincidence outward and the vocabulary of logic reappears as ordinary data. A pair — a value of `A` *and* a value of `B` — is a proof of *A and B*; you

cannot build the pair without building both halves. A tagged union — a value that is *either* an `A` or a `B` — is a proof of *A or B*; to have one you must have supplied a side. The type with no values at all is *falsehood*: a proof of it cannot exist, and the function that would turn one into anything you like is the old logical move *from a contradiction, anything follows*. The checker spells each of these out under its own name — the pair type, the sum type, the empty type with its `abort` — so that a proof is never a separate artifact stapled to the code. The proof *is* the code, and to check the proof is to type-check the program. Nothing extra runs; nothing extra is trusted.

This is what dependent-type people mean by *propositions as types*, and it is the hinge the rest of the chapter turns on. A logic and a programming language, held far enough apart to look unrelated, turn out to be one thing described in two dialects. The checker is the place where the two dialects are made to agree, symbol for symbol.

15.2 Dependent types

The previous book stopped one step short of here, and the step is worth naming, because it is the difference that matters. In an ordinary type system a type is fixed before any value flows through it. `Int` is `Int`; the type of a function is settled without knowing which integer you will pass. The type cannot mention the value, because when the type is written the value is not yet there.

A dependent type removes that wall: a type is allowed to *mention a value*. The clearest case is a function whose result type depends on its argument. Write the identity function that works for every type at once,

```
id : Pi(A : Type). A -> A
```

and read the `Pi` as *for all*: for all types `A`, a function from `A` to `A`. The first argument is a *type*, `A`, and the type of everything after it — the `A -> A` — mentions the value that `A` just bound. The ordinary arrow `A -> B` is only the special case of `Pi` where the result forgot to look at the argument. Its dual, the dependent pair `Sigma`, is a value together with another value whose type depends on the first — *there exists an `A` such that...* — and between them `Pi` and `Sigma` carry *for all* and *there exists*, the two quantifiers a first logic course spends a term on, now just two type formers a program is built from.

There is a cost, and it is the reason the previous book stopped. Once a type may contain a value, deciding whether two types are equal may require *running* the values inside them — to see that `A -> A` at `A := Nat` is the same type as `Nat -> Nat`, you evaluate. Type-checking now has a program buried in it, and the

checker has to be able to compute. That is why the next section is half about an evaluator.

15.3 The checker, read

The trusted base is smaller than its power suggests, and its smallness is the point — a proof is only as trustworthy as the thing that checked it, so the checker is written to be *read*. Three functions carry the weight, and their signatures are the shape of it entire:

```
infer(ctx, term)         -> type, or failure
check(ctx, term, type)   -> yes / no
conv (u, v)              -> are these two the same type?
```

The checker is *bidirectional*, which is a plain idea. Some terms announce their own type and some need to be told it. A variable, a function applied to an argument, a term with an explicit annotation — these `infer`: you can read their type off directly. A bare function, with no annotation, cannot — there is nothing to say what its argument type should be — so it `check`s: handed an expected `Pi` type from above, it takes the domain from that type and walks into the body. Everything that cannot be checked directly falls through one door: infer a type for it, then ask `conv` whether the inferred type matches the expected one. Two functions and a notion of equality — that is the checker.

The notion of equality is where the evaluator promised earlier comes in. `conv` does not compare terms as text; `Nat -> Nat` and `(Pi A. A -> A) applied to Nat` are the same type though they are written nothing alike. It compares them by *normalising* both — evaluating each to a canonical value and seeing whether the values agree. The technique is *normalisation by evaluation*: a term is evaluated into a semantic value, and a value is *quoted* back into a canonical term. Syntax carries variables as de Bruijn *indices* (count binders inward from the use); the semantic domain carries them as *levels* (count outward from the root), and the two constructions in `term.h` that everything else stands on — `Term` and `Val` — are that split made concrete. When evaluation reaches a variable whose value is not yet known, it produces a *neutral* value: a stuck computation, a head with a queue of eliminators waiting for it, which is how the checker computes underneath a binder without ever guessing what the bound variable will be.

The typing rules themselves are one `switch` in `check.c`, one arm per term form, each arm a near-transcription of a rule you could write on a blackboard. The arm for application checks that the function has a `Pi` type and that the argument fits its domain, then substitutes the argument into the codomain. The arm for a pair checks each side against its half of a `Sigma`. The arm for the

identity eliminator is longer, and the arms for the inductive families longer still, but none of them is deep — they are careful, not clever. You can read the trusted kernel — terms, the evaluator, the type checker — in an afternoon, and the point of keeping it to a few thousand lines of plain C is that an afternoon is a price a doubter can actually pay. Around that kernel sits a parser, an elaborator that fills in the arguments a human would rather leave implicit, and a small read-eval-print loop; those make the checker usable but are not where the trust lives. The trust lives in the switch.

Two arms are worth flagging because they are deliberately *not* careful: the general fixpoint and context weakening are marked in the source as *trusted*. They are escape hatches — a fixpoint that the kernel accepts without checking that it terminates, a weakening step taken as given — added because the metatheory of Chapter 16 needs them and proving them from scratch would cost more than the book can spend. A checker you trust is a checker whose trusted parts are labelled, and these are labelled. Naming them here is the same discipline the refinement chapter used when it admitted the checker does not read a divisor's value: the worth of the tool is in the honesty of its boundary.

15.4 Why HoTT is here

The checker carries more than this book uses, and the surplus has a name: *homotopy type theory*. Alongside the machinery already met, the source implements identity types and their eliminator, the univalence axiom, function extensionality, propositional truncation, and even the circle — a type with a point and a non-trivial loop back to it, the smallest space that is not contractible. These are the tools of a research programme that treats types as spaces and proofs of equality as paths, and the checker is, among other things, a laboratory for teaching it.

This book uses a fraction of that. The soundness proof of Chapter 16 leans on identity types and their eliminator, on the recursors for numbers and booleans, and on inductive families with the universe polymorphism and implicit arguments that make them bearable to write — and it never once needs univalence, never needs the circle, never needs a higher inductive type. The heavy homotopy machinery sits in the checker unused by us, present because the checker was built to reach further than this book walks.

It would be easy to leave that unsaid and let the apparatus imply that the proof to come uses all of it. It does not, and saying so costs nothing worth keeping. A trusted base earns trust by being legible, and part of legibility is a candid inventory — here is everything the checker can do, here is the small corner of it this book actually stands on. The corner is enough. What the machine can promise about a Lark program, it promises through the handful of rules named

above; the rest of the checker is a door left open onto mathematics this book declines to walk through, and it is worth knowing the door is there and worth knowing we do not use it.

Chapter 16

Proving the Language

Here is the claim, and it is not a small one: *every* program Lark's type checker accepts will run without getting stuck. Not most. Not the ones we tested. Every one — including the ones nobody has written yet. This chapter is how such a thing is stated, how much of it a machine has actually sealed, and where the sealing met a wall worth being plain about — because the truthful version of the claim is more interesting than the clean one, and it is that version the machine checks.

The claim is not proved by running programs. It is proved once, about the language itself, by writing the language's rules down as objects the proof checker of Chapter 15 can read, and then handing it a proof it verifies the way it verifies any other term. Four files hold the development — `lark-typing`, `lark-subst`, `lark-step`, and `lark-preservation`, about a thousand lines together, in `prove/lark-formal/`. What follows is their shape, told for a reader who does not read inference rules for a living.

16.1 Progress and preservation

The classical way to prove a language never jams splits the promise in two. The first half is *progress*: a well-typed program is either already a finished value, or it can take another step of evaluation — it is never stuck partway, holding an expression that is neither an answer nor able to move. The second half is *preservation*: when a program does take a step, its type does not change underneath it. Chain the two and the guarantee is complete. A well-typed program can always step (progress), and what it steps to is well-typed again (preservation), so it can step once more, and once more, until it arrives at a value — and it never, at any point along the way, reaches a stuck state, because a stuck state is what progress rules out. That is type soundness, and it is the theorem this chapter is about.

Now the part that stops resembling a textbook. Preservation, here, is not proved at all — because it cannot be stated as something with content to prove. The expressions in the formalisation are encoded *well-typed by construction*: a term is not first written down and then checked against a type, the way the compiler does it; instead the only way to build the object at all is to build a typing derivation, so an object of type "expression of type `t`" *is* a proof that it has type `t`. The step relation is written to carry a type on both ends — it relates an expression of type `t` to another expression of type `t` — and so a step that changed a program's type is not a step the proof checker will accept you writing down. Preservation is not a lemma that was proved; it is a sentence that could not be made false. The type discipline of the checker in Chapter 15 enforces it before there is anything to argue.

Progress is where the wall is. The full statement — *every* well-typed closed term is either a value or can step — is the one part of the classical theorem this development does *not* mechanise in general, and the reason is specific enough to name. Proving it the textbook way means recursion on the shape of an arbitrary expression, and at several cases the proof needs to look at a term of some context and refine that context to the empty one — to use, mid proof, the fact that the term is closed. That refinement of a type index is an operation the proof checker cannot perform; it is a known edge of the tool, and a general progress proof runs straight into it. So the development does something narrower and truthful instead. It proves progress *constructively for the cases that matter*: it exhibits, as actual checked terms, a witness that each kind of value is already done, and a witness that each reduction rule — a function meeting its argument, an `if` meeting `true`, a `let` binding a finished value — does fire and produce a next state. And it proves the theorem that stands in for progress in a small language: that evaluation is *sound* — if a closed term evaluates to something, that something is a value, never a stuck expression wearing a value's clothes. That last one is a real proof, by recursion on the evaluation, and the checker signs it.

The gap between "proved for every term" and "proved for the values and every reduction rule, plus evaluation soundness" is not nothing, and this chapter is not going to pretend it away. It is the difference between a theorem and a very well-supported claim, and the difference is a limit of the proof checker, not of the language. Naming it is the price of the trust the rest of the chapter asks for.

16.2 Formalising Lark

To prove anything about Lark you must first *write Lark down* — turn the running language into a mathematical object the checker can read — and every choice made in that writing-down is a place where the object could drift from the language it claims to describe. So the drift has to be named.

The object is small on purpose. Types are `Int`, `Float`, `Bool`, `String`, unit, and the function arrow, and nothing else carries a type variable. That is not a simplification snuck past the reader; it is where the formalisation deliberately begins. The polymorphism and the unification of Chapter 5 run *before* this layer and are finished by the time it starts: the model takes as its input the program as it looks after inference has resolved every type variable to a ground type, so Algorithm W is not re-proved here — its output is the raw material. A context is a de Bruijn list, a variable a count of how many binders to step outward, which is the same bookkeeping the checker of Chapter 15 uses internally. And an expression is one of nine forms — the literals, a variable, a lambda, an application, a `let`, an `if` — each corresponding one-to-one with a case of the inference pass, a correspondence the source files spell out line by line so that a reader can audit the match by eye.

Three things the real language has are absent from the model, and each is a seam worth marking. There are no user-defined data types and no `match` — the fragment is the functional core, not all of the surface syntax. There is no affine tracking in these four files; the discipline that each value is used once, which Chapter 5 built into the type checker, lives in a separate proof layer of its own and is not folded into the soundness result here. And the evaluation relation is a hand-written small-step machine that mirrors the CEK evaluator of Chapter 6 rule by rule — call the function, then the argument, then substitute — but it is a re-description of that evaluator, not the evaluator itself. Nothing forces the two to agree except the care that wrote them to, and the correspondence tables that let a reader check the writing. The model is a faithful fragment. It is not the compiler, and the chapter's last section is about the distance between them.

16.3 The proof, in `lcore`

The development has a spine, and it is worth seeing the order, because each piece holds up the next. It begins with the least glamorous and most load-bearing lemma in all of type theory: *substitution*. When a function meets its argument, evaluation replaces the parameter, everywhere it occurs in the body, with the argument — and the lemma that has to hold is that doing so lands you at a well-typed term of the right type. In this encoding that means: given a body that is well-typed in a context extended by one binding, and a value to fill that binding, produce a well-typed term in the smaller context. Most cases are routine. The lambda case is not, and it was the hard knot of the development: substituting under a binder means lifting the surrounding values past a newly bound variable, and an early attempt stalled on that very shift. It was resolved by generalising the lemma to substitute into *any* output context rather than only the empty one, and by leaning on a single primitive, provided by the proof checker itself, that

shifts de Bruijn indices — one of the trusted operations Chapter 15 named. With substitution in hand the rest follows its shape.

On top of substitution sits the step relation — eight rules, and no more. Four of them do real work: a function applied to a value reduces by substitution; an `if` on `true` takes its first branch and on `false` its second; a `let` whose value is finished substitutes it into the body. The other four are bookkeeping — the congruence rules that say "evaluate here first": step the function before applying it, step the argument once the function is a value, step an `if`'s condition before choosing a branch, step a `let`'s value before binding it. Those four are what make the machine *call-by-value* and left-to-right, and they are the entire operational character of Lark, written as eight lines a person can hold in their head.

Above the single step comes its reflexive-transitive closure — reduce in zero or more steps — and above that a big-step evaluation relation built from it, and at the top the one theorem the development proves in full generality: evaluation soundness. Read as a term, its statement is *if e evaluates to v, then v is a value*, and its proof is a recursion over the evaluation with two cases — if the term was already a value, hand that fact straight back; if it took a step and then evaluated, pass along what the recursion already established. Two cases, a few lines, and the checker accepts it. That is the shape of it: a hard substitution lemma at the bottom, a small operational semantics in the middle, and a short clean theorem at the top that the middle exists to support.

16.4 What is still trusted

A proof buys you certainty about one thing by spending trust on others, and the worth of the proof is in knowing where the spending went. Draw the boundary of the trusted base plainly, from the outside in.

- **The model is a fragment.** What is proved is proved about the functional core of Lark with ground types — no `match`, no user data types, no affine layer. Extend the language and the proof does not automatically follow.
- **The model's fidelity is argued, not proved.** That the formal step relation is the real evaluator of Chapter 6, and the formal typing rules the real inference of Chapter 5, rests on the correspondence tables and on care — a reader can check the match, but no theorem enforces it.
- **The runtime is not verified.** The C the compiler emits, and the C compiler beneath it, are trusted just as Chapter 8 left them. The proof is about Lark's rules, not about the machine that ultimately runs them.
- **The proof checker is trusted.** Everything rests on the small kernel of Chapter 15 being correct — and on its labelled escape hatches, one of

which, the index-shifting primitive, this development uses in its substitution lemma. The trusted base is small and it is read, but it is trusted.

- **Universal progress is argued, not mechanised.** As the first section set out: the values and every reduction rule are certified, and evaluation soundness is proved, but the statement quantified over *every* well-typed term meets a limit of the checker and stops there.

That is a longer list of caveats than a triumphant chapter would print, and printing it is the point. A proof whose trusted base you cannot see is a proof you are taking on faith under another name.

One thing has changed since this proof was first filed, and it changes what the proof *is for*. It was written as a terminus — prove the core sound, file the certificate, move on. It has since become a foundation. The work of making the refinement checker's verdicts trustworthy — the horizon of Chapter 19 — is built as a logical relation laid *over* this chapter's step relation and its notion of a finished evaluation. The thousand lines proved here are not a trophy set beside the rest of the book; they are the ground the next proof stands on, and the multi-step reduction and the evaluation relation of the previous section reappear there as the load the newer argument is allowed to lean its weight against. That is the strongest thing that can be said of a proof: not that it closed a question, but that something heavier was later set on top of it, and it held.

Chapter 17

Proving Programs

Type safety is a promise about the language. It says nothing at all about whether your sort function sorts. To promise *that*, the type system has to be able to say it — and then to check it. This chapter is about the smallest extension to Lark that makes such promises possible: it was built, it runs, and the suite that exercises it stands at seventy-five programs. This chapter presents the extension itself — what a refinement type is, what it catches, and how it fits the language; Chapter 18 is about what it cost to make its verdicts mean something, and Chapter 19 follows it into trees.

17.1 A type that says more

In Chapter 5 a type was a kind of thing. `Int` was the whole numbers; `List(Int)` was the lists of them; a function type named what went in and what came out. A type answered one question about a value — *what shape is it* — and answered it for every value of that shape at once. `Int` says nothing to tell three apart from thirty from minus one. It was never meant to.

A refinement type answers a second question: *which ones.* Write

```
{ v : Int | v >= 0 }
```

and you have named the whole numbers that are not negative — the same `Int`, now carrying a condition. Read it left to right the way you would say it aloud. The `v` is a stand-in for the value being described. The bar is the word *where.* What follows the bar is the condition that value has to meet. *The integers `v` where `v` is at least zero.* There is no new arithmetic here — `v >= 0` is the comparison a child can read — and there is deliberately no logic notation, no quantifier, no symbol

you have not seen. The only new thing is that the type system now reads the condition and holds the value to it.

That turns out to be enough to say the things a plain type could not. A parameter can carry a demand:

```
fn safe_div(n: Int, d: {v: Int | v != 0}) : Int =
  n / d
```

The divisor is not merely an `Int`; it is an `Int` the caller has promised is not zero, and inside the body the division may be written without a second thought, because a caller who could not keep the promise was turned away before the program ran. A result can carry a guarantee just as easily:

```
fn length(xs: List(Int)) : {v: Int | v >= 0} =
  ...
```

and now whoever calls `length` may lean on the result being non-negative without looking inside, the way you lean on a library you have not read. The first is a precondition, the second a postcondition; the language needs no separate word for either, because both are the same idea — a value that arrives already knowing something about itself.

Two things about the condition are worth being plain about, because they are what make it more than a comment. The first is that it is not a comment. A note in the margin saying *d must not be zero* is checked by nobody; the refinement is checked by the compiler, at every call, before the program is allowed to exist. The second is that it is not a test. A test that divides and watches for a crash checks the one divisor it was handed on the one run you gave it. The refinement checks every divisor on every run, and it does so without running anything — the value never has to appear for its condition to be enforced.

And when the program does run, the condition is gone. A refinement erases: `{v: Int | v >= 0}` compiles to `Int` and nothing else, no hidden field, no guard slipped in at the entry to `safe_div`, no cost of any kind. The proof happened once, at compile time, and left no residue. The machine that runs the program runs plain integers, just as it did before this chapter — which is the appeal of it. An assertion buys its safety at run time, with a branch on every call and a failure that arrives in the field, on a user's machine, at the worst possible moment. A refinement buys the same safety at compile time, for no run-time cost, and collects on the failure at the keyboard, before anyone else has seen it.

17.2 Why this and not full dependent types

If a type can carry a condition, the obvious next thought is to let it carry any condition at all. Why stop at `v != 0`? Why not *a list that is sorted*, *a function that terminates*, *a proof of Fermat's last theorem*? A type system rich enough to state those is not a fantasy; it is *dependent types*, and Lark has one — the proof checker of Chapter 15, where a type may be an arbitrary proposition and a program is only well-typed once it carries a proof. The power is real. So the question this section owes an answer to is why the refinement checker does not simply reach for it.

The answer is a single word: *decidability*, and the trade it buys is a good one. A refinement's condition is confined to a small, well-behaved logic — linear arithmetic over the integers, together with equality between terms the checker does not look inside. That confinement is not a limitation the checker apologises for; it is the design. A logic that small is one a machine can decide *on its own*. The programmer writes the condition — `v >= 0`, `i < string_length(s)` — and the checker finds the proof, or finds that there is none, without help. Nobody writes a proof. Nobody even sees one.

Set that beside Chapter 15 and the difference is the point. There, the logic is as strong as mathematics itself, and the price of that strength is that the machine can no longer find the proofs — it can only check the ones you supply. Every guarantee is a proof written by hand, line by line, and a program is a program with its proofs stapled to it. Here, the logic is weak on purpose, and the reward for keeping it weak is that the proofs write themselves. One system asks the programmer to prove everything and promises to check it; the other asks the programmer to prove nothing and promises to prove what it can. They are two ends of the same idea, and the distance between them is measured in how much of the work the machine is willing to do.

The weakness is not free, and it is worth naming where it bites. The logic knows `v >= 0` and `2*v <= w`; it does not know that your sort function sorts, not until you hand it a *measure* to reason with, which is the subject of Chapter 19. Multiply two unknowns together and the arithmetic leaves the fragment the checker can decide, and rather than guess it declines — the failures of Chapter 18 are mostly the checker refusing to bluff. The border is drawn where automation ends. Everything the machine can settle by itself is inside; everything that would need a human proof is out, and out is where Chapter 15 begins.

Why make this trade first, before the stronger tool? Because most of the bugs a working programmer actually ships are not deep. They are an index one past the end, a divisor that can be zero, a value promised and then not delivered. Every one of those lives inside the decidable fragment, which means every one of them can be caught for the price of writing a condition and none of the price

of writing a proof. A tool that stops those the day you adopt it, and asks for no proofs in return, earns its keep before a tool that can stop anything at the cost of a proof on every line. The mountain in Chapter 15 is there for those who need to climb it. This is the path most programs should take first.

17.3 What it catches

A refinement travels with the value, and the checker's whole job is to notice when it has been contradicted. Every division raises one *obligation* — a fact the program owes before it may proceed, here that the divisor is non-zero; every array or string index raises another, that the index is in range. These are raised automatically; the programmer never writes one. When the checker cannot discharge an obligation it does not merely say so — it prints the *goal* it could not reach and the *from* it had to reach it with: the guards you wrote and the contracts in scope, set beside the fact they were not enough to establish. The verdicts that follow are verbatim output of the refinement checker, which lives in `prove/oracle/` and rides on the same compiler the earlier chapters built.

Each example is a matched pair — a program that is safe, and a mutant of it carrying one real bug. The seventy-five of them sit in `prove/fixtures/`, each safe program beside its mutant, and both halves earn their place for opposite reasons. A safe program that does *not* check would mean the checker is too weak to use; an unsafe one that *does* check would mean it is worse than useless.

01 DIVISION BY ZERO

The obligation nobody has to ask for

Every `/` owes a proof that its divisor is not zero, contract or no contract. A refinement on a parameter is how a function *states* the demand; the caller meets it with a literal, a guard, or a contract of its own.

```
fn safe_div(n: Int, d: {v: Int | v != 0}) : Int =
  n / d

fn halve(n: Int) : Int =
  safe_div(n, 2)

fn ratio(n: Int, d: Int) : Int =
  if d != 0 then safe_div(n, d) else 0

fn scale(n: Int, k: {v: Int | v > 0}) : Int =
  n / k
```

VERDICT

```
ok: 4 obligation(s) proved
```

NOW THE MUTANT

✗ **The lie.** Drop the guard from `ratio`; guard the wrong thing (`d >= 0` lets `0` through); divide by `k - 1` where the contract only promised `k > 0`.

```
cannot prove: division by zero: d must be non-zero
    goal: d /= 0
cannot prove: call to safe_div: argument 'd'
    goal: $v /= 0
    from: (d >= 0 and $v == d)
cannot prove: division by zero: (k - 1) must be non-zero
    goal: (k - 1) /= 0
    from: k > 0
3 of 4 obligation(s) unproved
```

> Read the `from` lines. `d >= 0` really does not rule out zero, and `k > 0` really does not make `k - 1` non-zero. The checker is not guessing — it is showing the gap.

02 ARRAY AND STRING BOUNDS

An index in range, without computing the length

The contract on `char_at` puts the index in $[0,$ `string_length(s)`$)$. The checker cannot evaluate a string's length — and does not need to. It carries the *same* opaque term `string_length(s)` out of the guard and into the call, and sees that the two agree.

```
fn char_at(s: String, i: {v: Int | v >= 0 and v <
  ↪ string_length(s)}) : Int =
  string_index(s, i)

fn first_char(s: String) : Int =
  if string_length(s) > 0 then char_at(s, 0) else 0

fn last_char(s: String) : Int =
  let n = string_length(s) in
  if n > 0 then char_at(s, n - 1) else 0
```

VERDICT

```
ok: 4 obligation(s) proved
```

NOW THE MUTANT

✗ **The lie.** Call `char_at(s, 0)` unguarded; guard with a tautology (`string_length(s) >= 0`); index at `n` instead of `n - 1`.

```
cannot prove: call to char_at: argument 'i'
    goal: ($v >= 0 and $v < string_length(s))
    from: $v == 0
cannot prove: call to char_at: argument 'i'
    goal: ($v >= 0 and $v < string_length(s))
    from: ((n == string_length(s) and n > 0) and $v == n)
3 of 4 obligation(s) unproved
```

> The second `from` is the off-by-one in full: the checker knows `n > 0` and `$v == n`, and that is not enough for `$v < string_length(s)`. One past the end.

Those two are toys, chosen small enough to read at a glance. Here is one nobody chose. Sweeping the checker across the sample programs in `self/samples/` turned up a *true positive in Lark's own source*:

```
| Div(a, b) => eval(a) / eval(b)
```

in `self/samples/05_expr.lark`, and again in `self/samples/09_parser.lark` — the parser we compiled, optimized, self-hosted, and in Chapter 12 *ran on a microcontroller*. Nothing divides by zero in the sample inputs, so it has never once failed. It had been sitting there, well-typed, for the length of this book, and the refinement checker is the first thing in the entire stack to notice — on the first run, without being asked.

Well-typed does not mean correct, and the gap is not hypothetical: it is in our own code, in a file the reader has already met four times. Note the exact shape of the verdict. It is not "your program is wrong" but "I cannot prove this cannot happen" — the more careful claim, and the only one the machine is entitled to make. Chapter 14's distinction between the two is the setup; this is the payoff.

The bug stays. Open `self/samples/05_expr.lark` and it is still there, on line 28 — and a latent fault that the last chapter's tool discovers in the first chapter's code is worth more as a finding than as a fix.

17.4 How it fits

The extension is smaller than the ideas in it make it sound, and it is small for one reason: it does not touch the type system it sits on. The inference of Chapter 5 already walks the program once, collecting constraints about what type each term must have and solving them by unification. The refinement checker adds a second walk of the same shape. Wherever a value with a condition flows into a place that demands one — an argument reaching a refined parameter, a body reaching a refined return type, an index reaching an array — the walk emits an *obligation*: a small claim that, given everything known at this point in the program, the demanded condition holds. The obligations are collected and handed one at a time to the solver. Unification finds substitutions; this second pass finds proofs. Same skeleton, a different engine at the leaves.

What does not break is the type structure, and this is what keeps the extension conservative. Because refinements erase, the inference of Chapter 5 runs first and runs unchanged — it never sees a condition, only the plain type underneath — and the refinement pass reads its results afterward. A refinement never alters what type a term has; it only decides whether that term is allowed to stand where it stands. The checker is a layer on top of the type system, not a new type system, and a program that fails refinement checking is a well-typed program that made a promise it could not keep. That is why the apparatus of this chapter could be bolted onto a language that already existed without disturbing a line of what came before.

There is one more thing this chapter is the place to record, and it is not a refinement bug at all. Stress-testing the checker meant feeding it programs no human would write — among them a single expression three thousand terms deep — and watching where the time went. It went somewhere unexpected. The refinement pass held up; the slowdown was in the inference *underneath* it, and it had been there since Chapter 5. Algorithm W, at every node it visits, applies its accumulated substitution to the entire type environment, and on a deep enough term that turns a linear walk into a quadratic one. It has nothing to do with refinements. The refinement checker did not cause it; it revealed it, because the torture test built to strain the new solver ran the old inference too, harder than any real program ever had.

We did not fix it. The compiler these measurements describe is the one the earlier chapters built and then stopped touching, and it is because it stopped changing that the numbers in this chapter are evidence rather than preference — rewrite the thing you are measuring and you have measured nothing. So the quadratic stays, documented and unrepaired, a cost real programs never reach and this book will not disturb its baseline to remove. It is the clearest case in

these pages of a measurement that changes what you *know* without changing what you *do* — which is, often enough, just what a measurement is for.

Chapter 18

What a Verifier May Not Do

The previous chapter ended with a tool that says `ok: N obligation(s) proved` and means it. This chapter is about the road to meaning it — which ran through nine occasions on which the tool said it and was wrong. A false proof is the worst object in this book: a program stamped safe that then divides by zero. Nearly every bug in two parts of self-hosting and optimization was caught by disagreement between two implementations; a false proof is a statement about the *world*, and no second implementation is entitled to certify it. The methods of this chapter — an adversary that runs what the checker proves, a torture suite that measures what it costs, and invariants that turn every shipped bug into a standing check — exist because the differential method hit its ceiling here.

Nine times over the life of this checker the tool printed `ok` and was wrong. Four were caught by hand, three by the first adversary, the eighth by an invariant suite built to stop trusting luck, and the ninth by that same adversary once it had learned to speak refined lambdas. Each is told below in its place. Two things about the nine are worth putting up front, because they shape everything that follows.

The first is where they lived. Every one of the nine was in the checker's translation layer — the pass that reads a program and emits the obligations — and not one was in the solver that decides them. The decision procedure of the last chapter, the congruence closure and the Omega test, never once proved a false thing put to it correctly. What failed, nine times, was the wiring that decided *what* to put to it: the code that chooses which facts count as known and which claims must be earned. A sound engine fed a crooked question answers the crooked question soundly, and reports success.

The second is that they fall into three shapes, and the shapes are the lesson.

- **Believing what was not established.** A `Float` handed to a logic in which `x == x` is always true — and false at run time, for the NaN that

no float is equal to, itself included. An `if` condition the checker could not translate, quietly read as `true`, so that the else branch inherited *false* as a premise and proved anything at all from it.

- **Speaking about a name that had changed hands.** A local `size` shadowing the global `size` a contract was written against — one symbol in the logic, two different functions in the program, and a congruence closure that cannot tell them apart because nothing told it they were two.
- **Never asking.** Several sites *translated* a sub-expression where they should have *walked* it, so that a division buried in an `if` condition, a comparison, a `not`, or a unary minus raised no obligation at all. The output read `ok: 0 obligation(s) proved`, exit 0 — and it had been reading that, over the sieve of Chapter 2, for the entire life of the checker.

The rules that came out of the nine arrived in the order the bugs did. *A budget must give up in the sound direction*, so that running out of room is never mistaken for a proof. *A hypothesis the checker cannot read must be dropped; a goal it cannot read must be asked anyway* — silence about what you may assume is modesty, silence about what you must prove is a lie. And the one this chapter ends on, because it is the shape of all three above it: a checker's silence is a verdict, and a report must never let silence read as approval.

18.1 Run what you proved

The first four false proofs were caught by reading. Someone wrote a program that should have been rejected, watched the checker accept it, and went looking for why. Reading works, and it found real bugs, but it does not scale and it does not sleep, and a person who has just fixed a bug is the worst-placed reader of the code around it. So the fifth, sixth, and seventh were caught by a machine.

The machine is `prove/harness/adversary.py`, and its idea is small enough to state in a sentence: generate a random program, check it, and if the checker says `ok`, *run* it. A refinement is a claim about the world — this divisor is never zero, this index is always in range — and the world is the one court entitled to overrule it. Type safety could be trusted to a second implementation, because two compilers that disagree have found a bug between them without either being right; but no second implementation is entitled to certify that a division will not fault, because that is not a fact about the code, it is a fact about every value the code will ever see. Only running it settles that, and even then only for the values that turned up. So the adversary runs. If the checker proved a program safe and the program then divides by zero, the disagreement is not between two opinions — it is between an opinion and an event, and the opinion loses.

That three of the first seven came back from the adversary, against four found by hand, is the argument for building it early. It is a tireless reader that does not know what it is supposed to think, which is the one advantage no author of the checker has.

It has a blind spot, and the blind spot names a discipline. A generator only produces the programs its grammar can describe, and for three revisions this one's grammar could not describe a lambda, though its own documentation claimed it could. A fuzzer that cannot say a construction cannot test it, and will report green over a hole it simply never approached. The only defence is to *watch it fail*: after any fix, and after any new check, put the bug back — the real one, or a planted stand-in — and confirm the net closes on it before restoring the code byte for byte. A test that has never once failed is the uncalibrated instrument of Chapter 12, brought indoors: it reads zero whether the quantity is zero or the needle is stuck.

18.2 No answer at all

The adversary asks whether a proof is true. There is a second question it cannot ask, because it waits for an answer that may never come: not *is the verdict right* but *does a verdict arrive at all*. A checker that loops forever on a legal program has failed as surely as one that lies, and no assertion about its output will catch that, because there is no output to assert about. The tool for this failure is `torture.py`: programs that are hostile but legal — deeply nested, absurdly large, built to strain the machine rather than fool it — run against a wall clock, where the grade is not the verdict but whether the clock ran out.

It found what a wall clock is for. Every fence inside the solver was local — a depth limit here, a term-count cap there — while the caller that drove the solver looped, calling it again on the next obligation with the counters reset. A budget that resets is not a budget; it is a speed bump, and a program with enough obligations sails over an unbounded number of them. The fix was to make the budget belong to the entire run, not to each call.

The best of the findings was a crash that was concealing a cost. A certain input overflowed the stack, and the obvious repair — raise the stack limit — turned the crash into a hang. The overflow had not been the bug; it had been the mercy, killing a cubic algorithm early enough that nobody noticed the algorithm was cubic. Repairing the crash without measuring the cost underneath it would have moved the failure from a place the tests could see — a traceback — to a place they cannot: a program that simply takes too long, on inputs slightly larger than any the suite happened to try. A crash is a loud failure, and a loud failure is a gift; the temptation is to silence it rather than read it.

18.3 The walk nobody writes

Measuring where the time went turned up two redundant traversals, and they were fixed, and then came the question that separates a profile from a fix: was that all of them? It was not, and the profiler had to say so, because the third traversal appears nowhere in the source. It is `hash`.

A term is a frozen record, so Python hashes it by hashing the tuple of its fields — and a field of a term is, more often than not, another term. A term is a tree, and its hash is a walk of that tree, computed fresh on demand. Walked once, that is free and nobody would notice. But a congruence closure does nothing all day except look terms up in dictionaries: every merge, every `find`, every subterm it registers is a hash. An $O(n)$ hash inside an $O(n)$ loop is a quadratic, and it is the worst kind of quadratic — the kind nobody wrote down, because nobody wrote the loop. It is not a line of code. It is the absence of a line of code, the caching that was never added, made visible only when the profiler reports four and a third million calls to `hash` on a single six-hundred-term sum, forty percent of the run spent re-walking trees that had not changed.

Because terms are immutable and heavily shared, the fix is one cached integer per node: hash once, remember it, hand back the memory ever after. The rule generalises past this program. An immutable structure may be walked once; every walk after that is a bug you have not noticed yet. And it is the same rule as the cost buried in Chapter 5 — Algorithm W re-applying its substitution to the entire environment at every node — which is worth naming as a pattern, because both cases teach it. An asymptotic problem is rarely an algorithm somebody chose badly. It is usually an innocent line doing something reasonable in a place it is reached far more often than the person who wrote it ever counted.

18.4 Findings, and their ceiling

Here is the criticism that reframes everything above it. Every one of those seven was a *finding*. A hostile program got lucky, or a careful reader got lucky, or a profiler got lucky. Seven times the luck held, and its holding is just what hides the trouble with it: a method that finds bugs by being lucky cannot tell "there are none left" from "I stopped looking." That is the very distinction this chapter has spent its whole length forcing the *checker* to make about its budgets — give up in the sound direction, never let running out of room read as a proof. To demand that of the tool and not of the people building it is a poor joke.

So the rules stop being prose and become a program. `invariants.py` runs over the entire corpus on every build, and each check it makes is a bug we shipped, generalised to its shape. *Coverage*: every expression node in a checked body is visited once and once only — *missed* is the shape of the never-asking

bug, *revisited* is the shape of the quadratic, and one question catches both. *Asking*: every division that is walked raises one obligation, which coverage alone cannot see, because there the node *was* visited and the checker chose not to ask. *Work*: the number of nodes visited is linear in program size, measured by counting rather than timing, so that it is a regression test and not a wall-clock coin flip.

It found the eighth false proof within minutes, and it was the first one found on purpose. Fifty-eight nodes, across seven files, came back never walked, and every one of them was the same thing: the body of a lambda. The checker has two ways into an expression — `check`, which is handed an expected type, and `synth`, which must work the type out for itself. `check` walks a lambda's body, because it has a function type to push into it and match against. `synth` did not; it returned an opaque type without ever looking inside, and an unannotated `let h = fn (a) => …` goes through `synth`. So the body went unread, and a division inside it raised nothing:

```
let h = fn (a : Int) => 100 / a in h(0)
```

reported `ok: 0 obligation(s) proved`, exit 0, and then raised a division by zero at run time. Zero obligations is not "there was nothing to prove." It is "I did not look," printed in the words of "there is nothing wrong." One of the fifty-eight unwalked nodes was a real division, in the file the suite calls *torture*, unexamined for the fork's entire life. With the lambda case taught to walk its body, the same mutant now answers straight:

```
cannot prove: division by zero: a must be non-zero
    goal: a /= 0
1 of 1 obligation(s) unproved
```

> "Every sub-expression is walked once and once only" is not a fact about expressions. It is a fact about every place in a program where code can hide — and the body of a lambda is one of those places.

The generator had the identical hole, in the identical shape: its own documentation had claimed lambdas for three revisions, and it could not emit one. A generator only ever finds bugs in the language it can speak; its grammar is a claim about which programs exist, and an unexamined claim of that kind is what this book has spent its length refusing to make.

18.5 The language it can speak

The aphorism paid the day it was written. Teaching the generator to speak the words it had been claiming — `min` and `max` in contract position, and lambdas carrying real refinements — closed the documented gap, and the closing caught the ninth false proof on arrival. It is the second half of the lambda story, one step along from the eighth. The eighth was: a body is code, so `synth` must walk it. The ninth is: a lambda's parameter is a contract, and a contract is discharged at the call.

Walking the body fixed half of it. Inside `fn (k : {v | v != 0}) => 100 / k`, the parameter `k` is bound with its refinement, so the body may assume `k != 0` and the division proves. That is correct — but it is only half a promise. The other half is that whoever calls this function must *deliver* a non-zero `k`, and delivery happens at the call. `synth` was returning an opaque type for the lambda, one that carried no parameter contract at all, so the application had nothing to discharge:

```
let g = fn (k : {v : Int | v != 0}) => 100 / k in
g(x - x)        -- g(0): the divisor is zero, and always was
```

proved the body's obligation and asked nothing of the call, reported `ok: 1 obligation(s) proved`, exit 0, and divided by zero. One obligation proved was true and beside the point: it said the body is safe *if* the parameter is non-zero, a promise the caller was never made to keep. The fix is that the lambda case in `synth` now returns a function type carrying its parameters' refinements, the way a top-level `fn` always had, so that the existing rule for applications raises the precondition where it belongs. Two hand-audits had walked past this; the generator found it the day it could say it:

```
cannot prove: call to g: argument 'k'
    goal: $v /= 0
    from: $v == (x - x)
1 of 2 obligation(s) unproved
```

The generator kept going to school. Taught to speak *nested measures* — a tree whose depth is consumed as a divisor, a search tree with its ordering stated as a contract — it began generating programs whose safety turns on facts no arithmetic fragment can settle by itself, which is the country Chapter 19 opens. It was given a third judge, too: an independent transcription of each measure into plain Python, so that a proof the transcription calls false is recorded as a false proof even when the generated program happens not to crash — the world's verdict, made to speak up on the safe runs as well as the fatal ones. The limits

here are named and kept: the nested traps bite reliably only at high volume, and the search-tree traps are checked against a hand-written witness rather than fuzzed under a planted regression. Neither is a hole in the checker; both are places the adversary is weaker than the thing it guards, and saying so is the discipline.

The last false proof of the chapter is the one that never shipped, and it is here because a verification method earns its keep by what it is allowed to forbid. When the equality seam — the gap discussed in this part's interlude — was measured, the obvious repair was to route the runtime == through the machine's existing value-equality helper. The helper handles the five scalar kinds and quietly returns `false` for everything else, functions included. So a branch the checker had proved dead, sitting behind a test of one function against itself, would have gone live under the fix: `g == g` reads `false` for a function `g`, the else branch runs, and it divides by zero — on the bench, produced by the repair, before anyone shipped it. The fix was declined and the seam kept as a documented boundary. A verification method is working when it is permitted to veto the fix to its own findings; the nets that guard the checker had grown teeth enough to bite the hand that was trying to help.

Chapter 19

Measures and Trees

A refinement on an integer can say `v != 0`, and Chapter 17 showed what that buys. But most of what a program promises is about *data*: this list is non-empty, this tree is ordered, this index is inside this buffer. For the logic to hold a program to such a promise it must be able to speak about structure — and the vocabulary it gets is the *measure*: a small, structurally recursive function that exists for the logic's benefit and is erased before anything runs. This chapter is the story of teaching the checker to follow a measure into a tree — first a measure with a `match` inside, then a search tree the program builds, and finally the hard direction, a search tree the program takes *apart* — and of what each step cost.

19.1 Ghost functions

A refinement's condition is written in a small logic, and until this chapter that logic knew only arithmetic and equality. It could compare two integers, and it could carry an opaque term like `string_length(s)` from a guard into a call without ever evaluating it, as Chapter 17 showed. What it could not do was look inside a list or a tree. A *measure* extends its vocabulary, one word at a time, without extending the logic itself.

A measure is a function, written beside the program, structurally recursive over a single argument of an algebraic type:

```
measure len(xs : List(Int)) : Int =
  match xs with
  | Nil           => 0
  | Cons(_, rest) => 1 + len(rest)
  end
```

It looks like ordinary code, and it is not. A measure lives in the program's measure table, never among its declarations; the inference of Chapter 5 never sees it, and neither does the machine that runs the program. It is *ghost* — erased along with the refinements, so that naming one in a contract costs nothing when the program runs. The corollary is free: because a measure is not part of the code, calling one from real code is an unbound-name error, caught by the type checker, and no rule had to be written to forbid it. A mention is not a use — the same distinction the affine work turned on, arriving here as a fact about where the function is allowed to live.

What the checker does with a measure is turn each of its equations into an axiom the solver may use, but only where it can see which constructor the argument was built with. From `len` it learns two: `len(Nil) == 0`, and `len(Cons(_, rest)) == 1 + len(rest)`. Given a term whose head constructor is known, the matching equation fires and rewrites it; given an opaque term, nothing fires, and the checker stays silent rather than guess. So a contract can demand a nonempty list, and the demand is discharged without anyone computing a length:

```
fn head(xs : {v : List(Int) | len(v) > 0}) : Int = ...

fn main(io : IO) : IO =
  print(io, show(head(Cons(3, Cons(4, Nil)))))
```

VERDICT

```
ok: 1 obligation(s) proved
```

The one obligation is `len(Cons(3, Cons(4, Nil))) > 0`, and it is discharged because the two equations unfold the count to 2 and `2 > 0` is arithmetic the solver already owns. `len`, a running `sum`, a tree's `depth`: each is one word added to what a promise may say, and each disappears before the program runs.

19.2 A match inside a measure

The measures so far take their argument apart once. A search tree needs more. To speak of a tree's rightmost value — its largest, when the tree is ordered — you have to look at the shape of a *child*: the maximum of a node is its own value when the right child is empty, and the right subtree's maximum otherwise. That is a `match` on a field of the thing you just matched, a match two deep. The fixtures use the length of a tree's right spine the same way, as a divisor:

```
measure rdepth(t : Tree) : {v : Int | v >= 1} =
```

```
match t with
| Leaf => 1
| Node(l, v, r) =>
    match r with
    | Leaf              => 2
    | Node(rl, rv, rr) => 1 + rdepth(r)
    end
end
```

The inner match asks whether the right child is a `Leaf` — the spine ends — or a `Node` to descend into. The checker flattens the nesting into one equation per path through the constructors, and those equations fire at *concrete subfields*: the depth-two arm only where the right child is literally a leaf, the recursive arm only where it is literally a node. Given `rdepth(t)` for an opaque `t`, nothing fires — the same sound silence a flat measure keeps at an opaque argument. A standing invariant pins the flattening, so the arms the checker fires can never drift from the definition it read.

Two things earn their keep, one per direction. A measure may carry a declared result, the way a function carries a contract: `rdepth` promises `{v | v >= 1}`, and the checker proves it one arm at a time, the nested arm by an induction that has to reach a concrete-subfield target. That promise is what lets a consumer divide by a depth it cannot see — a function taking an opaque tree and a `d` tied to that tree's depth may write `100 / d`, because a right-spine depth is at least one. Both directions hold at once:

VERDICT

```
ok: 6 obligation(s) proved
```

Mutate the claim — have a call insist that a one-node tree has depth zero — and the equations, firing at the built shape, refuse it:

```
cannot prove: call to use: argument 'd'
    goal: $v == rdepth(Node(Leaf(), 5, Leaf()))
    from: $v == 0
1 of 6 obligation(s) unproved
```

> The nested equation fires at the literal `Node(Leaf, 5, Leaf)` and computes its depth as 2; the claim was `0`. A measure that fires at concrete structure cannot be talked into a false value for a tree it can see.

19.3 Building a search tree

With a two-deep match in hand, the search-tree invariant itself is not a new mechanism — it is assembly. `maxv` and `minv` are a tree's rightmost and leftmost values; `bst` matches both children and, at each combination, states the order — the root above everything in its left subtree, below everything in its right, and both subtrees search trees in their own right:

```
measure bst(t : Tree) : Bool =
  match t with
  | Leaf => true
  | Node(l, v, r) =>
      match l with
      | Leaf =>
          match r with
          | Leaf        => true
          | Node(...)   => v < minv(r) and bst(r)
          end
      | Node(...) =>
          match r with
          | Leaf        => maxv(l) < v and bst(l)
          | Node(...)   => maxv(l) < v and v < minv(r)
                and bst(l) and bst(r)
          end
      end
  end
```

Nothing here is a change to the checker. It is the nested match of the last section, now on both children, plus one thing that mechanism quietly already allowed — a measure that mentions another measure, `bst` calling `maxv` and `minv` on a child. A program that builds an ordered tree and claims it is a `bst` is proved:

VERDICT

```
ok: 1 obligation(s) proved
```

because the nested equations fire at the concrete subfields of the tree the program holds in its hands, and the cross-measure calls reach the physical subterms for free. Disorder the tree — leave the constructors in place but put the values out of order — and the claim is refused, one obligation of one unproved: the checker walks the built shape, computes the actual order, and finds it is not a search tree.

And here is the limit, stated when the rung was climbed. This proves the *building* direction and only that. Given an opaque `bst(t)`, the guarded equations

decline — there is no concrete structure for them to fire at — so a consumer cannot read the order back out of an abstract tree. The obvious next fixture, descending into a tree already proved ordered, could not even be attempted. That gap is the next section.

19.4 Taking the tree apart

Nobody builds a search tree to admire it. The point of the order is to descend by it: look at the root, compare, throw half the tree away, and know that the half you kept holds everything you could want. That knowing is a fact about an *opaque* tree. The consumer holds a binder `t`, a promise `bst(t)`, and a `match` that opens one node; everything below the node's binders is invisible. The building direction of the last section never had this problem, because a builder holds the concrete tree in its hands. A consumer holds nothing but the promise, and the question of this section is how the checker lets it cash one.

The route as planned did not survive contact with the problem, and the way it failed is the most instructive thing in the chapter. The plan said: give `minv` and `maxv` *declared results* — let each measure carry an invariant, the way a function carries a contract, so that an opaque `bst(t)` would arrive with its bounds attached. But a declared result on a measure must hold of *every* tree the measure applies to, and a measure is a total function: `minv` is defined on the disordered tree too, and of a disordered tree `minv(t) <= maxv(t)` is simply false. There is no universal bound invariant to declare — not because the checker was too weak to admit one, but because the statement asked for is not true. The framing did not fail at a technical hurdle; it asked the logic to certify a falsehood, and the logic's refusal was the finding. When a specification resists being written down, the resistance is information.

What *is* true of every tree, ordered or not, is an equation:

```
bst(Node(l,x,r)) <=> lt(l,x) and gt(r,x) and bst(l) and
bst(r)
```

where `lt(t,b)` says every value in `t` lies below `b`, and `gt(t,b)` the mirror. The reformulation moves the order out of a side-promise about bounds and into the meaning of `bst` itself — and it pays twice. First, `lt` and `gt` are measures with an *extra parameter*, which is the vocabulary the bound needed all along: "below the root" is a relation between a tree and a value, and no unary measure can say it. Second, `bst` becomes a *single* match on the root, and a single-match equation is unguarded — it fires wherever a `Node` appears, even when `l`, `x` and `r` are opaque binders. Fired at the consumer's one opened node, it hands out `lt(l,x)`, `gt(r,x)`, `bst(l)` and `bst(r)`: the order facts a descent runs on, read back out of a tree the

consumer cannot see. (The spike that found this also found that single-match measures could destruct opaque trees *already* — the blocker in the last section was the two-deep nesting specifically, not destructing in general. It also spent its best hours on a red herring: a binder named the same as the checker's internal value binder, so the fix looked wrong until the binders were renamed. Some debugging is philosophy; some is hygiene.)

One new capability was needed, and only one. A unary measure's equation fires when a constructor appears, but an extra-parameter equation is about an *application* — `b` lives only there — so there is no bare constructor to fire on. Instead the checker discharges *demands*. A demand `lt(c,b)` is answered by the shape of its first argument: constructor-headed, the arm fires and its own recursive demands are followed down the structure with `b` held fixed; opaque-headed, the demand stays an atom — sound silence, the same modesty as everywhere else in the checker; variable-headed, the demand bridges to the constructor terms the variable equals, which is the step congruence closure gives a unary measure for free. A worklist runs the demands to a fixpoint. The solver was not touched — the entire capability lives in which true equations the checker chooses to put in front of it, and holding `b` fixed while following structure is what keeps the choice from crossing one node's bound with another's. The section on cost, next, is about what happens if you do not.

The fixtures hold both directions at once. The safe one (`36_bstdestruct_safe` in the suite) builds a tree and proves it a `bst`, as the last section could; then, from an opaque `bst(t)`, it proves the left child is a `bst` and descends on a divisor that only the handed-out order facts discharge — six obligations, all proved, and the program runs. The unsafe twin lies in the direction the last section could not even attempt: it claims a `bst` for `Node(r,x,l)` — the swap — and the obligation this raises needs `lt(r,x)`, the negation of the `gt(r,x)` the equation hands out. One obligation of four refused, which is the right count: the swap is one lie.

And because the new machinery emits only true equations, its failure mode is not unsoundness but *vacuity* — a capability that fires nothing is silent, and silence looks safe. So the watch this time is for load-bearing rather than for lying: disable the firing, and the two destructing obligations in the safe fixture fail while the building fixture of the last section does not notice — the capability carries the new direction and nothing else leans on it. Restored byte for byte, the suite closes green. The adversary got the same teeth (a generated consumer that reads `gt(r,x)` out of an opaque tree and divides by it, with an independent transcription of the measures as its oracle), and genuine destructing programs now prove at the same rate as built ones.

That is the boundary crossed. A promise about a structure was made where the structure was concrete, carried through a function type as one opaque word, and cashed on the other side — order out of a tree no one can see into. It is

worth saying what crossed it: not a stronger solver, not a cleverer invariant, but a reformulation that stopped asking the measure to say something false. The checker's part was small and plain — fire the equations the definition already licenses, at the places the consumer already has — and that is the pattern this book keeps finding at its best moments: the capability was in the definitions all along, and the work was learning what not to ask them for.

19.5 The cost of firing

The first version of extra-parameter firing was blunt: instantiate every pair, each node's bound crossed with every other node's. Profiling confirmed the shape a blunt version has to have — 29, 121, 497, then 2017 axioms emitted for trees of 3, 7, 15, and 31 nodes, a clean quadratic, doubling the tree roughly quadrupling the work. The demand-driven version of the last section follows structure with the bound held fixed, and comes out $O(n \log n)$ on a balanced tree: 13, 33, 81, 193 over the same sizes. A degenerate right-spine tree is quadratic even so — but intrinsically, the true order facts being themselves quadratic in number — and fast in absolute terms regardless; a sixty-three-node spine settles in about a fifth of a second.

This belongs beside two costs the book has already met: Algorithm W re-applying its substitution at every node in Chapter 5, and the uncounted `hash` inside congruence closure in Chapter 18. It is the same lesson a third time, with one difference that is the section's point. This quadratic was profiled *before* the capability was accepted, not after a torture test tripped over it in the field. The measurement was a precondition, not a postmortem. Profile the blunt version first, and let the number drive the design.

19.6 What stays outside

Every capability in these chapters was drawn with a boundary, and honesty about a tool is mostly honesty about its boundaries. Four are worth collecting here, each named when it was drawn and none dissolved by anything in this chapter.

Integers are unbounded in the logic and thirty-two bits on the metal. The solver reasons over the mathematical integers; the machines of Chapter 12 compute in a fixed width. A proof that a sum stays positive is a proof about numbers that never overflow — true of the logic, and one wrap-around away from false on the hardware.

Partial correctness. A proved program that returns an answer returns a right one. Nothing here promises it returns one at all; termination is a separate obligation, and the checker of these chapters waves past it.

The equality seam. One operator is granted by the type system, believed by the logic, and only partly implemented by the machine — the gap this part's interlude takes up, and the source of the tenth false proof, the one that was caught before it shipped.

Concrete-subfields one-sidedness. Guarded measure arms fire only where structure is visible. The destructing rung crossed that boundary for single-match extra-parameter measures; it did not cross it for every nested shape, and the arms that need concrete subfields still keep their sound silence over an opaque binder.

A boundary written down is a promise about where the promises stop.

19.7 The horizon

Type safety in this stack is not asserted; it is a theorem. The metatheory in `prove/lark-formal/` carries the typing judgment, the small-step semantics, substitution with its lemmas, and preservation — about a thousand lines that close in `lcore` at zero errors. The refinement checker these last three chapters have been reporting on enjoys no such thing. It is tested to the edge of what testing can reach, guarded by invariants that turn each shipped bug into a standing check, and it is still, in the end, *trusted*. The language was proved; the tool that proves things about programs written in it was not. That asymmetry is the horizon, and the plan gives it a name: V3.

The slogan is *prove the prover*. The trouble with the slogan is that it names three different mountains, and only one of them is the size of a chapter.

- **Prove the calculus.** Model refinement types — base types, predicates, subtyping — inside `lcore`, and prove that subtyping implies semantic implication: a value the rules admit at `{v:b|p}` really satisfies `p`. This proves the *rules* sound. It never mentions `refine.py`.
- **Prove the solver.** Show that the from-scratch decision procedure — congruence closure, the Omega test, DPLL(T) — is sound and complete for the fragment it claims. This is the metatheory of an SMT core, and it is not a chapter; it is a book other people have been writing for thirty years.
- **Prove the code.** Show that `refine.py` and `solver.py`, as running algorithms, decide the relation the calculus defines — which first requires porting them to Lark, then relating the port to the model.

The discipline Chapter 18 demanded of the checker — that it never let silence read as approval — is the discipline to apply here: say which mountain you mean. We

mean the first. Naming the other two and setting them aside is not evasion; it is the refusal to let *prove the prover* read as more than it is, which is the same care that made "I cannot prove this cannot happen" the right verdict rather than "your program is wrong".

Choosing the calculus leaves one decision that shapes everything after it, and it is worth seeing rather than swallowing. To prove the rules sound you need a meaning for the types to be sound *against*, and there are two.

Give the types a denotational meaning — `{v:b|p}` is the set of values of type `b` for which `p` holds, and subtyping soundness is set inclusion. Clean, and wrong for this language, because Lark has ordinary recursion: `descend` calls `descend`, and a function that may not terminate has no value to denote without dragging in domains and the apparatus of a semantics that appears nowhere else in the book. It would be a second, parallel meaning of a Lark program, built beside the operational one already proved and joined to it by nothing.

Or read a type as a relation over the semantics the stack already has — the step relation of the metatheory, the one preservation is already stated about. A value belongs to `{v:b|p}` when it belongs at `b` *and* `p` holds of it; recursion is admitted by indexing the relation on the number of steps it is allowed to take. Subtyping soundness then falls out of the one lemma such constructions always turn on. This is the choice, and its recommendation is that it stands *on* the thousand lines already closed instead of beside them.

There is a dividend in that second reading, and it is the reason to write the section down. Recall the finding from V1, the one the affine chapter half-anticipated: a refinement names a value but does not *use* it, because refinements are erased — a mention is not a use. In a relational semantics that fact stops being a rule you enforce and becomes the shape of the proof itself. The refinement is a predicate riding on the relation; the term underneath it is the plain typed term the existing metatheory already handles. The erased program *is* the thing that was already proved safe, and the refinement is a claim laid over it. A decision made at the level of typing, three axes earlier, turns out to be the seam along which the soundness proof wants to be cut. That recurrence — a design choice reappearing later as the structure of its own justification — is close to the most that can be asked of a design, and a reader is owed the sight of it.

None of which says start big. The method of this book has been spine first: a checker before an inference engine, a built tree before a destructed one, the smallest thing that can answer the question before the version that answers it well. The first cut of V3 is the same — refinements on base integers, no measures, no functions passed as values, and a single subtyping-soundness lemma, closed in `lcore` at zero errors. If that spine closes, the mountain is real and has a path; if it fights, the fight is on fifty lines and not five hundred. The richer soundness content — measures as total functions in the logic, and the termination obligation

they drag in that V2 was allowed to wave past — comes after the spine holds, not instead of it.

And when it closes, it will not close everything, and the chapter's own standard forbids pretending otherwise. The proof is against a logic where integers are unbounded, while the RISC-V and C backends compute in thirty-two bits. It is a proof of partial correctness: it says a program that returns an answer returns a right one, not that it returns one. And until the checker is ported and related to the model, the theorem is about the *rules*, with the Python that implements them still trusted to implement them. Three boundaries, all named in the plan and none dissolved by V3. To be plain about which "proven" you have earned is not a footnote to the result; after eighteen chapters spent forcing a tool to be plain about what it has and has not established, it is the result, held to its own rule.

Past that lies the last horizon, and the book stops at its edge on purpose. Some specifications do not fit in any logic a machine decides for free — they need induction, or quantifiers, or an argument no decision procedure was built to find. For those the road is the one Chapter 15 walked: elaborate the Lark term into `lcore` and write the proof by hand, the refinement checker discharging what it can and a person discharging the rest. That is full dependent types in all but name, and the reason to stop short of it here is the reason to have built the refinement checker at all — that the pragmatic thing, the promise a machine can keep without being asked for a proof, is worth reaching first, and reaching well, before reaching further.

Interlude: The Equality Seam

One operator lives in three worlds, and the three do not agree. To the type system, == is polymorphic: any two values of the same type may be compared. To the refinement logic, it is reflexive: `x == x` is valid, because congruence closure *is* the axioms of equality and nothing else. To the machine, it is partial: implemented for integers and floats, an error for a string, a constructor, a tuple, a function. Every seam in this part of the book is a disagreement between two of those worlds, and this interlude walks the pair of seams that taught us the most.

Float: an equality that exists and lies. The first seam is the loud one. A Float has an ==, and it lies: no float equals NaN, itself included, so `x == x` is false at run time for the one value that breaks it. A logic in which `x == x` is unconditionally valid — which congruence closure must be, or it is not the theory of equality — therefore proves a thing the machine will contradict the moment a Float appears. That is the first shape of Chapter 18's false proofs, and the cure is blunt: make Floats *unspeakable* in the logic. No term is ever built at the sort of floats, and so no reflexivity axiom is ever applied to one.

What is worth stopping over is that this single rule is enforced by two different mechanisms, chosen by whether a formula is optional or mandatory at the point the Float turns up. Where the checker merely *might* say something — a Float sitting as a field of a record it is reasoning about — it says nothing, and the silence is sound: the value flows untouched, the surrounding integer obligations are still raised and still caught. Where the checker *must* produce a formula and a Float is all it has, silence is not available, and it refuses outright:

```
refinement error: not a predicate in the decidable
  ↪ fragment: (v > 0.0)
```

for a Float written into a refinement predicate, and

```
refinement error: measure 'bx', arm 'B': not expressible in
  ↪ the predicate language: g
```

for one reached inside a measure. Same fact — Floats have no interior the logic may name — realised as silence in one place and an error in another, both sound, each pinned by its own fixture.

String and function: an equality that does not exist and is believed. The second seam is the quiet one, and it is the first at opposite polarity. The logic grants `s == s` for a string, because it must; withholding reflexivity from a value would make congruence closure unsound. But the machine implements == for integers and floats and nothing else, and on a string it raises `cannot apply '=='` before either branch of a comparison is taken. So a program can be proved and still refuse to run:

```
fn f(s : String, d : Int) : Int =
  if s == s then 0 else div(100, d)
```

VERDICT

```
ok: 2 obligation(s) proved
```

The checker proves the division in the else branch, and it is right to, on its own terms: `s == s` is valid, its negation unsatisfiable, the else branch dead code, and dead code proves anything put to it, division by `d` included. The machine never reaches that division — it raises on the operator first. So there is no division by zero: the narrow promise a proof makes here, that a proved program will not divide by zero or step outside an array, is kept, because the crash preempts the branch. But the proof leaned on an equality the machine does not provide, and this row is *proved* in a strictly weaker sense than every other in the suite. That distance is the seam. For a function the granted equality is not merely unimplemented but undecidable; there is nothing one could wire up.

The fix that was vetoed. There is an obvious way to close the string half: route the runtime == through the value-equality helper the machine already uses to match structures, and `s == s` becomes `true` and the program runs. Tried against the nets first, in the discipline Chapter 18 insists on, it produced a false proof on the bench — the tenth, the one that never shipped. The helper returns `false` for a function, so a branch the checker had proved dead came alive and divided by zero. A sound closing is a rung of its own; it was priced, written down, and parked. A documented seam is a smaller debt than an undocumented false proof.

The moral. Float and String are one seam seen from two sides: an equality that exists and lies, and an equality that does not exist and is believed. What the type system admits, what the logic reasons about, and what the machine implements are three different sets, and == is the operator where they come apart. That the trouble is the operator and not the type is the point of keeping a control alongside — a string guarded through `string_length` discharges a division just as an integer would, and runs, because that path crosses no seam. The boundary is documented, pinned by fixtures, and left standing: a property of the language, shared with the frozen oracle of Part I, not a bug in the checker. A seam you can name and point at is the same thing Chapter 19 closed its ledger on — a promise about where the promises stop.

Conclusion

Three questions, three answers, none of them entirely clean.

Can it build itself? Yes — and the surprise was what the exercise turned into. Rewriting the compiler in its own language was the most searching review the language ever got. Every awkwardness in Lark became a paragraph of the compiler that was harder to write than it should have been; every missing primitive announced itself as a thing the lexer could not say. Self-hosting did not just prove the language powerful enough to describe itself. It made the language argue with its own author, and the language mostly won.

Can it be made faster without changing what it means? Yes, and the cost of insisting on purity was a wall we walked into twice. A functional compiler builds new structure where an imperative one would overwrite old, and twice that habit turned a linear job quietly quadratic — once in the emitter, once in the optimizer's own fixpoint — and both times the fix was the same lesson learned again: an append that copies is not an append. The optimizer runs, it makes real programs smaller and faster, and it computes the same answers as the reference on every input we have. It also carries a quadratic we chose to leave standing and to describe rather than hide, because a book that only showed the parts that came out clean would be teaching the wrong thing.

What can it promise? More than we expected when the book began. Type safety it promises absolutely — a well-typed program does not get stuck, and that is proved about the language once and for all, in a proof a small machine checks. Beyond that, named properties of a program's behaviour: no division by zero, no index out of range, an order read back out of a search tree — checked mechanically, program by program, by a decision procedure that asks nothing of you but the property itself. What it promises, it promises inside boundaries it states aloud rather than papering over. The integers it reasons about are the mathematical ones, not the thirty-two-bit residents of the metal. It proves partial correctness — that the answers a program returns are right, not that it returns one. There is a seam at equality it does not fully close. And the verifier that discharges all of it is itself trusted code, not yet proved correct by the same stan-

dard it holds programs to — which is where the work goes next, and has begun: turning the checker's own logic into something the proof kernel can certify, so that the tool that keeps the promises is held to the promises it keeps.

That leaves the oldest question, the one raised in the first chapter and set aside. Ken Thompson's lecture was about a compiler that hides a backdoor in itself, invisibly, so that reading the source can never find it, because the source is not what compiles you. A self-hosting language is the kind of thing his attack lives in: our compiler's behaviour is inherited from a binary, carried forward through a fixpoint, and no amount of reading the Lark source would reveal a betrayal planted in the machine that compiles it. This book did not make that threat hypothetical — it built the very ladder Thompson described, the compiler reproducing itself to a fixed point, and watched it happen. What defends against it is not more source-reading but *diverse double-compiling*: build the compiler with a second, independent compiler, and a backdoor that survives in one will not survive being rebuilt through the other. We did not do that here. Naming it is the only fair way to close: the same fixpoint that is the triumph of Part I is the hiding place Thompson warned about, and the two are the same fact seen from two sides.

So the language builds itself, runs itself faster, and keeps a widening circle of promises about the programs it runs and about itself — each of those a little more than a programmer starting this book would have thought was theirs to ask for, and each fenced by a boundary named in plain sight. The border into logic that looked, at the start, like a hard one turns out to have no guard on it. You reach it by wanting to say something true about a program, and finding that the machine can be made to check that you are right. That is the distance the book set out to carry you, and if it worked you are standing on the far side of a border you were told was closed.

Chapter A

The Lark Language

A reference, not a tutorial: everything the programs in this book use, in one place. It assumes you have met the language already — the introduction's two-page tour, or the programs as they went by — and want the rules written down.

A.1 Syntax

Lark's syntax is delimiter-based: blocks close with a keyword (`end`), not with indentation, so structure lives in the grammar and layout carries no meaning. The productions below are the core; lexical detail (the spelling of a digit, an identifier's character set) is omitted where it is obvious.

```
program    = module_decl { import_decl } { top_decl }
module_decl = "module" Name
import_decl = "import" Name [ "exposing" "(" name_list ")" ]
top_decl   = [ "export" ] decl

decl       = fn_decl | let_decl | type_decl
           | trait_decl | impl_decl

fn_decl    = "fn" name [ "[" bounds "]" ]
             "(" [ params ] ")" [ ":" type ] "=" expr
let_decl   = "let" name [ ":" type ] "=" expr
type_decl  = "type" Name { name } "=" type_body
trait_decl = "trait" Name { name } "=" "{" { method } "}"
impl_decl  = "impl" Name { atom_type } "for" atom_type
             "=" "{" { method } "}"

expr       = let_expr | if_expr | match_expr
```

```
           | lambda_expr | infix_expr
let_expr   = "let" name [ ":" type ] "=" expr "in" expr
if_expr    = "if" expr "then" expr "else" expr
match_expr = "match" expr "with"
             { "|" pattern "=>" expr } "end"
lambda_expr = "fn" "(" [ params ] ")" "=>" expr
infix_expr = unary { op unary }
apply_expr = atom { "(" [ args ] ")" }

type       = atom_type "->" type | atom_type
atom_type  = Name | Name "(" type { "," type } ")"
           | name | "()" | "(" type ")"
           | "(" type "," type { "," type } ")"
type_body  = type
           | "|" Name [ "of" type ]
             { "|" Name [ "of" type ] }

pattern    = "_" | literal | name
           | Name [ "(" pattern { "," pattern } ")" ]
           | "(" pattern { "," pattern } ")"

op         = "+" | "-" | "*" | "/"
           | "==" | "!=" | "<" | "<=" | ">" | ">="
           | "and" | "or"
```

Two lexical conventions do real work and are worth stating. An identifier that begins with a lowercase letter (`name`) is a variable or function; one that begins with an uppercase letter (`Name`) is a type or a constructor. The distinction is not a style rule, it is how the parser tells `Cons(x)`, a constructor applied, from `f(x)`, a function called, without a symbol table. Comments are `(* … *)` and nest.

A.2 Types

Lark is statically typed with Hindley–Milner inference. You may write a type signature and it is checked; where you omit one, it is reconstructed, and the reconstruction is principal — the most general type the code admits. A function with a full signature may recurse at that signature, including polymorphically, which is a subtlety that cost the type checker a chapter to get right (Chapter 5).

Algebraic data types are declared with `type`, as a choice between named constructors, each optionally carrying fields: `type List a = Nil | Cons of a, List(a)`. A type may take parameters (`a` above), which makes it polymorphic. Values are taken apart by pattern matching, and a `match` is checked for a shape it does not cover.

Affine use is the rule that sets Lark apart. A value may be used *at most once* — referenced, or passed on, but not both — so that the moment a value is consumed the space it occupied can be reclaimed, with no garbage collector and no surprise pauses. All of Part III's ambition rests on this discipline being cheap to enforce, and it is enforced in the type system. The escape is the `Copy` trait: a type that is `Copy` holds values that may be used freely, because copying them is trivial and reclaiming them needs no care. Integers, booleans, floats, and strings are `Copy`; a user type is made `Copy` by an empty `impl`, as `impl Copy for List(Int) = {}`, which the programs in this book do wherever a value is read more than once.

Traits name a set of methods a type can implement, and *bounds* on a function's type parameters (the `[Copy a]` in a signature) require them. In the language as this book uses it, trait bounds are recorded and dispatched at run time rather than resolved statically, which is a simplification worth knowing about and no obstacle to any program here.

A.3 Built-ins

Every primitive the language provides, with its type in Lark notation (`->` associates to the right, so `IO -> String -> IO` is a function of two arguments). These are the only names in scope before any declaration.

primitive	type	does
`print`	`IO -> String -> IO`	write a line
`read`	`IO -> (IO, String)`	read a line
`read_all`	`IO -> (IO, String)`	read all input
`show`	`a -> String`	any value to text
`int_to_float`	`Int -> Float`	widen
`float_to_int`	`Float -> Int`	truncate
`float_to_bits`	`Float -> Int`	raw bits
`int_to_string`	`Int -> String`	
`float_to_string`	`Float -> String`	
`int_abs`	`Int -> Int`	
`float_abs`	`Float -> Float`	
`float_sqrt`	`Float -> Float`	
`float_floor`	`Float -> Float`	
`float_ceil`	`Float -> Float`	
`string_length`	`String -> Int`	
`string_index`	`String -> Int -> Int`	codepoint at
`string_slice`	`String -> Int -> Int -> String`	substring
`char_to_string`	`Int -> String`	codepoint to text
`string_to_int`	`String -> Result(Int, String)`	parse
`string_to_float`	`String -> Result(Float, String)`	parse

`show` is the one primitive that works at every type; it is how a value of any shape becomes printable. The string primitives are the ones self-hosting forced into being — a lexer cannot be written in a language that cannot look inside a string — and their story, and the reason a codepoint is an `Int`, is Appendix C. Indices and codepoints are `Int`; a parse that can fail returns a `Result`, never a thrown error, because Lark does not throw.

A.4 What Lark does not have

The absences are deliberate, and each buys something.

No mutation. There is no assignment and no mutable cell. A "changed" value is a new value, and the old one is untouched — which is what lets the compiler reason about a program by substituting equals for equals, and what the third part of the book depends on.

No exceptions. A computation that can fail says so in its type, by returning a `Result`. There is no invisible second exit from a function, so a type signature is the complete contract, and `string_to_int` handing back `Result(Int, String)` is the language refusing to lie about what it might do.

No modulo. The operators stop at `+ - * /`; there is no `%`. Integer division truncates, and a remainder is written `n - n / d * d` — which is why `divides` in the introduction's program asks its question the way it does. The omission is not principled so much as unpaid-for; it simply never earned its place.

No separately compiled modules. A file is a `module` and may `import` another, but the unit of compilation is the whole program. There is no linking of independently built objects, and so no question of two modules disagreeing about a type across a compilation boundary.

No zero-argument functions. A function takes at least one argument. A computation that takes none is a value — there are no side effects to make its evaluation interesting — and `let` is how you name a value. In a language where nothing happens on the side, a function of no arguments and the constant it would return are the same thing, so the language keeps only the constant.

Chapter B

The Method: Differential Harnesses

Every claim in this book is checked by a program. This appendix is how to run them, how to read what they print, and how to tell a real regression from a run that only looks like one.

B.1 The harnesses

The first two parts test by *comparison*. Each strand ships two implementations of the same language — a Python oracle and the Lark port — and a harness that feeds both the same inputs and demands the same answer. One harness rides on each compiler stage, so a disagreement is reported at the stage that caused it rather than at the end.

In `self/harness/` the differentials run down the pipeline: `lex_difftest.py` compares token streams, `parse_difftest.py` syntax trees, `infer_difftest.py` inferred types, `typecheck_difftest.py` the accept/reject verdict on the error suite, `cek_difftest.py` the value a program evaluates to, and `emit_c_difftest.py` the C the back end writes. Two more are not differentials but self-checks: `bootstrap.py` compiles the compiler with itself and `bootstrap_tc.py` does the same through the type checker. Each has a make target, and because every one runs an interpreter inside an interpreter they are slow; run them one at a time:

```
make lextest        # token streams agree
make infertest      # inferred types agree
make cektest        # evaluated values agree
make test           # all of the above, in order
```

```
make bootstrap      # the compiler compiles itself
```

`optimize/harness/` has the same shape one stage lower, where the optimizer lives. `lower_difftest.py` checks the lowering to the three-address form, `opt_difftest.py` checks that an optimized program computes what the un-optimized one did, `optcompiler_difftest.py` checks the optimizer built by the self-hosted compiler against the one built by the oracle, and `emit_tac_c_difftest.py` checks the emitted C. The optimizer's own question — *did meaning survive* — is `opt_difftest.py`'s to answer, input by input.

Part III's harnesses are a different genus, and they live in `prove/harness/`. A verifier does not claim two implementations agree; it claims something about every future run of one program, and no second implementation can referee that. So the method turns adversarial. Three harnesses ship:

- `prove_difftest.py` sweeps the seventy-five fixtures — each safe program beside the mutant that should fail — and checks that the verifier's verdict on every one matches the verdict pinned for it, then demonstrates that a proved refinement erases and leaves the runtime untouched.
- `adversary.py` generates hostile programs on purpose, has the checker rule on them, and then *runs* what it claimed to prove, letting the world overrule the proof. It carries independent oracles — among them a transcription of each measure into Python, computed a second way — so that a proof and its refutation come from different code.
- `solver_fuzz.py` throws random formulas at the from-scratch decision procedure and checks its yes/no against a brute-force reference, so that the part of the verifier nobody can eyeball is exercised where it cannot hide.

```
make prove          # the seventy-five fixtures, verdict by
  ↪ verdict
make adversary      # generate, prove, and run the
  ↪ refutation
make solver         # fuzz the decision procedure against
  ↪ brute force
```

One discipline governs all three and deserves stating on its own, because it is the reason to believe a green count at all. It is the *watched-to-fail* rule: no check is trusted until it has been seen to fail. After every bug the work fixed, the bug was re-planted and the harness watched to catch it before the fix was trusted again; a green that has never once gone red is a green nobody has tested. Nine

false proofs were found over the course of Part III, and each became a fixture that still fails on demand.

B.2 Reading the numbers

A differential prints a line of the form `N ok / M fail / K skip`, and the three counts are read differently.

`fail` is the only one that is ever a bug. A failure means both implementations finished a program and returned different answers — a disagreement about meaning, which is a fact about the compiler and holds on any machine. The contract everywhere in this book is `0 fail`. Anything else is a defect until proved otherwise.

`skip` is not a failure and not swept under the rug. A file is skipped when there is no meaningful comparison to make: the oracle itself rejects it (the error suite is meant to be rejected), or it is an import fragment with no `main` to run, or — for the meta-circular harnesses — this machine could not finish it inside the wall-clock budget. Every skip is one of those, and the baseline file lists which skips are which. A skip that changes its reason is worth investigating; a skip that is a slow file on a slow box is not.

`ok` is a pair that ran and agreed. The count of `ok` can move between machines — a faster box turns a timed-out skip into an `ok` — without any claim about the compiler changing, because what is promised is not the size of `ok` but the emptiness of `fail`.

B.3 The baselines

Each strand pins its numbers in a `BASELINES.md` — `self/harness/BASELINES.md` and `optimize/harness/BASELINES.md`. A baseline is the count a clean run is supposed to print, and its purpose is to make a regression loud: if a run disagrees with the file, either the code changed or the harness did, and you find out which before you touch the file.

The baseline draws a line its own prose insists on, because it is the line between a claim and a measurement. Some numbers are *invariant* — true on any machine. `0 fail` everywhere is one; the accept/reject verdicts of the error suite are another; the fixpoint's content hash is a third, because the emitted C is deterministic and a different box reproduces the same bytes. Other numbers are *environment-dependent* — the wall-clock budget, and the `ok`/`skip` split that falls out of it. These are budgets, not promises, and the baseline says so of each. A change that moves an invariant number is a regression to explain; a change that moves a budget number moved the box, not the compiler.

B.4 Reproducing the fixpoint

The self-hosting claim comes down to a single reproducible event. In `self/` the compiler compiles itself:

```
make bootstrap
```

and in `optimize/` the optimizing compiler compiles itself, and then compiles the result, and then compiles that:

```
make fixpoint
```

The three emitted C files are compared. They are identical — the same content hash, `8f9596d9`, for the first stage and every stage after it: $C_1 = C_2 = C_3$. That equality is the fixpoint. A compiler that reproduces its own output byte for byte has reached the place where compiling it again changes nothing, and the number that proves it is a hash any reader can regenerate on their own machine and check against the one printed here. It is the shortest claim in the book and the one Chapter 2 spends the longest earning.

Chapter C

What We Changed in the Oracle

Every green count in this book is a claim of one shape: *the port cannot be told apart from the reference*. Every such claim is empty if the reference moves and nobody writes down how. You can always reach agreement by walking toward each other.

The reference moved. This appendix is the ledger, and it exists because for most of the project it did not — we would have told you, in good faith, that the oracle had been thawed three times, and we would have been wrong. The number is kept in the repository now, and it is regenerated from the version history rather than from anyone's memory, because a ledger you maintain by remembering is a ledger about your memory.

Since the port began, the Python oracle has taken roughly three and a half thousand added lines across thirteen files. Most of that is harmless. The point of sorting it is that "harmless" is a claim too, and it needs saying which kind.

Additions: new files

Three files are new — the optimizing passes, the second code generator, and a register allocator. They are a *second axis* (Part II) built beside the checking path rather than through it. Nothing that any earlier baseline pinned could move, because none of this code existed when those baselines were cut and nothing on the old path calls into it. This is the cheap kind of change, and it is the only kind that needs no defence.

Growth: the language got bigger because the port needed it

Eight primitives were added to Lark during the port, and every one of them was added because the compiler could not otherwise be written:

primitive	added for
`string_length`	measuring how long the source is
`string_index`	indexing the source text
`string_slice`	taking a token out of it
`char_to_string`	building a string back up
`string_to_int`	scanning an integer literal
`string_to_float`	scanning a float literal
`read_all`	reading a whole program from standard input
`float_to_bits`	emitting a float constant bit-exactly

When the port began, Lark could compare strings and concatenate them, and that was all. It could not look inside one. You cannot write a lexer in a language that cannot index a string, and so the first milestone of the entire project (Chapter 2) was not a line of the compiler — it was making the language capable of having a compiler written in it.

This is the price of self-hosting, and the reason it is worth putting in an appendix rather than a footnote is that **it is paid in the language, not in the compiler, and it is almost never quoted**. Every self-hosting claim you have ever read has some version of this table behind it, and most of them do not show it to you. Ours grew by eight. A language that had been designed for self-hosting from the start would have had them already, and would have been a different language, and the exercise would have proved less.

None of these changed an answer the oracle already gave. They extended what it could be asked. That is a real distinction and it is doing real work here — but it is not the same as "the oracle was frozen", and we should not have implied that it was.

Corrections: the oracle was wrong

Two, and they are the ones that matter, because each of them re-opened — in principle — every fixpoint and every pinned count in this book, and each had to be gated on all of them coming back unchanged.

The first is the string/codepoint confusion of Chapter 2: the oracle read characters out of strings as Unicode codepoints and wrote them back as bytes, so the two operations stopped being inverses above `U+007F`. The port was right; Python was changed to match.

The second is the subject of Chapter 5, and it is the better story: a function with a complete type signature could not recurse polymorphically, because the checker computed its type from the signature and then threw that answer away. The symptom was not a rejection but a non-terminating type check. It

was worked around three times, in three different modules, before anybody understood what it was.

A third entry belongs here for honesty's sake, though it is a test and not the compiler. The expected output of the one test that covers the eight primitives above is embedded in the test file as a comment. Somebody extended the test and did not grow the comment, so the suite reported one failure for weeks, the failure was never real, and everyone learned to scroll past it. An expectation that has stopped covering the program has stopped being a claim about it — which is the same disease as a stale baseline, one level further down, and it is worth knowing that it can happen to you at every level at once.

Why this matters more than it looks

The oracle is not a fixed point. It is a *tracked* point — and that is the best that is actually available, and it is fine, *so long as the tracking exists*. What is not fine, and what we did for several months, is to believe in the freeze while quietly editing the thing that was frozen.

The reason to be strict about this is that it does not end with the port. Part II argues that an optimization is correct when it does not change what the program says, and every one of those arguments is made against the same reference. Anything built after that — a proof system, a verified pass, whatever comes next — inherits the same dependency. If the reference drifts in silence, then what you have proved is that two things you were adjusting toward each other eventually met, which is not a theorem about compilers. It is a theorem about you.

Chapter D

A Map of the Code

The companion repository is three independent trees, one per part. This appendix says what is in each and in what order to read it. Every tree has the same five or six top-level names, so learning one is learning all three:

name	what it holds
`oracle/`	the slow, readable Python reference
`lark/`	the subject of the part, written in Lark
`samples/`	the programs the harnesses run
`harness/`	the differentials that hold one against the other
`tests/`	the corpus the harnesses draw on
`Makefile`	the commands, with `make help` listing them
`README.md`	the entry point, and where to start reading

Start each tree at its `README.md`, run `make help` to see its targets, and read `oracle/` before `lark/`, because the Python says plainly what the Lark then says in the language under test.

D.1 `self/`

The self-hosting strand answers Part I's question — can a language build itself? Its `oracle/` is the reference compiler in Python: `lexer.py`, `parser.py`, `infer.py`, the type representation `ty.py`, the evaluator `cek.py`, and the C emitter `emit_c_ast.py`, together with a C runtime the emitted code links against. Its `lark/` is the same compiler rewritten in Lark, file for file: `lex.lark`, `parse.lark`, `infer.lark`, `types.lark`, `cek.lark`, `emit_c.lark`. The nine programs in `samples/` are what both compilers are made to agree on, and `harness/` holds the per-stage differentials and the two bootstrap scripts. This is the tree to read first, because the other two assume the compiler it builds.

D.2 `optimize/`

The optimization strand answers Part II's question — can the compiler be made fast without changing what a program means? It stands entirely on its own, with its own `oracle/` and its own Lark `lark/`, both carrying machinery the self-hosting strand has no need of: a three-address intermediate form (`tac.lark`), a lowering onto it (`lower.lark`), the optimizer itself (`opt.lark`), and a back end that emits C from the lowered form (`emit_tac_c.lark`). The oracle mirrors each of these in Python. Its `harness/` is where `opt_difftest.py` lives — the check that an optimized program computes what the un-optimized one did — and where the fixpoint is reproduced.

D.3 `prove/`

The proof strand answers Part III's question — what can the language actually promise? — and it is shaped differently from the other two, because its subject is a verifier rather than a compiler. Its `oracle/` is the checker, grown from the type inferencer into one that discharges refinement obligations. Its `fixtures/` are the seventy-five refinement programs, each safe program paired with the mutant that should fail. Its `harness/` holds the three adversarial programs of Appendix B. And it carries a second body of work the others do not: `lark-formal/`, the machine-checked metatheory — the type system's soundness written as a proof and checked by `lcore/`, the small proof kernel that certifies it. This is the tree where the language stops testing itself and starts proving things, about programs and then about itself.

D.4 Deliberate duplication

The same lexer appears in `self/` and `optimize/`. The same oracle files recur across the trees. This repetition is on purpose, and it is worth saying why, because a software engineer's first instinct is to factor it out.

The trees are not modules of one system; they are three artifacts, each meant to be opened, read, and run on its own, by a reader who may never look at the other two. A reader who came only for the optimizer should not have to clone the self-hosting strand to get a lexer, and should not have to hold two directories in their head to follow one differential. Sharing the lexer through a common library would make the repository smaller and every tree harder to read alone. The book chose the reader over the byte count. Nothing here is a shared dependency: each tree is a complete thing, and the cost of that — a lexer written down more than once — is a cost the book pays on purpose.

Bibliography

[Aho et al., 2006] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition. The Dragon Book.

[Damas and Milner, 1982] Damas, L. and Milner, R. (1982). Principal type-schemes for functional programs. In *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 207–212, New York, NY. ACM.

[Hindley, 1969] Hindley, R. (1969). The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146:29–60.

[McKeeman, 1998] McKeeman, W. M. (1998). Differential testing for software. *Digital Technical Journal*, 10(1):100–107.

[Milner, 1978] Milner, R. (1978). A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375.

[Nystrom, 2021] Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning. Also freely available at `https://craftinginterpreters.com`.

[Pierce, 2002] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.

[Thompson, 1984] Thompson, K. (1984). Reflections on trusting trust. *Communications of the ACM*, 27(8):761–763.

[Tov and Pucella, 2011] Tov, J. A. and Pucella, R. (2011). Practical affine types. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 447–458. ACM.

[Wadler, 1990] Wadler, P. (1990). Linear types can change the world! In Broy, M. and Jones, C. B., editors, *Programming Concepts and Methods*. North-Holland.

Index